




UNIVERSITÉ DE FIANARANTSOA

ÉCOLE NATIONALE D'INFORMATIQUE

MÉMOIRE DE FIN D'ÉTUDES POUR L'OBTENTION DU DIPLÔME DE MASTER PROFESSIONNEL

Mention : Intelligence Artificielle

Parcours : Gouvernance et Ingénierie des Données

Intitulé :

CONCEPTION ET DÉVELOPPEMENT D'UN SYSTÈME INTELLIGENT DE GÉNÉRATION AUTOMATIQUE DES ÉCRITURES COMPTABLES À PARTIR DES FACTURES COMMERCIALES (UTILISANT L'OCR, LE NLP ET LE MACHINE LEARNING)

Présenté le : 11 novembre 2026

Par : Monsieur FANEVA Solomampionona

Membres du Jury :

Président : Monsieur
Examinateur : Monsieur
Rapporteurs : Monsieur RALAIVAO Jean Christian, Maître de Conférences
Mademoiselle MALALATIANA Marie Philippine, Directeur Adjoint

Année Universitaire 2025-2026

CURRICULUM VITAE



Nom : FANEVA

Prénom : Solomampionona

Date et lieu de naissance : 1er septembre 2004 a Fianarantsoa

Nationalité : Malagasy

Téléphone : 034 16 843 71

Email : fanevasolomampiona@gmail.com

LinkedIn : linkedin.com/in/faneva-solomampionona-6166/

FORMATIONS ET DIPLÔMES OBTENUS :

2025-2026 : Master II en Intelligence Artificielle (en cours)

Parcours : Gouvernance et Ingénierie de données

École Nationale d'Informatique

2024-2025 : Master I en Intelligence Artificielle

Parcours : Gouvernance et Ingénierie de données

École Nationale d'Informatique

2023-2024 : Licence en Informatique Appliquée

Parcours : Services informatiques aux organisations

École Supérieure de Management et d'Informatique Appliquée

2020-2021 : Baccalauréat série D mention Assez-Bien

Saint François Xavier-Fianarantsoa

STAGES, EXPÉRIENCES PROFESSIONNELLES ET PROJETS ACADÉMIQUES

2025-aujourd'hui : Développeur full-stack, Univers Ventes

Société MANAO Consulting

Mai 2024-Aout 2024 : Stage en développement web

Caisse d'épargne Madagascar Fianarantsoa

Création d'une application web capable de suivre en temps réel le trafic du crédit dans l'organisation.

Juin 2023-Aout 2023 : Stage en développement web

BIONNEX

Développement d'une application web pour la gestion de situation de demande d'achat.

Avril 2026-Octobre 2026 : Projet académique de fin d'études (Master II)

TETIKA SARL – groupe Manao

Conception et développement d'un système intelligent de génération automatique des écritures comptables (OCR, NLP, Machine Learning).

COMPÉTENCES EN INFORMATIQUE

-Maîtrise de Java, Python, JavaScript, C#, PHP

-**Développement web** : HTML, CSS, JavaScript, React.js, Vue.js

-Maîtrise de **Framework et bibliothèque** : Laravel, Next.js, Flask, Symfony, Three.js, Node.js

-**Base de données** : SQL (MySQL, PostgreSQL), NoSQL (MongoDB)

-**Méthodologies de développement** : Agile, Scrum

-UI/UX Design : Prototypage, Design d'interfaces, Outils comme Figma

-Intelligence artificielle : Machine Learning (scikit-learn, XGBoost), OCR (EasyOCR, Tesseract), NLP (spaCy)

-Systèmes d'exploitation : Windows 10/11, Linux (Ubuntu)

-Outils de versionnage et de déploiement : Git, GitHub, Docker

CONNAISSANCES LINGUISTIQUES

Aptitude	**Comprendre à l'audition**					**lire**					**Écrire et rédiger**					**Communiquer oralement**				
	TB	**B**	**AB**	**P**	**NS**	**TB**	**B**	**AB**	**P**	**NS**	**TB**	**B**	**AB**	**P**	**NS**	**TB**	**B**	**AB**	**P**	**NS**
français	**X**					**X**						**X**				**X**				
Anglais			**X**				**X**						**X**						**X**	

GRILLE D'ÉVALUATION

TB :Très bien

B :Bien

AB :Assez-bien

P :Passable

NS :Niveau Scolaire

SOMMAIRE

CURRICULUM VITAE i
SOMMAIRE iii
REMERCIEMENTS v
LISTE DES FIGURES vi
LISTE DES TABLEAUX vii
LISTE DES ABRÉVIATIONS viii
INTRODUCTION GÉNÉRALE 1
PARTIE I. PRÉSENTATIONS 3
CHAPITRE 1 Présentation de l'Ecole Nationale d'Informatique 3
CHAPITRE 2 : Présentation de l'établissement d'accueil 12
2.1 Présentation générale du groupe Manao 12
2.2 Historique 12
2.3 Philosophie et valeurs 13
2.4 Domaines d'expertise 13
2.5 Branches et activités 14
2.6 Implantations 14
2.7 La plateforme logicielle Manao 14
2.8 Organisation et organigramme 15
2.9 Place du stagiaire et lien avec le projet 16
CHAPITRE 3 : Description du projet 17
3.1 Formulation : 17
3.2 Objectif et besoin de l'utilisateur : 18
3.3 Moyens nécessaires à la réalisation du projet : 19
3.4 Résultats Attendus 22
3.5 Chronogramme : 23
PARTIE II. ANALYSE ET CONCEPTION 25
CHAPITRE 4 : Analyse préalable 25
4.1 Analyse de l'existant 25
4.2 Critique de l'existant 32
4.3 Conception avant-projet 34
CHAPITRE 5 : Analyse conceptuelle 37
5.1 Désignation des rôles de l'équipe SCRUM 37
5.2 Étapes de l'élaboration du Product Backlog 38
5.3 Product Backlog 39
5.4 Sprint Backlog 40

5.5 Dictionnaire des données 42
5.6 Règles de gestion 43
5.7 Diagramme de cas d'utilisation global 44
5.8 Modèle du domaine : 46
5.9 Diagramme de classe de conception pour chaque Sprint : 48
5.10 Diagramme de classe de conception globale : 55
5.11 Diagramme de paquetage : 58
5.12 Diagramme de déploiement : 58
CHAPITRE 6 : Intégration IA dans la conception 61
6.1 Définition du module IA 61
6.2 Données nécessaires au fonctionnement du module IA 61
6.3 Choix du modèle IA et justification 63
6.4 Méthode d'intégration de l'IA 64
PARTIE III. RÉALISATION 68
CHAPITRE 7 : Mise en place de l'environnement de développement 68
7.1 Installation et configuration des outils 68
7.2 Architecture de l'application 69
7.3 Déploiement des environnements (développement, test, production) 70
CHAPITRE 8 : Développement de l'application 73
8.1 Création et structuration de la base de données 73
8.2 Développement du frontend 73
8.3 Développement du backend 76
8.4 Développement et entraînement du module IA 77
8.5 Intégration IA–application 78
CHAPITRE 9 : Tests et validation 80
9.1 Tests unitaires et d'intégration 80
9.2 Validation fonctionnelle 80
9.3 Évaluation de la performance du module IA 82
CONCLUSION 83
BIBLIOGRAPHIE x
WEBOGRAPHIE xi
GLOSSAIRE xii
ANNEXE xiv

REMERCIEMENTS

Je tiens à exprimer ma profonde gratitude envers le Tout-Puissant, source d'inspiration et guide de mes facultés intellectuelles tout au long de l'élaboration de ce mémoire.

Mes sincères remerciements vont au Pr. HAJALALAINA Aimé Richard, Président de l'Université de Fianarantsoa, pour son engagement envers la qualité de la formation dispensée aux étudiants de l'université.

Je suis également reconnaissant envers le Pr. MAHATODY Thomas, Directeur de l'École Nationale d'Informatique, pour sa direction éclairée et son soutien continu.

Je remercie aussi le Chef de Mention Intelligence Artificielle ainsi que le Responsable du parcours Gouvernance et Ingénierie des Données, pour la qualité de la formation et de l'encadrement pédagogique dont j'ai bénéficié.

Un remerciement particulier est adressé à mon encadreur pédagogique, le Dr. RALAIVAO Jean Christian, Maître de Conférences, dont la guidance experte, les conseils avisés et le soutien constant ont été déterminants pour la réussite de ce travail. J'apprécie profondément l'opportunité d'avoir bénéficié de son encadrement éclairé.

J'adresse également mes vifs remerciements à mon encadreur professionnel, Mlle MALALATIANA Marie Philippine, Directeur Adjoint au sein du groupe Manao, pour le temps consacré à l'explication du cahier des charges et pour son accompagnement tout au long du stage.

Mes remerciements vont aussi à l'ensemble du personnel du groupe Manao, mon établissement d'accueil, pour son accueil chaleureux et pour les conditions de travail mises à ma disposition.

Enfin, j'exprime ma reconnaissance envers mes parents, ma famille et mes collègues, dont le soutien indéfectible a été un moteur essentiel tout au long de mes études. Cet ouvrage reflète l'aboutissement de ces efforts, et je tiens à leur témoigner ma sincère gratitude à travers ces lignes.

LISTE DES FIGURES

Figure 1: Organigramme de l'Ecole Nationale d'Informatique 6
Figure 2: Architecture des études correspondant au système LMD 7
Figure 3: Logo du groupe Manao 12
Figure 4: Organigramme de la direction du développement du groupe Manao 16
Figure 5: Diagramme de Gantt du projet 24
Figure 6: Workflow manuel de traitement d'une facture 28
Figure 7: Diagramme de cas d'utilisation global 46
Figure 8: Modèle du domaine 48
Figure 9: Diagramme de classes de conception – Sprint 1 49
Figure 10: Diagramme de classes de conception – Sprint 2 50
Figure 11: Diagramme de classes de conception – Sprint 3 51
Figure 12: Diagramme de classes de conception – Sprint 4 52
Figure 13: Diagramme de classes de conception – Sprint 5 53
Figure 14: Diagramme de classes de conception – Sprint 6 54
Figure 15: Diagramme de classes de conception – Sprint 7 55
Figure 16: Diagramme de classes de conception globale 57
Figure 17: Diagramme de paquetage 58
Figure 18: Diagramme de déploiement 60
Figure 19: Processus d'intégration de l'IA dans le système (diagramme d'activité) 66
Figure 20: Architecture d'intégration du module IA 67
Figure 21: Architecture en couches de l'application (adaptée de la Clean Architecture) 70
Figure 22: page DashboardPage — vue d'ensemble des factures et indicateurs 75
Figure 23: page UploadPage — dépôt d'un PDF ou d'une image de facture 75
Figure 24: page InvoiceDetailPage — résultat OCR, entités extraites et écriture proposée avec score de confiance 76
Figure 25: page EntriesPage — correction et validation d'une écriture comptable 76
Figure 26: page AdminPage — gestion des utilisateurs et réentraînement du modèle 76
Figure 27: Pipeline de traitement automatisé d'une facture, de l'import à la validation comptable 79
Figure 28: Déroulement d'un scénario de validation fonctionnelle complet 82

LISTE DES TABLEAUX

Tableau 1: Organisation du système de formation pédagogique de l'Ecole 6
Tableau 2: Mention et parcours au sein de l'ENI 7
Tableau 3: Liste des formations existantes à l'ENI 8
Tableau 4: Débouchés éventuels des jeunes diplômés 9
Tableau 5: Historique du groupe Manao 12
Tableau 6: Branches du groupe Manao 14
Tableau 7: Univers de la plateforme logicielle Manao 14
Tableau 8: Ressources matérielles nécessaires au projet 20
Tableau 9: Outils et logiciels du projet 20
Tableau 10: Technologies et composants du système 21
Tableau 11: Récapitulation des étapes du processus traditionnel 26
Tableau 12: Moyens matériels utilisés pour le projet 30
Tableau 13: Logiciels utilisés et versions 30
Tableau 14: Comparaison des solutions envisagées 35
Tableau 15: Choix et justification des outils 36
Tableau 16: Product Backlog 39
Tableau 17: Présentation du travail à faire (releases et sprints) 40
Tableau 18: Planification du sprint (tâches par sprint) 41
Tableau 19: Dictionnaire des données 42
Tableau 20: Comparaison des approches et des modèles 63
Tableau 21: Indicateurs d'évaluation du module IA 82

LISTE DES ABRÉVIATIONS

API	Application Programming Interface (Interface de Programmation Applicative)
CPU	Central Processing Unit (unité centrale de traitement)
CSS	Cascading Style Sheets (feuilles de style en cascade)
ENI	École Nationale d'Informatique
GID	Gouvernance et Ingénierie des Données
HT	Hors Taxes
HTML	HyperText Markup Language
HTTPS	HyperText Transfer Protocol Secure (protocole de transfert hypertexte sécurisé)
IA	Intelligence Artificielle
JSON	JavaScript Object Notation
JWT	JSON Web Token
LMD	Licence – Master – Doctorat
MGA	Ariary malgache (code de devise)
ML	Machine Learning (Apprentissage Automatique)
NER	Named Entity Recognition (Reconnaissance d'Entités Nommées)
NIF	Numéro d'Identification Fiscale
NLP	Natural Language Processing (Traitement du Langage Naturel)
OCR	Optical Character Recognition (Reconnaissance Optique de Caractères)
ORM	Object-Relational Mapping
PCG	Plan Comptable Général
PDF	Portable Document Format
RBAC	Role-Based Access Control (Contrôle d'Accès Basé sur les Rôles)
REST	Representational State Transfer
SGBD	Système de Gestion de Base de Données
SQL	Structured Query Language
TF-IDF	Term Frequency – Inverse Document Frequency
TTC	Toutes Taxes Comprises
TVA	Taxe sur la Valeur Ajoutée
UI/UX	User Interface / User Experience (interface et expérience utilisateur)
UML	Unified Modeling Language (langage de modélisation unifié)
VPS	Virtual Private Server (serveur privé virtuel)

INTRODUCTION GÉNÉRALE

À l'ère de la transformation numérique, les entreprises sont confrontées à une croissance constante du volume des données administratives et financières qu'elles doivent traiter quotidiennement. Parmi ces données, les factures commerciales occupent une place centrale dans la gestion comptable et financière. Le traitement manuel de ces documents implique plusieurs tâches répétitives telles que la lecture des informations, la vérification des montants, l'identification des comptes comptables appropriés et la saisie des écritures comptables. Cette approche traditionnelle est souvent chronophage, coûteuse et susceptible de générer des erreurs humaines.

Face à ces défis, l'intelligence artificielle offre aujourd'hui des solutions innovantes permettant d'automatiser et d'optimiser les processus de gestion documentaire. Grâce aux avancées réalisées dans les domaines de la reconnaissance optique de caractères, du traitement automatique du langage naturel et de l'apprentissage automatique, il devient possible d'extraire automatiquement les informations contenues dans les factures et de les exploiter pour produire des résultats pertinents dans un contexte comptable.

Dans de nombreuses organisations, notamment dans les petites et moyennes entreprises, la génération des écritures comptables à partir des factures demeure encore largement manuelle. Cette situation entraîne non seulement une perte de temps importante, mais également un risque accru d'erreurs pouvant affecter la qualité des informations financières produites. Dès lors, l'automatisation intelligente de ce processus représente un enjeu majeur pour améliorer la productivité, la fiabilité et la rapidité du traitement comptable.

C'est dans ce contexte que s'inscrit le présent mémoire intitulé : « Conception et développement d'un système intelligent de génération automatique des écritures comptables à partir des factures commerciales utilisant l'OCR, le NLP et le Machine Learning ».

L'objectif principal de ce travail est de concevoir et de développer une solution capable de lire automatiquement une facture commerciale, d'en extraire les informations essentielles, d'interpréter son contenu et de générer les écritures comptables correspondantes selon les règles comptables préétablies. Le système proposé vise ainsi à réduire les interventions manuelles tout en améliorant la précision et l'efficacité du processus comptable.

Pour atteindre cet objectif, plusieurs questions de recherche se posent :

- Comment extraire automatiquement les données pertinentes contenues dans une facture commerciale ?
- Comment interpréter ces informations afin d'identifier les comptes comptables appropriés ?
- Comment utiliser les techniques d'intelligence artificielle pour automatiser la génération des écritures comptables ?
- Dans quelle mesure cette automatisation peut-elle améliorer la performance du traitement comptable ?

Afin d'apporter des réponses à ces interrogations, ce mémoire est structuré en trois parties. La première partie présente l'École Nationale d'Informatique, l'établissement d'accueil et le projet. La deuxième partie est consacrée à l'analyse de l'existant, à la conception du système selon la méthode SCRUM et à l'intégration de l'intelligence artificielle dans cette conception. Enfin, la troisième partie détaille la réalisation : la mise en place de l'environnement de développement, le développement de l'application et du module d'intelligence artificielle, puis les tests et la validation de la solution.

PARTIE I. PRÉSENTATIONS

CHAPITRE 1 Présentation de l'Ecole Nationale d'Informatique

L'École Nationale d'Informatique (ENI) est reconnue comme un établissement de référence à Madagascar dans le domaine de la formation en informatique.

Information d'ordre générale

L'Ecole Nationale d'Informatique, en abrégé ENI, est un établissement d'enseignement supérieur rattaché académiquement et administrativement à l'Université de Fianarantsoa. Le siège de l'Ecole se trouve à Tanambao-Antaninarenina à Fianarantsoa. L'adresse pour la prise de contact avec l'Ecole est : Ecole Nationale d'Informatique (ENI) Tanambao, Fianarantsoa. Le numéro de sa boîte postale est 1487 avec le code postal 301. Téléphone : 038 96 566 96 ou 034 76 591 95. Son adresse électronique est **eni@eni.mg**. Il dispose également d'un site web : **www.eni.mg**

Missions et historiques

L'ENI se positionne sur l'échiquier socio-éducatif malgache comme étant le plus puissant secteur de diffusion et de vulgarisation des connaissances et des technologies informatiques.

Cette Ecole Supérieure peut être considérée aujourd'hui comme la vitrine et la pépinière des élites informaticiennes du pays.

De façon formelle, l'ENI était créée par le décret N° 83- 185 du 24 Mai 1983, comme étant le seul établissement Universitaire Professionnalisé au niveau national, destiné à former des techniciens et des Ingénieurs de haut niveau, aptes à répondre aux besoins et exigences d'Informatisation des entreprises, des sociétés et des organes implantés à Madagascar.

L'ENI a pour conséquent pour mission de former des spécialistes informaticiens compétents et opérationnels de différents niveaux notamment :

- En fournissant à des étudiants des connaissances de base en informatique ;
- En leur transmettant le savoir-faire requis, à travers la professionnalisation des formations dispensées et en essayant une meilleure adéquation des formations par rapport aux besoins évolutifs des sociétés et des entreprises ;
- En initiant les étudiants aux activités de recherche dans les différents domaines des Technologies de l'Information et de la Communication (TIC) ;

La filière de formation d'Analystes Programmeurs a été mise en place à l'Ecole en 1983, et a été gelée par la suite en 1996, tandis que la filière de formation d'ingénieurs a été ouverte à l'Ecole en 1986.

Une formation de troisième cycle a été ouverte à l'Ecole depuis l'année 2003 – 2004 grâce à la coopération académique et scientifique entre l'Université de Fianarantsoa pour le compte de l'ENI et l'Université Paul Sabatier de Toulouse (UPST).

Cette filière avait pour objectif de former certains étudiants à la recherche dans les différents domaines de l'Informatique, et notamment pour préparer la relève des Enseignants-Chercheurs qui étaient en poste.

Pendant l'année 2007-2008, la formation en vue de l'obtention du diplôme de Licence Professionnelle en Informatique a été mise en place à l'ENI avec les deux parcours de formation:

- Génie Logiciel et base de Données.
- Administration des Système et réseaux.

La mise en place à l'Ecole de ces deux options de formation devait répondre au besoin de basculement vers le système Licence – Master – Doctorat (LMD).

En vue de surmonter les difficultés de limitation de l'effectif des étudiants accueillis à l'Ecole, notamment à cause du manque d'infrastructures, un système de « Formation Hybride » a été mise en place à partir de l'année 2010. Il s'agit en effet d'un système de formation semi présentielle et à distance avec l'utilisation de la visioconférence pour la formation à distance. Le système de formation hybride a été ainsi créé à Fianarantsoa ainsi qu'Université de Toliara. Cette formation est à l'origine du parcours Informatique Générale.

En 2023, la mention Intelligence Artificielle (IA) a été ouvert au sein de l'Ecole pour répondre les besoins des entreprises. La formation est destinée aux étudiants titulaires du diplôme de licence (Bac +3) en Mathématiques ou en Statistiques ou en Informatique, etc. La mention IA comporte deux parcours :

- Gouvernance et Ingénierie de Données (GID),
- Objets connectés et Cybersécurité (OCC).

Le principe de l'enseignement pour le parcours GID offre aux l'étudiants des compétences scientifiques et techniques spécialisées en Science de données. Pour le parcours OCC, les étudiants octroient la double spécialité premièrement en internet des objets et deuxièmement en cybersécurité. La formation de master est axée sur l'ensemble d'applications de l'Intelligence Artificielle.

Depuis l'année universitaire 2024-2025, la mention Expertise Digitale est officiellement ouverte au grand public. Cette mention propose deux (2) parcours distincts :

- Métiers du Digital (MDi) – disponible aux niveaux Licence et Master. Ce parcours est destiné aux étudiants titulaires du diplôme de Baccalauréat, toutes séries confondues. Il est élaboré pour équiper les étudiants des compétences essentielles dans les domaines du marketing digital, de la communication digitale, et de la gestion de projets digitaux.
- Audit des Systèmes d'Information (ASI) – proposé au niveau Master uniquement. Ce parcours s'adresse aux étudiants titulaires d'une licence (Bac +3) en Mathématiques, en Statistiques ou en Informatique. Il est structuré pour approfondir les connaissances théoriques et pratiques nécessaires à l'audit efficace des systèmes d'information.

Le système d'enseignement adopté pour cette mention est un système en ligne. Ainsi, les cours et ressources pédagogiques sont accessibles en permanence via la plateforme numérique Moodle. Par ailleurs, des regroupements en présentiel ou en ligne seront organisés périodiquement, conformément au planning défini par les responsables pédagogiques.

Organigramme institutionnel

L'organigramme de l'Ecole est inspiré des dispositions du décret N° 83-185 du 24 Mai 1983. L'ENI est administrée par un Conseil d'Ecole, et dirigée par un directeur nommé par un décret adopté en Conseil des Ministres. Le Collège des enseignants regroupant tous les enseignants-chercheurs permanents de l'Ecole est chargé de résoudre les problèmes liés à l'organisation pédagogique des enseignements. Le Conseil Scientifique propose les orientations pédagogiques et scientifiques de l'établissement, en tenant compte notamment de l'évolution du marché de travail et de l'adéquation des formations dispensées par rapport aux besoins des entreprises.

La figure 1 représente l'organigramme actuel de l'Ecole Nationale d'Informatique.

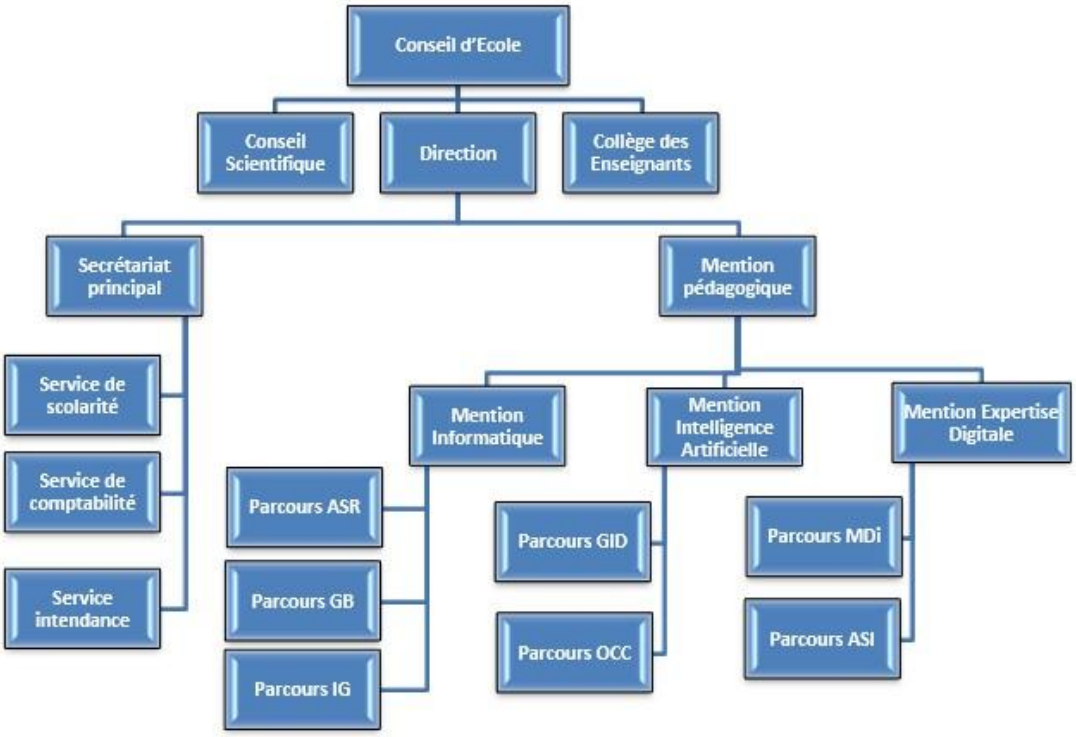


Figure 1: Organigramme de l'Ecole Nationale d'Informatique

Domaine de spécialisation

Les activités de formation et de recherche organisées à l'ENI portent sur cinq domaines :

- Génie logiciel et Base de Données ;
- Administration des Systèmes et Réseaux ;
- Informatique Générale ;
- Modélisation informatique et mathématique des Systèmes complexes ;
- Intelligence artificielle.

Le tableau 1 décrit l'organisation du système de formation pédagogique de l'Ecole.

Tableau 1: Organisation du système de formation pédagogique de l'Ecole

Formation Théorique	Formation Pratique
➢ Enseignement théorique ➢ Travaux dirigés ➢ Travaux pratiques ➢ Conférences	➢ Étude de cas ➢ Travaux de réalisation ➢ Projets/ Projets tutorés ➢ Voyages d'Études ➢ Stages en entreprise

Architecture des formations pédagogiques

Le recrutement des étudiants à l'ENI se fait uniquement par voie de concours d'envergure nationale en première année. Les offres de formation organisées à l'Ecole ont été validées par la Commission Nationale d'Habilitation (CNH). Au sein de l'ENI, il existe trois mentions et sept parcours. Le tableau 2 récapitule les mentions et les parcours au sein de l'Ecole :

Tableau 2: Mention et parcours au sein de l'ENI

Mention	Parcours
Informatique	Génie logiciel et Base de Données (GB)
	Administration des Systèmes et Réseaux (ASR)
	Informatique Générale (IG)
Intelligence Artificielle	Gouvernance et Ingénierie de Données (GID)
	Objets Connectés et Cyber sécurités (OCC)
Expertise Digitale	Métiers du Digital (MDi)
	Audit des Systèmes d'Information (ASI)

La figure 2 représente l'architecture des études correspondant au système LMD.

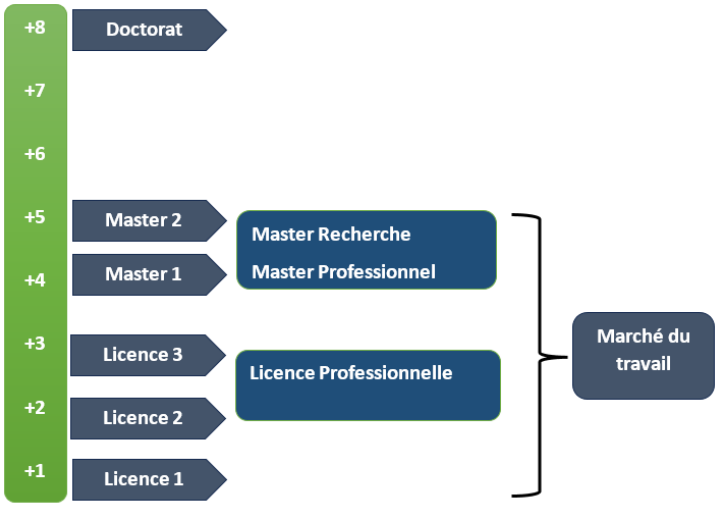


Figure 2: Architecture des études correspondant au système LMD

La licence peut avoir une vocation générale ou professionnelle. Le master peut avoir une vocation professionnelle ou de recherche. L'accès en première année de MASTER se fait automatiquement pour les étudiants de l'Ecole qui ont obtenu le diplôme de Licence Professionnelle.

Le tableau 3 illustre la liste des formations existantes à l'ENI.

Tableau 3: Liste des formations existantes à l'ENI

	FORMATION	
	LICENCE PROFESSIONNELLE	MASTER
Condition admission	Par voie de concours	Par voie de concours pour la mention IA
Condition d'accès	Bac de série C, D ou Technique	Être titulaire de licenceprofessionnelle
Durée de Formation	3 ans	2 ans
Diplôme délivré	Diplôme de Licence Professionnelle	Diplôme de Master Professionnel Diplôme de Master Recherche

Le Master Recherche permet à son titulaire de poursuivre directement des études en doctorat et de s'inscrire directement dans une Ecole Doctorale.

Les étudiants diplômés de l'Ecole sont plutôt bien accueillis dans les instituts universitaires étrangères (Canada, Suisse, France, …)

Relation de l'ENI avec les organismes externes

Les stages effectués chaque année par les étudiants mettent l'Ecole en rapport permanent avec plus de 400 entreprises et organismes publics, semi-publics et privés, nationaux et internationaux. L'Ecole dispose ainsi d'un réseau d'entreprises, de sociétés et d'organismes publics et privés qui sont des partenaires par l'accueil en stage de ses étudiants, et éventuellement pour le recrutement après l'obtention des diplômes par ces derniers. Les compétences que l'Ecole cherche à développer chez ses étudiants sont l'adaptabilité, le sens de la responsabilité, du travail en équipe, le goût de l'expérimentation et l'innovation.

En effet, la vocation de l'ENI est de former des licenciés et des ingénieurs de niveau MASTER avec des qualités scientifiques, techniques et humaines reconnues, capables d'évoluer professionnellement dans des secteurs d'activité variés intégrant l'informatique. Les stages en milieu professionnel permettent de favoriser une meilleure adéquation entre les formations à l'Ecole et les besoins évolutifs du marché de l'emploi.

Parmi les sociétés, les entreprises et les organismes partenaires de l'Ecole, on peut citer : ACCENTURE Mauritius, AKATA Goavana, Air Madagascar, Ambre Associates, Airtel, Agence Universitaire de la Francophonie (AUF), AXIAN, B2B, Banque Centrale, , BIANCO, BlueLine, CNaPS, Bureau National de Gestion des Risques et des Catastrophes (BNGRC), CEDII-Fianarantsoa, Data Consulting, Central Test, Centre National Antiacridien, CNRE, COLAS, Direction Générale des

Douanes, DLC, E-Tech Consulting, , FID, FIHARY Soft, FTM, GNOSYS, GENIUS AT WORK, Hello Tana, IBONIA, INGENOSIA, INSTAT, IOGA, JIRAMA, JOUVE, MADADEV, MAEP, MANAO, MEF, MEN, MESupRES, MFB, , MININTER, Min des Postes/Télécommunications et du Développement Numérique, NEOV MAD, Ny Havana, Madagascar National Parks, OMNITEC, ORANGE, OTME, PRACCESS, QMM Fort-Dauphin, SG Madagasikara SMMC, SMMEC, SNEDADRS Antsirabe, Sénat, Société d'Exploitation du Port de Toamasina (SEPT), SOFTWELL, Strategy Consulting, TELMA, VIVETEC, Société LAZAN'I BETSILEO, WWF, UGD, ARATO, MANAO, MNDPT, NG ACADEMY.NG, Relia, Spoon Consulting …

Débouchés professionnels et diplômés

Les formations proposées par l'Ecole permettent aux diplômés d'être immédiatement opérationnels sur le marché du travail avec la connaissance d'un métier complet lié à l'informatique aux TIC.

L'Ecole apporte à ses étudiants un savoir-faire et un savoir-être qui les accompagnent tout au long de leur vie professionnelle. Elle a une vocation professionnalisante. Les diplômés en LICENCE et en MASTER issus de l'ENI peuvent faire carrière dans différents secteurs.

L'Ecole bénéficie aujourd'hui de plus de 40 années d'expérience pédagogiques et de reconnaissance auprès des sociétés, des entreprises et des organismes. C'est une Ecole Supérieure de référence en matière informatique.

D'une manière générale, les diplômés de l'ENI n'éprouvent pas de difficultés particulières à être recrutés au terme de leurs études. Cependant, l'ENI recommande à ses diplômés de promouvoir l'entrepreneuriat en TIC et de créer des cybercafés, des SSII ou des bureaux d'études.
Le tableau 4 représente les débouchés éventuels selon la mention.

Tableau 4: Débouchés éventuels des jeunes diplômés

Mention	Débouchés professionnels
INFORMATIQUE	GB : Développeur logiciel / développeur full-stack, Ingénieur en développement d'applications (desktop, web, mobile), Architecte logiciel, Intégrateur d'applications, Développeur d'API, Administrateur de bases de données (DBA), Ingénieur en conception de bases de données, Développeur SQL / PL-SQL, Spécialiste en migration et optimisation de bases de données, Data engineer (ingénieur en traitement de données), Analyste programmeur, Analyste fonctionnel, Chef de projet informatique, Scrum master / Product owner (en méthodologie Agile), Ingénieur QA (Quality Assurance), Testeur logiciel / testeur automatisé, Spécialiste maintenance

	applicative, Expert en refactoring et optimisation de code, Analyste de données (Data analyst), Concepteur d'outils décisionnels (BI), Ingénieur en Big Data, Machine Learning Engineer, Consultant fonctionnel ou freelance ASR : Administrateur systèmes et réseaux, Ingénieur systèmes et réseaux, Administrateur systèmes Linux / Windows, Administrateur cloud (AWS, Azure, Google Cloud), Ingénieur virtualisation (VMware, Hyper-V, Proxmox), Administrateur sécurité (Security Administrator), Ingénieur cybersécurité, Analyste SOC (Security Operations Center), Spécialiste firewall / VPN, Responsable sécurité informatique (RSSI – avec expérience Technicien support informatique (N2/N3), Technicien réseaux et télécoms, Ingénieur exploitation, Administrateur messagerie (Exchange, Microsoft 365), Technicien datacenter, Architecte réseaux, Architecte systèmes, Consultant infrastructure, Chef de projet systèmes et réseaux, Ingénieur DevOps, Administrateur Kubernetes / Docker, Cloud engineer, Spécialiste automatisation (Ansible, Terraform) IG : GB et SR en même temps
INTELLIGENCE ARTIFICIELLE	GID : Développeur IA, Data Scientist, Entrepreneur IA, Consultant IA, Responsable marketing IA, Business Developer IA, Product Owner, Consultant en cybersécurité, Directeur de l'Innovation ; OCC : Analyste en cybersécurité, Ingénieur en cybersécurité, Spécialiste en réponse aux incidents, Architecte de sécurité, Consultant en cybersécurité, Auditeur en cybersécurité, Expert en sécurité des données, Responsable de la sécurité des informations, Chercheur en cybersécurité, Formateur en cybersécurité, Ingénieur IoT, Développeur IoT, Architecte IoT, Spécialiste en sécurité IoT, Analyste de données
EXPERTISE DIGITALE	MDi : Spécialiste en Marketing Digital, Gestionnaire de Contenu Web, Analyste UX/UI, Product Owner, Chef de Projet Digital, Stratège en Médias Sociaux, Analyste de Performance Digitale, Chef de Projet Digital, Stratège en Médias Sociaux, Concepteur-Rédacteur Web Analyste de Performance Digitale, Expert en Commerce Électronique, Expert en Commerce Électronique, Spécialiste SEO/SEM, Manager de Communauté (Community Manager), Conseiller en Stratégie Digitale, Responsable CRM (Customer Relationship Management), Spécialiste en Analytique et Intelligence

	d'Affaires (Business Intelligence), Consultant en Transformation Digitale, Responsable des Partenariats Digitaux, Spécialiste en Cybersécurité Marketing, Directeur de l'Innovation Digitale, Manager de l'E-réputation ASI : Auditeur des Systèmes d'Information, Consultant en Sécurité des Systèmes d'Information, Analyste de Risques IT, Chef de Projet SI, Responsable de la Conformité IT, Spécialiste en Cyber Forensics, Directeur de l'Audit et de la Sécurité des Systèmes d'Information, Expert en Gouvernance IT, Formateur ou Éducateur en Sécurité des Chercheur en Sécurité IT

Ressources humaines

Les ressources humaines sont présentées selon leurs responsabilités :

- Directeur de l'Ecole : Monsieur MAHATODY Thomas, Professeur
- Responsable de la Mention « Informatique » : Monsieur RALAIVAO Jean Christian, Maître de Conférences
- Responsable de la Mention « Intelligence Artificielle » : Monsieur DIMBISOA William Germain, Maître de Conférences

L'ENI compte treize (13) enseignants permanents dont un (01) Professeur Titulaire, deux (02) Professeurs, huit (08) Maîtres de Conférences, deux (02) Assistants d'Enseignement Supérieur et de Recherche.

CHAPITRE 2 : Présentation de l'établissement d'accueil

2.1 Présentation générale du groupe Manao

Le stage s'est déroulé au sein du **groupe Manao**, un groupe d'entreprises spécialisé dans l'édition de logiciels de gestion en ligne, le développement d'applications sur mesure, l'externalisation de la comptabilité et de la paie, ainsi que le conseil en organisation et en gestion. Né en France, à Montpellier, le groupe s'est progressivement développé à Madagascar, où se trouve aujourd'hui l'essentiel de ses équipes de production, de développement, d'assistance et de formation.

Ses collaborateurs (développeurs, techniciens, formateurs, gestionnaires et commerciaux), répartis entre la France et Madagascar, sont au service d'une centaine d'entreprises clientes, des TPE et PME jusqu'aux grands groupes, ainsi que d'établissements d'enseignement.

La figure 3 présente le logo du groupe.



Figure 3: Logo du groupe Manao

2.2 Historique

L'histoire du groupe est celle d'une croissance continue, partie d'un éditeur de logiciels et étendue progressivement à l'hébergement, au développement sur mesure puis aux services de gestion. Le tableau 5 en retrace les principales étapes.

Tableau 5: Historique du groupe Manao

Année	Événement
2008	Création à Montpellier de **MANAO**, éditeur de logiciels de gestion, à l'initiative d'un comptable souhaitant proposer une solution d'organisation et de gestion en ligne pour les PME et les établissements d'enseignement. Rachat de la société Lauréats Informatique (logiciels de gestion des notes et des absences).
2009	Création de la plateforme logicielle **SCOLARIS** (d'abord appelée CAMPUS) pour les établissements scolaires, réunissant les logiciels dans un lanceur commun.
2010	Création du premier datacenter à Montpellier (baie de 60 serveurs) : la compétence en hébergement s'ajoute au développement. SCOLARIS devient une plateforme en ligne utilisée par 20 000 utilisateurs chaque mois.
2011	Création de la **plateforme MANAO** pour la gestion des PME, avec un premier logiciel de gestion de la trésorerie, puis un logiciel de prélèvements SEPA en 2012.
2013	Implantation à Madagascar : ouverture d'une filiale de production pour le développement, l'assistance et la formation, les datacenters restant en France.

Année	Événement
2015	Lancement de la branche de développement sur mesure de logiciels en ligne pour des entreprises françaises.
2018	Croissance de dix personnes par an, jusqu'à 40 collaborateurs. Lancement à Madagascar de deux activités : l'externalisation de la comptabilité et de la paie pour des entreprises françaises, et le conseil en organisation pour des entreprises malgaches.
2019	Organisation en quatre activités autonomes : SIDINA, MANAO, EZAKA et TETIKA.
2020	La branche de développement sur mesure devient une société à part entière, **SIDINA** (« décollage » en malgache).
2021	Installation de SIDINA à Fianarantsoa, ville réputée pour son université et les nombreux développeurs qu'elle forme chaque année.
2022	L'équipe atteint 60 collaborateurs, dont 40 développeurs, 10 techniciens système et datacenter et 10 techniciens et formateurs, et mène en moyenne 10 projets sur mesure en parallèle.

Les applications des clients sont hébergées sur trois datacenters distincts : un datacenter de production, un datacenter de sauvegardes quotidiennes et un datacenter de redémarrage en quatre heures, testé quotidiennement. Cette infrastructure permet au groupe de prendre en charge des projets manipulant des données financières ou personnelles sensibles, fortement sécurisées contre le piratage et les rançongiciels.

2.3 Philosophie et valeurs

Le projet du groupe repose sur la réussite de ses collaborateurs malgaches, résumée par l'ambition de « réussir à partir de Madagascar » : bâtir un groupe d'entreprises performantes, travaillant à l'international depuis Madagascar. Cette philosophie s'articule autour de trois valeurs :

- **Confiance** : chacun fait confiance à ses collègues dans le travail comme dans le comportement, ce qui exclut toute forme de triche.
- **Progression** : l'erreur est permise et doit servir à progresser ; la performance, l'exigence et l'expertise permettent aux collaborateurs de progresser jour après jour.
- **Responsabilité** : tout le monde n'est pas chef, mais chacun est responsable de ses tâches, de ses missions et de ses réalisations, en analysant le contexte et les conséquences de ses actions.

2.4 Domaines d'expertise

Le groupe réunit deux grandes familles de compétences, rarement associées au sein d'une même structure :

- **En gestion** : comptabilité, fiscalité, social et organisation d'entreprise, auprès de TPE, de PME et de grands groupes ;

- **En développement informatique** : applications de gestion (logiciels, sites e-commerce), développement web (PHP, HTML, Python, Java), développement mobile et développement en intelligence artificielle ;
- **Dans les domaines connexes** : bases de données, infrastructure serveurs, datacenter, hébergement, télécommunications, assistance, formation et activité commerciale.

2.5 Branches et activités

Le groupe est organisé en branches autonomes, chacune portant une activité distincte (tableau 6).

Tableau 6: Branches du groupe Manao

Branche	Activité	Site web
MANAO	Édition de logiciels standards de gestion pour les entreprises	www.manao.eu
SIDINA	Développement de logiciels sur mesure	www.sidina.eu
EZAKA	Externalisation de la comptabilité, de la paie et des services administratifs	www.ezaka.eu
TETIKA	Conseil et assistance en organisation et en gestion	www.tetika.eu
SCOLARIS	Plateforme logicielle pour les établissements d'enseignement	scolaris.info

2.6 Implantations

Le groupe est présent en France et dans trois villes de Madagascar :

- **Montpellier (France)**, siège de Manao : 14 rue des Anoubles, 34070 Montpellier ;
- **Antananarivo** : Tana Water Front, Lot IVP PRES ;
- **Toamasina** : Lot N°7 secteur I Tanandava, Toamasina 501 ;
- **Fianarantsoa** : Antanifotsy, Fianarantsoa 301.

2.7 La plateforme logicielle Manao

La plateforme Manao regroupe les logiciels de gestion en ligne édités par le groupe. Elle est organisée en « univers » fonctionnels, présentés dans le tableau 7.

Tableau 7: Univers de la plateforme logicielle Manao

Univers	Modules
Ventes	Gestion commerciale (Gescom), paiements, achats, espace clients
Gestion	Comptabilité, trésorerie, prélèvements SEPA
Ressources humaines	Paie, gestion des intermittents, bulletins de paie web et mobile
Plateforme	Licences utilisateurs, licence maître
Production	Gestion de projet, CMS (sites internet)
Communication	Messagerie, envoi de SMS

Univers	Modules
Bureautique	Gestion électronique des documents (GED), contacts, agenda, traitement de texte, tableur

2.8 Organisation et organigramme

Les activités de développement sont placées sous la responsabilité d'un Directeur du développement (DGO Développement), comme le montre la figure 4. Elles sont organisées en trois pôles :

- la **Plateforme MANAO**, dirigée par un directeur adjoint, qui regroupe les équipes de chaque univers (Ventes, Production, Plateforme, Site internet CMS, Bureautique, RH, Gestion, Communication), ainsi que Scolaris et la Réussite clients ;
- le pôle **Tests**, chargé des tests automatisés et des tests de qualité de l'ensemble des produits ;
- le pôle **DSM** (développement sur mesure), dirigé par un second directeur adjoint, qui porte les projets réalisés pour les clients et l'assistance associée.

Chaque équipe est animée par un chef de projet et, le cas échéant, par un référent de projet. Au-delà des équipes de production (développeurs et gestionnaires), le groupe s'appuie sur des services supports transversaux : direction administrative et financière, gestion du personnel et recrutement, commerciaux, graphistes, techniciens d'assistance et techniciens système et réseau.

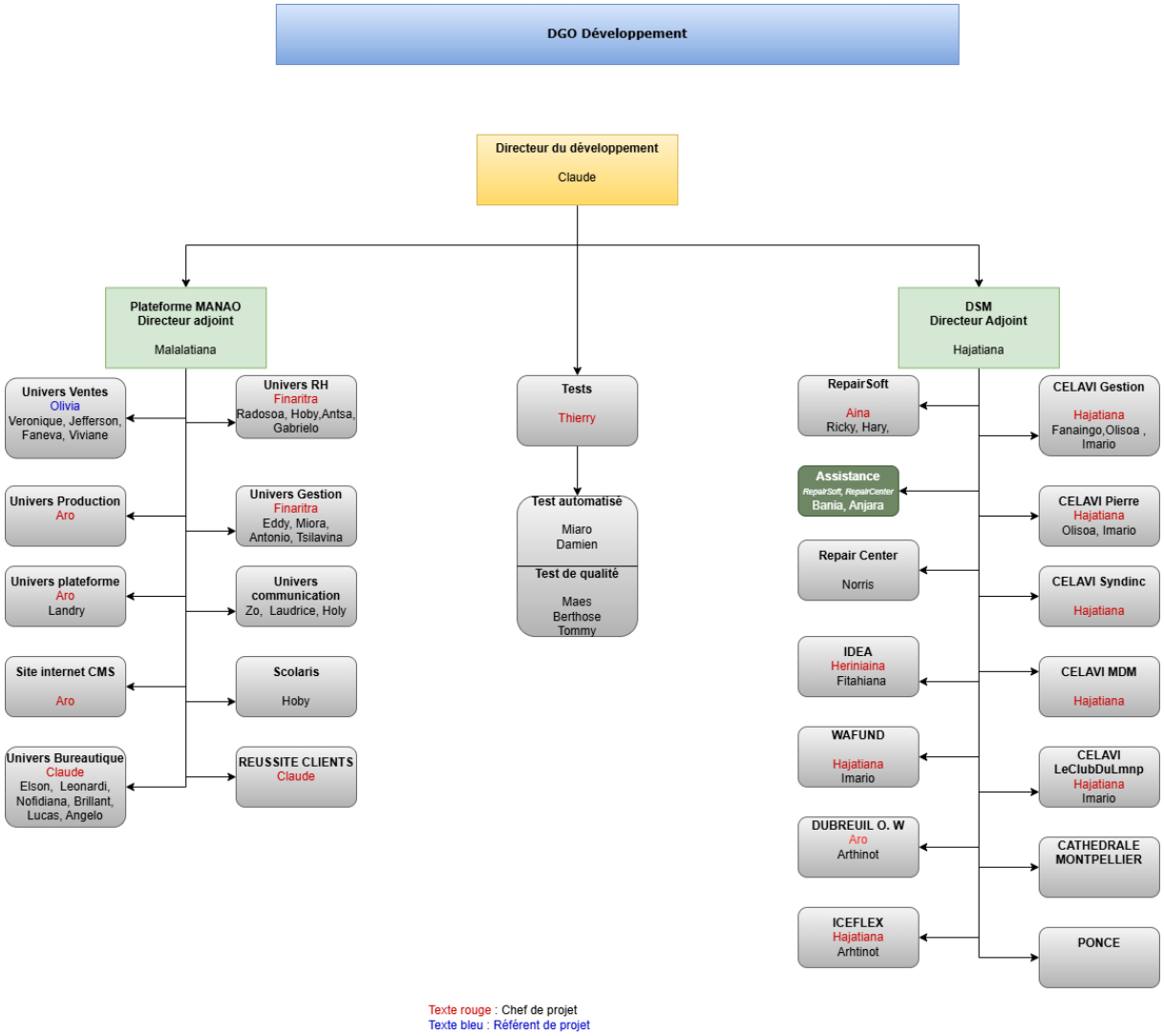


Figure 4: Organigramme de la direction du développement du groupe Manao

2.9 Place du stagiaire et lien avec le projet

Le stage a été effectué au sein de l'équipe **Univers Ventes** de la plateforme MANAO, au poste de développeur full-stack. Cette équipe développe les modules de gestion commerciale, de paiements, d'achats et d'espace clients, qui produisent et reçoivent les factures commerciales.

Le sujet de ce mémoire s'inscrit directement dans le cœur de métier du groupe, qui associe des compétences en gestion comptable et en développement informatique. Les factures émises et reçues via l'univers Ventes alimentent l'univers Gestion, où les écritures comptables sont enregistrées, et les activités d'externalisation comptable d'EZAKA traitent quotidiennement des volumes importants de factures. L'automatisation de la génération des écritures comptables à partir de ces factures répond donc à un besoin concret de l'environnement dans lequel le stage s'est déroulé.

CHAPITRE 3 : Description du projet

Ce chapitre décrit le projet réalisé : il en formule la problématique, précise les objectifs et les besoins des utilisateurs, recense les moyens nécessaires à sa réalisation, puis présente les résultats attendus et le chronogramme de travail.

3.1 Formulation :

L'évolution rapide des technologies d'intelligence artificielle transforme progressivement les méthodes utilisées par les entreprises pour traiter et exploiter leurs données. Dans le domaine de la gestion comptable, cette transformation offre de nouvelles possibilités pour automatiser certaines tâches répétitives et améliorer la fiabilité du traitement des informations financières. Parmi les documents les plus importants dans ce processus figurent les factures commerciales, qui constituent une source essentielle d'informations pour l'enregistrement des opérations comptables.

Traditionnellement, le traitement d'une facture commerciale nécessite plusieurs interventions humaines. Le comptable doit prendre connaissance du document, identifier les informations relatives à l'opération, déterminer sa nature, choisir les comptes comptables appropriés, puis saisir manuellement l'écriture correspondante dans un système comptable. Lorsque le nombre de factures à traiter devient important, cette procédure peut devenir longue et entraîner des risques d'erreurs de saisie, de mauvaise interprétation ou d'imputation comptable.

Face à ces contraintes, l'automatisation intelligente du traitement des factures constitue un enjeu important pour améliorer l'efficacité des services comptables. Cependant, automatiser ce processus ne consiste pas uniquement à extraire les informations présentes sur une facture. Le système doit également être capable de comprendre la nature de l'opération, d'identifier les éléments pertinents sur le plan comptable et de proposer une écriture respectant le principe de la partie double, tout en permettant au comptable de contrôler le résultat obtenu.

L'utilisation combinée de la Reconnaissance Optique de Caractères (OCR), du Traitement Automatique du Langage Naturel (NLP) et du Machine Learning permet d'envisager une solution capable de répondre à ces différentes exigences. L'OCR permet d'extraire les informations présentes dans les factures sous forme d'images ou de documents PDF. Le NLP permet ensuite d'analyser et d'interpréter les informations textuelles extraites, tandis que le Machine Learning peut être utilisé pour identifier la nature des opérations et proposer les comptes comptables appropriés. Ces techniques seront complétées par un moteur de règles comptables, permettant de garantir la conformité des écritures générées avec le référentiel comptable retenu.

Ces constats amènent à poser le problème central qui guide ce projet :

Comment concevoir un système intelligent capable d'analyser automatiquement les factures commerciales, d'en extraire et d'interpréter les informations pertinentes, puis de générer des écritures comptables fiables et conformes aux règles comptables, tout en permettant au professionnel de contrôler et de valider les résultats proposés ?

C'est dans cette perspective que s'inscrit le présent projet intitulé :

« Conception et développement d'un système intelligent de génération automatique des écritures comptables à partir des factures commerciales utilisant l'OCR, le NLP et le Machine Learning. »

3.2 Objectif et besoin de l'utilisateur :

3.2.1 Objectif :

L'objectif principal de ce projet est de concevoir et de développer un système intelligent capable de générer automatiquement des écritures comptables à partir des factures commerciales, en s'appuyant sur les technologies de la Reconnaissance Optique de Caractères, du Traitement Automatique du Langage Naturel et du Machine Learning.

Ce système a pour vocation d'automatiser le processus de traitement comptable des factures, de réduire les interventions manuelles et d'améliorer la fiabilité ainsi que la rapidité de la production des écritures comptables.

Plus spécifiquement, le projet vise à :

- Concevoir une architecture logicielle robuste et évolutive permettant l'intégration des modules d'OCR, de NLP, de Machine Learning et de gestion comptable.
- Développer un module capable d'extraire automatiquement les informations essentielles contenues dans les factures commerciales, telles que le fournisseur, la date, les montants HT, la TVA, le montant TTC ainsi que la nature des opérations.
- Mettre en œuvre un modèle intelligent permettant d'interpréter les informations extraites et d'identifier automatiquement les comptes comptables appropriés conformément au plan comptable de l'entreprise.
- Générer automatiquement des écritures comptables équilibrées, tout en offrant au comptable la possibilité de les vérifier, de les modifier et de les valider avant leur enregistrement.
- Évaluer les performances du système en termes de précision de l'extraction des données, de pertinence des écritures générées, de rapidité de traitement et de réduction des erreurs de saisie.

- Proposer une solution intuitive, fiable et évolutive contribuant à l'amélioration de la productivité des services comptables et à la modernisation des processus de gestion financière des entreprises.

3.2.2 Besoin de l'utilisateur :

L'utilisateur d'un système intelligent de génération automatique des écritures comptables à partir des factures commerciales à besoin d'une solution permettant de simplifier et d'accélérer le traitement des documents comptables. Le système doit réduire les tâches manuelles tout en garantissant la fiabilité des informations extraites et des écritures générées.

Parmi les principaux besoins de l'utilisateur, on peut citer :

- **Facilité d'utilisation** : disposer d'une interface simple, intuitive et accessible permettant de déposer et de traiter facilement une facture.
- **Gain de temps** : réduire le temps consacré à la lecture, à l'analyse et à la saisie manuelle des factures.
- **Extraction automatique des informations** : récupérer automatiquement les données importantes contenues dans une facture commerciale.
- **Interprétation intelligente** : comprendre la nature de l'opération commerciale et identifier les éléments nécessaires à la comptabilisation.
- **Génération automatique des écritures** : obtenir une proposition d'écriture comptable à partir des informations extraites.
- **Fiabilité des résultats** : garantir que les écritures générées respectent le principe de la partie double, avec l'équilibre entre le débit et le crédit.
- **Contrôle et validation** : permettre au comptable de vérifier, modifier et valider les écritures proposées par le système.
- **Traçabilité** : conserver l'historique des factures traitées, des écritures générées et des modifications effectuées.
- **Polyvalence** : permettre le traitement des factures d'achat et des factures de vente sous différents formats, notamment PDF et image.

3.3 Moyens nécessaires à la réalisation du projet :

La réalisation du projet a mobilisé des moyens humains, matériels et logiciels, présentés dans les sections suivantes.

3.3.1 Moyens humains :

Le projet a été réalisé individuellement par un étudiant en deuxième année de Master Intelligence Artificielle, parcours Gouvernance et Ingénierie des Données, qui a assuré l'ensemble

des activités : analyse, conception, développement, tests et déploiement. Il a bénéficié de l'encadrement pédagogique du Dr. RALAIVAO Jean Christian, Maître de Conférences à l'École Nationale d'Informatique, et de l'encadrement professionnel de Mlle MALALATIANA Marie Philippine, Directeur Adjoint au sein du groupe Manao. Au-delà du suivi de l'avancement, l'encadreur professionnel a expliqué le cahier des charges (CDC) du projet : les besoins exprimés, le périmètre fonctionnel attendu et les contraintes à respecter.

3.3.2 Moyens matériels :

Le projet nécessite plusieurs ressources matérielles pour assurer le développement, les tests et le déploiement du système. Le tableau 8 les présente.

Tableau 8: Ressources matérielles nécessaires au projet

Ressource	Caractéristiques	Rôle dans le projet
Ordinateur de développement	Processeur Intel Core i5 ou équivalent ; 8 Go de RAM minimum (16 Go recommandés) ; SSD de 256 Go minimum ; carte graphique dédiée facultative	Développer l'application, exécuter les tests, traiter les factures et entraîner les modèles de Machine Learning
Serveur local ou distant	Serveur privé virtuel (VPS) sous Linux, loué chez un hébergeur	Héberger l'application, l'API, la base de données et les modules d'intelligence artificielle
Scanner ou smartphone	Numérisation de qualité suffisante pour une lecture nette du texte	Transformer les factures papier en images exploitables par le module OCR
Connexion Internet	Connexion haut débit	Télécharger les bibliothèques et les modèles, effectuer les mises à jour et le déploiement
Espace de stockage	Disque local et volume persistant sur le serveur	Conserver les factures originales, les résultats OCR, les données d'entraînement et l'historique

3.3.3 Moyens logiciels :

Le projet requiert des outils et logiciels spécifiques pour le développement et le fonctionnement du système, présentés dans le tableau 9.

Tableau 9: Outils et logiciels du projet

Logiciel	Rôle dans le projet	Référence
Visual Studio Code	Éditeur de code utilisé pour le frontend React, le backend FastAPI et les modules d'IA (autocomplétion, débogage, terminal intégré, intégration Git)	[10]
Visual Paradigm	Modélisation UML : diagrammes de cas d'utilisation, de classes, d'activité et de déploiement	[28]

Logiciel	Rôle dans le projet	Référence
MySQL	Système de gestion de base de données relationnelle : utilisateurs, entreprises, factures, résultats OCR, prédictions, écritures, corrections et historique	[12]
Git et GitHub	Gestion des versions du code et sauvegarde du dépôt	[18]
Docker et Docker Compose	Conteneurisation et déploiement reproductible de l'application sur le serveur	[13]

3.3.4 Langages, bibliothèques et composants :

Le tableau 10 présente les langages, bibliothèques et composants retenus, ainsi que leur rôle dans le système.

Tableau 10: Technologies et composants du système

Technologie	Utilisation dans le projet	Référence
React 18 et TypeScript 5	Interface web : authentification, import et visualisation des factures, affichage du texte extrait et des écritures proposées, correction, validation, historique et tableau de bord	[21], [27]
Tailwind CSS	Mise en forme moderne, responsive et ergonomique de l'interface	[25]
FastAPI	Couche backend : logique métier, API REST documentée automatiquement (Swagger/OpenAPI), communication entre l'interface, la base de données et les modules d'IA	[17]
Python 3.10	Langage des modules d'IA : prétraitement des images, extraction et traitement du texte, classification, entraînement et évaluation des modèles	[15]
OpenCV	Prétraitement des images : redressement, débruitage et binarisation	[14]
EasyOCR et Tesseract	Reconnaissance optique des caractères (moteur principal et moteur de secours) : numéro, date, fournisseur, montants HT, TVA et TTC	[1], [8], [19], [26]
spaCy et expressions régulières	Traitement automatique du langage naturel : repérage des entités et de la nature de l'opération	[24]
scikit-learn et XGBoost	Apprentissage automatique : classification du type d'opération et suggestion de comptes avec un score de confiance	[2], [7], [22]
Moteur de règles comptables	Vérification de la cohérence des comptes, du traitement de la TVA et de l'équilibre débit/crédit selon le PCG 2005	[5]
SQLAlchemy	ORM entre l'application Python et MySQL : modèles de données, accès aux factures, aux écritures, aux utilisateurs et à l'historique	[16]

3.4 Résultats Attendus

Le projet vise à développer un système intelligent capable d'automatiser le traitement des factures commerciales et de proposer automatiquement les écritures comptables correspondantes.

3.4.1 Extraction et compréhension des factures

Le système devra être capable de recevoir des factures sous différents formats, notamment PDF et images, puis d'en extraire automatiquement les informations pertinentes grâce à l'OCR.

Le module NLP devra ensuite permettre d'interpréter les informations extraites afin d'identifier la nature de l'opération commerciale.

3.4.2 Génération automatique des écritures comptables

Le résultat principal attendu est la génération automatique d'une écriture comptable à partir des informations contenues dans une facture.

3.4.3 Contrôle et validation par le comptable

Le système ne devra pas remplacer totalement le contrôle humain.

L'écriture proposée par l'intelligence artificielle devra être présentée au comptable avec les informations ayant conduit à sa génération ainsi qu'un score de confiance.

Le comptable pourra :

- Accepter l'écriture ;
- Modifier les comptes ;
- Modifier les montants si nécessaires ;
- Corriger les informations extraites ;
- Rejeter la proposition.

Les corrections validées pourront ensuite être utilisées comme données d'apprentissage pour améliorer progressivement les performances du système.

3.4.4 Gain de temps et réduction des erreurs

Le système devra permettre de réduire considérablement les opérations manuelles liées au traitement des factures.

Les résultats attendus sont notamment :

- Une diminution du temps de saisie ;
- Une réduction des erreurs de transcription ;
- Une amélioration de la rapidité de traitement ;
- Une meilleure cohérence des écritures ;

- Une réduction des tâches répétitives pour les comptables.

3.4.5 Traçabilité et sécurité

Le système devra conserver un historique des traitements effectués.

Pour chaque facture, il devra être possible de retrouver :

- La facture originale ;
- Les informations extraites ;
- Les résultats de l'analyse ;
- L'écriture proposée ;
- Les modifications effectuées ;
- L'utilisateur ayant effectué la validation ;
- La date de validation.

Cette fonctionnalité permettra d'assurer la traçabilité des opérations comptables et facilitera les contrôles ultérieurs.

3.4.6 Architecture évolutive

Enfin, le projet devra aboutir à une architecture modulaire, robuste et évolutive, permettant d'intégrer ultérieurement de nouvelles fonctionnalités.

Le système pourra notamment évoluer vers :

- La gestion des avoirs ;
- La gestion des notes de frais ;
- L'analyse des relevés bancaires ;
- L'intégration avec des logiciels comptables ;
- L'amélioration continue du modèle de Machine Learning ;
- L'ajout d'autres référentiels comptables ;
- La prise en charge de nouveaux formats de documents.

Ainsi, le résultat final attendu n'est pas uniquement une application permettant de lire une facture, mais une solution intelligente d'aide à la production comptable, combinant l'extraction automatique des données, leur compréhension et la génération contrôlée des écritures comptables.

3.5 Chronogramme :

La figure 5 présente le diagramme de Gantt du projet, qui s’est déroulé sur six mois, du 7 avril 2026 au 6 octobre 2026.

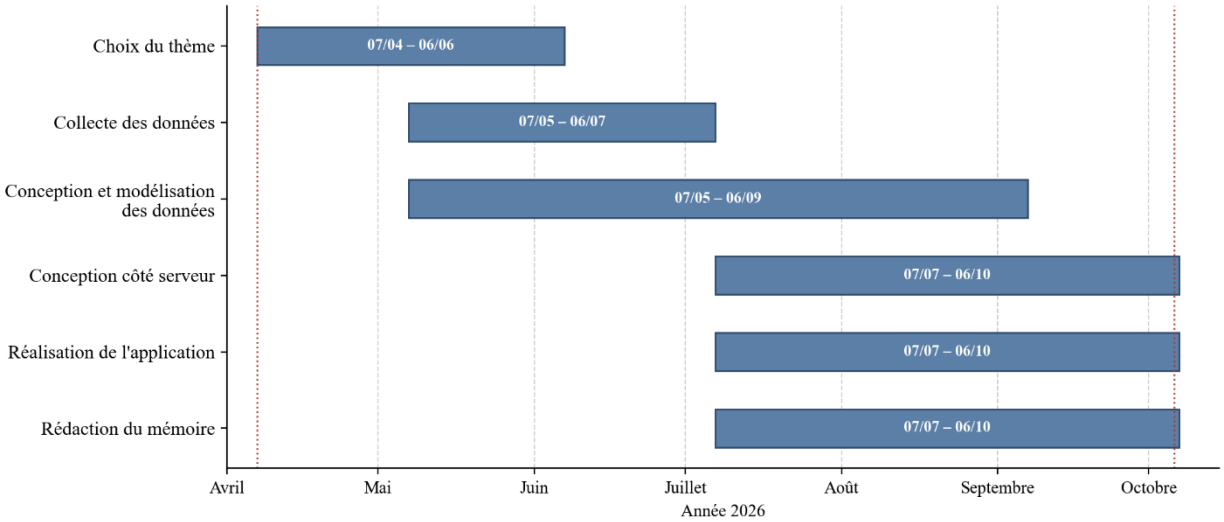


Figure 5: Diagramme de Gantt du projet

PARTIE II. ANALYSE ET CONCEPTION

CHAPITRE 4 : Analyse préalable

Ce chapitre analyse le traitement actuel des factures afin d'en dégager les points forts et les points faibles, puis présente les solutions envisagées et la démarche de conception retenue.

4.1 Analyse de l'existant

4.1.1 Organisation actuelle

Avant la mise en place d'un système intelligent de génération automatique des écritures comptables, le traitement des factures commerciales repose principalement sur une intervention humaine. Le comptable doit effectuer successivement plusieurs opérations, depuis la réception de la facture jusqu'à son enregistrement dans le système comptable.

Le personnel impliqué dans ce traitement comprend principalement le comptable, chargé de la lecture, de la saisie et de l'imputation des factures, et le responsable comptable ou financier, qui contrôle les écritures et valide les travaux avant la clôture des comptes.

Ce processus nécessite l'analyse du contenu de la facture, l'identification des informations importantes, la détermination de la nature de l'opération, la recherche des comptes comptables appropriés et enfin la saisie de l'écriture comptable.

Lorsque le nombre de factures à traiter est important, cette méthode devient particulièrement chronophage. Elle augmente également les risques d'erreurs humaines, notamment lors de la transcription des informations, du choix des comptes comptables ou de la saisie des montants.

a) Description du processus traditionnel

Le processus traditionnel peut être décomposé en plusieurs étapes :

Réception de la facture : La facture est reçue sous différents formats, notamment sous forme papier, PDF ou image. Le comptable doit alors récupérer et organiser les documents avant leur traitement.

Lecture et identification des informations : Le comptable consulte manuellement la facture afin d'identifier les informations nécessaires telles que le numéro de facture, la date, le fournisseur ou le client, les produits ou services, les montants hors taxes, la TVA et le montant TTC.

Saisie manuelle des informations : Les données identifiées sur la facture sont ensuite saisies manuellement dans le logiciel comptable ou dans un autre outil de gestion.

Analyse de l'opération : Le comptable doit déterminer la nature de l'opération commerciale afin de savoir quels comptes comptables doivent être utilisés.

Recherche des comptes comptables : À partir de la nature de l'opération, le comptable recherche dans le plan comptable les comptes correspondant à l'opération concernée.

Génération et saisie de l'écriture : Les comptes identifiés sont ensuite utilisés pour créer l'écriture comptable en respectant le principe de la partie double.

Vérification et validation : Le comptable vérifie ensuite l'exactitude des informations saisies et s'assure que le total des débits est égal au total des crédits avant de valider l'écriture.

Archivage et suivi : Enfin, la facture et l'écriture comptable sont conservées afin de permettre leur consultation ultérieure et d'assurer la traçabilité des opérations.

Ce processus manuel présente donc plusieurs limites liées au temps de traitement, aux risques d'erreurs de saisie, à la répétitivité des tâches et à la dépendance à l'expertise du comptable.

Le tableau 11 récapitule ces étapes, avec les observations et les limites associées à chacune d'elles.

Tableau 11: Récapitulation des étapes du processus traditionnel

Étape	Action	Observation	Limites
Réception de la facture	Réception du document PDF, image ou papier	Formats différents selon les fournisseurs	Organisation manuelle des documents
Lecture de la facture	Consultation visuelle du document	Les informations doivent être repérées manuellement	Temps de lecture important
Extraction des informations	Identification du numéro, date, fournisseur, montants, TVA, etc.	Les données sont relevées manuellement	Risque d'erreur de transcription
Saisie des données	Saisie des informations dans le logiciel comptable	Plusieurs champs doivent être renseignés	Tâche répétitive et chronophage
Analyse de l'opération	Détermination de la nature de l'achat ou de la vente	Analyse basée sur l'expérience du comptable	Risque de mauvaise interprétation
Recherche des comptes	Recherche des comptes dans le plan comptable	Consultation du référentiel comptable	Temps supplémentaire
Génération de l'écriture	Création des lignes débit et crédit	Les montants sont saisis manuellement	Risque d'erreur comptable
Vérification	Contrôle de l'équilibre débit/crédit et des informations	Vérification humaine nécessaire	Temps de contrôle
Validation	Validation définitive de l'écriture	L'écriture est enregistrée	Intervention humaine obligatoire

Archivage	Conservation de la facture et de l'écriture	Historique des opérations	Gestion et recherche parfois difficiles

b) Cartographie du processus actuel

Afin de mieux comprendre le fonctionnement du processus traditionnel et d'identifier les différentes étapes pouvant être automatisées, le workflow actuel de traitement d'une facture commerciale est représenté par la figure 6.

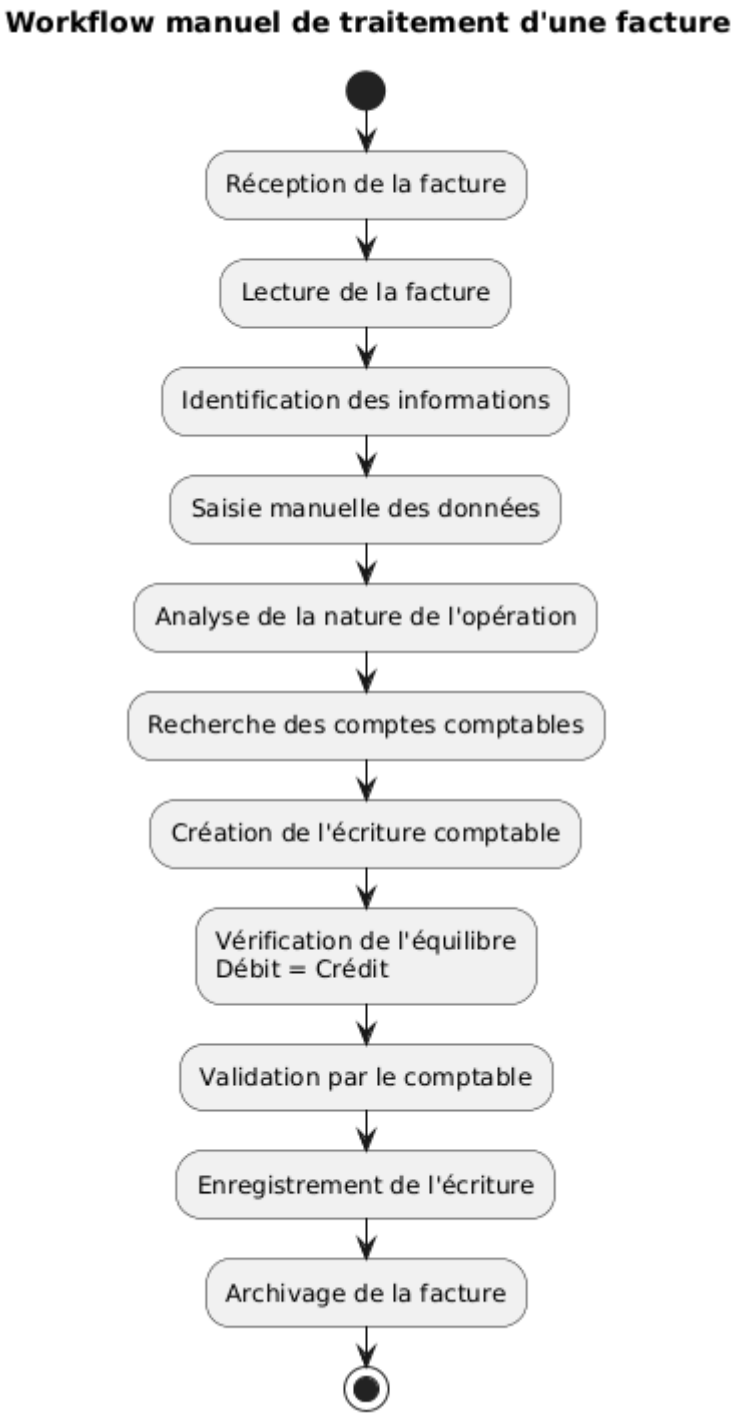


Figure 6: Workflow manuel de traitement d'une facture

c) Description de chaque nœud du workflow

Le schéma représente les différentes étapes du processus manuel utilisé avant la mise en place de l'automatisation.

- **Réception de la facture**

Le processus commence par la réception d'une facture commerciale provenant d'un fournisseur ou destinée à un client. Le document peut être reçu sous forme papier, PDF ou image.

- **Lecture de la facture**

Le comptable consulte le document afin de prendre connaissance de son contenu et d'identifier les informations nécessaires à son traitement.

- **Identification des informations**

Les principales informations sont recherchées manuellement :

  - numéro de facture ;
  - date ;
  - fournisseur ou client ;
  - désignation ;
  - quantité ;
  - prix unitaire ;
  - montant HT ;
  - TVA ;
  - montant TTC.
- **Saisie manuelle des données**

Les informations identifiées sont ensuite saisies dans le logiciel comptable ou dans le système de gestion utilisé par l'entreprise.

- **Analyse de la nature de l'opération**

Le comptable analyse le contenu de la facture afin de déterminer s'il s'agit par exemple d'un achat de marchandises, d'un achat de matériel, d'une prestation de services ou d'une vente.

- **Recherche des comptes comptables**

Une fois la nature de l'opération déterminée, les comptes appropriés sont recherchés dans le plan comptable.

- **Création de l'écriture comptable**

Le comptable crée ensuite l'écriture en renseignant les comptes, les montants au débit et au crédit ainsi que le libellé de l'opération.

- **Vérification**

L'écriture est contrôlée afin de vérifier sa cohérence et son équilibre :

Total Débit = Total Crédit

- **Validation**

Après vérification, le comptable valide définitivement l'écriture.

- **Enregistrement et archivage**

L'écriture est enregistrée dans le système comptable et la facture est conservée afin de permettre son suivi et sa consultation ultérieure.

4.1.2 Inventaire des moyens matériels et logiciels

Cette section recense les moyens matériels et logiciels effectivement utilisés pour réaliser le projet. Le tableau 12 présente le matériel.

Tableau 12: Moyens matériels utilisés pour le projet

Matériel	Caractéristiques	Système d'exploitation	Carte graphique
Ordinateur portable ASUS TUF Gaming FX505GM	Processeur Intel Core i7-8750H (6 cœurs, 12 threads, 2,20 GHz) ; 16 Go de RAM ; SSD NVMe Intel de 512 Go	Windows 11 Éducation 64 bits	NVIDIA GeForce GTX 1060 et Intel UHD Graphics 630 (intégrée)
Serveur privé virtuel (Contabo)	Serveur distant loué, hébergeant les conteneurs de production	Linux (Ubuntu)	—

Le tableau 13 présente les logiciels et bibliothèques utilisés, avec leur version.

Tableau 13: Logiciels utilisés et versions

Désignation	Version	Description
Visual Studio Code	1.145	Éditeur de code
Git / GitHub	2.50	Gestion des versions et hébergement du dépôt
Python	3.10	Langage du backend et des modules d'IA
FastAPI	0.115	Framework de l'API REST

Désignation	Version	Description
SQLAlchemy	2.0	ORM d’accès à la base de données
MySQL	8.0	SGBD en production (conteneur Docker)
MariaDB et phpMyAdmin (XAMPP)	10.4	SGBD et administration de la base sur le poste de développement
OpenCV	4.10	Prétraitement des images
PyMuPDF	1.25	Lecture des PDF et conversion en images
EasyOCR	1.7.2	Moteur OCR principal
Tesseract OCR	5.5	Moteur OCR de secours
spaCy	3.7.5	Traitement automatique du langage naturel
scikit-learn	1.5.2	Apprentissage automatique (TF-IDF, régression logistique)
XGBoost	2.1.3	Apprentissage automatique (gradient boosting)
pytest	8.3	Tests automatisés du backend
Node.js / npm	22.23 / 10.9	Environnement d’exécution et gestionnaire de paquets du frontend
React	18.3	Bibliothèque d’interface utilisateur
TypeScript	5.7	Typage statique du code frontend
Vite	6.0	Outil de construction du frontend
Tailwind CSS	3.4	Mise en forme de l’interface
Docker	29.7	Conteneurisation et déploiement
Visual Paradigm	—	Modélisation UML

Au sein du service comptable, le traitement des factures repose sur les postes informatiques du service et sur un logiciel de gestion comptable, qui permet principalement la saisie des écritures à partir des informations relevées sur les factures, ainsi que leur modification et leur correction.

Ce logiciel enregistre les écritures que le comptable lui fournit. L'identification des informations de la facture, la détermination de la nature de l'opération et le choix des comptes à utiliser restent donc des tâches effectuées manuellement par le comptable.

Les moyens actuels permettent ainsi d'assurer la saisie et la correction des écritures comptables, mais l'extraction des informations des factures et la recherche des comptes demeurent entièrement manuelles, comme le décrit l'organisation actuelle présentée précédemment. Cette situation justifie la recherche d'une solution capable d'automatiser une partie de ce travail.

4.2 Critique de l'existant

L'analyse du processus traditionnel de traitement des factures met en évidence plusieurs difficultés qui peuvent affecter la rapidité, la fiabilité et l'efficacité du travail comptable.

4.2.1 Points forts

Le processus manuel présente néanmoins des points forts. Le comptable garde la maîtrise complète de chaque écriture, son expertise lui permet de traiter les cas ambigus ou atypiques, et le logiciel de gestion comptable en place assure de manière fiable la saisie et la correction des écritures. Ces atouts doivent être préservés par la solution proposée.

4.2.2 Points faibles

L'analyse met également en évidence des points faibles, regroupés en six thèmes : le temps de traitement, le risque d'erreurs, la difficulté d'interprétation des factures, le risque de mauvaise imputation comptable, la dépendance à l'expertise humaine et les limites de la traçabilité.

Ces points faibles constituent les principales raisons justifiant la conception d'une solution intelligente capable d'automatiser une partie du processus tout en conservant une validation humaine.

4.2.3 Temps

Le traitement manuel d'une facture nécessite plusieurs opérations successives : lecture du document, identification des informations, saisie des données, analyse de l'opération, recherche des comptes et génération de l'écriture.

Lorsque l'entreprise reçoit un nombre important de factures, l'accumulation de ces opérations peut représenter une charge de travail considérable pour le comptable.

La répétition de ces tâches limite ainsi le temps que le professionnel peut consacrer à des activités nécessitant davantage d'analyse et de contrôle.

L'automatisation de l'extraction et de la génération des écritures permettrait donc de réduire considérablement le temps consacré aux tâches répétitives.

4.2.4 Risque d'erreurs

La saisie manuelle constitue une source importante d'erreurs potentielles.

Une erreur peut notamment concerner :

- le numéro de facture

- la date
- le montant HT
- le montant de la TVA
- le montant TTC
- la quantité
- le compte comptable
- le sens débit/crédit.

Par exemple, une simple erreur lors de la saisie d'un montant peut entraîner une incohérence dans l'écriture comptable.

L'utilisation d'un système OCR permettrait d'automatiser l'extraction des informations présentes dans les factures et ainsi de réduire les erreurs liées à la retranscription manuelle.

4.2.5 Difficulté d'interprétation des factures

Les factures commerciales ne présentent pas toutes la même structure.

Selon le fournisseur, une facture peut utiliser :

- une mise en page différente ;
- des intitulés différents ;
- des formats de date différents ;
- différentes positions pour les informations ;
- différents formats de tableaux ;
- différents types de documents PDF ou image.

Le système doit donc être capable de comprendre les informations indépendamment de leur position exacte dans le document.

Cette difficulté justifie l'utilisation de techniques d'OCR et de NLP, permettant respectivement d'extraire les informations et d'interpréter leur signification.

4.2.6 Risque de mauvaise imputation comptable

L'une des étapes les plus importantes du processus consiste à déterminer les comptes comptables correspondant à l'opération.

Par exemple, une facture indiquant : « Achat d'un ordinateur portable »

Doit être interprétée comme une opération correspondant à l'acquisition d'un matériel informatique et non simplement comme une dépense quelconque. Le choix du compte dépend donc de la nature de l'opération et des règles du référentiel comptable utilisé. Une mauvaise interprétation peut entraîner une mauvaise imputation comptable. Le projet devra donc combiner le Machine Learning avec un moteur de règles comptables, afin de proposer des comptes pertinents tout en vérifiant la cohérence de l'écriture.

4.2.7 Dépendance à l'expertise humaine

Le processus traditionnel dépend fortement des connaissances et de l'expérience du comptable. Deux personnes peuvent parfois interpréter différemment une opération lorsqu'une facture contient des informations ambiguës ou insuffisamment détaillées. Cette dépendance rend difficile la standardisation complète du processus. L'utilisation d'un système intelligent peut permettre de standardiser les propositions tout en conservant la possibilité pour le comptable de corriger ou de valider le résultat.

4.2.8 Limites de la traçabilité

Dans un processus fortement manuel, le suivi des différentes étapes du traitement peut être difficile lorsque les informations sont réparties entre plusieurs documents ou applications.

Il peut être nécessaire de retrouver :

- la facture originale ;
- les données extraites ;
- l'écriture générée ;
- les modifications effectuées ;
- l'utilisateur ayant effectué la modification ;
- la date de validation.

Le futur système devra donc intégrer un mécanisme d'historisation permettant de conserver les différentes étapes du traitement et d'assurer une meilleure traçabilité.

4.3 Conception avant-projet

Cette section présente les solutions envisagées pour répondre aux limites identifiées, puis les méthodes et les outils retenus pour concevoir le système.

4.3.1 Proposition des solutions

Pour remédier aux limites du traitement manuel, trois solutions ont été envisagées :

S1 – Traitement hybride : le comptable continue de saisir les écritures dans le logiciel comptable existant, avec quelques aides ponctuelles (numérisation des factures, modèles d'écritures, tableurs) ;

S2 – Acquisition d'un logiciel payant : mise en place d'un progiciel de gestion intégré (ERP) du marché, doté d'un module de comptabilité et de numérisation des factures ;

S3 – Développement du système proposé : une application qui lit automatiquement la facture (OCR), en extrait les informations (NLP), propose les comptes (Machine Learning) et génère une écriture contrôlée par un moteur de règles du PCG 2005, soumise à la validation du comptable.

Le tableau 14 compare ces solutions selon leurs avantages et leurs inconvénients.

Tableau 14: Comparaison des solutions envisagées

Solution	Avantages	Inconvénients
S1 – Traitement hybride	Coût quasi nul ; aucune nouvelle compétence requise ; maîtrise totale du comptable	Traitement toujours lent et répétitif ; risque d'erreurs de saisie ; dépendance à l'expérience du comptable ; traçabilité limitée
S2 – Logiciel ERP payant	Solution éprouvée couvrant l'ensemble de la gestion ; support de l'éditeur	Coût élevé des licences, du paramétrage et de la maintenance ; adaptation limitée au PCG malgache et aux factures locales (français, malgache, ariary) ; dépendance vis-à-vis de l'éditeur
S3 – Système intelligent proposé	Automatisation de la lecture, de l'extraction et de l'imputation ; adapté au PCG 2005 et aux factures locales ; validation humaine conservée ; amélioration continue grâce aux corrections ; technologies libres et coût maîtrisé	Temps de développement important ; besoin d'un volume suffisant de factures validées pour entraîner le modèle ; précision de l'OCR dépendante de la qualité des documents

Solution retenue

La solution S3 est retenue. C'est la seule qui automatise l'extraction des informations et la proposition des écritures tout en restant adaptée au contexte malgache. Elle conserve aussi la maîtrise humaine du processus : chaque proposition reste soumise à la validation du comptable avant toute comptabilisation définitive, ce qui réduit le temps de saisie et les risques d'erreur sans retirer au comptable sa responsabilité.

4.3.2 Méthodes de conception et outils utilisés

La conception du système s'appuie sur la méthodologie agile SCRUM, retenue pour sa capacité à livrer régulièrement des incréments fonctionnels testables et à intégrer les retours au fil des sprints

(détail des rôles et du backlog au chapitre 5). La modélisation UML (cas d'utilisation, classes, paquetages, déploiement) est réalisée avec Visual Paradigm. Ce choix méthodologique permet une conception progressive, en cohérence avec la nature exploratoire de l'intégration d'un module d'intelligence artificielle dans l'application.

Le tableau 15 présente, pour chaque besoin, l'outil retenu, les alternatives envisagées et la justification du choix.

Tableau 15: Choix et justification des outils

Besoin	Outil retenu	Alternatives	Justification du choix	Réf.
Gestion de projet	Scrum	Cycle en V, Kanban	Livraisons incrémentales et testables ; adapté à un projet exploratoire intégrant de l'IA	[23]
Modélisation	UML (Visual Paradigm)	Merise	Langage standard couvrant les vues statique, dynamique et de déploiement	[3], [28]
Langage de développement (backend et IA)	Python	Java, JavaScript (Node.js)	Écosystème IA le plus riche (OpenCV, EasyOCR, spaCy, scikit-learn)	[15]
Framework API	FastAPI	Flask, Django	Performances, validation automatique des données, documentation OpenAPI générée	[17]
Interface web	React + TypeScript	Vue.js, Angular	Composants réutilisables, large communauté, typage statique limitant les erreurs	[21], [27]
Traitement d'image et OCR	OpenCV + EasyOCR (Tesseract en secours)	Services OCR en ligne payants	Exécution locale et gratuite, multilingue, données non transmises à un tiers	[1], [8], [14], [19]
NLP	Expressions régulières + spaCy	Modèles Transformers	Léger, explicable, sans carte graphique ; adapté aux entités à format fixe	[24]
Modèle / algorithme	Mots-clés, puis TF-IDF + régression logistique et XGBoost	Réseaux de neurones profonds	Efficace sur de petits volumes de données, interprétable, rapide à réentraîner	[2], [4], [7]
Environnement d'exécution	Docker et Docker Compose sur un VPS Linux	Installation manuelle sur le serveur	Déploiement reproductible et isolé	[13]
SGBD	MySQL 8	PostgreSQL	Transactions fiables (ACID), contraintes d'intégrité, outils d'administration disponibles	[12]

CHAPITRE 5 : Analyse conceptuelle

Ce chapitre présente la modélisation du système selon la méthode Scrum : l'organisation de l'équipe, les backlogs du produit et des sprints, le dictionnaire des données, les règles de gestion et les différents diagrammes UML.

5.1 Désignation des rôles de l'équipe SCRUM

Dans le cadre de ce projet de fin d'études, la réalisation du système a été effectuée individuellement. Afin de conserver une organisation structurée du développement tout en respectant les principes de la méthodologie Scrum [23], les différents rôles Scrum ont été assumés par le même étudiant.

Ainsi, les responsabilités du Product Owner, du Scrum Master et de l'équipe de développement sont regroupées au sein d'une seule personne.

5.1.1 Rôle de Product Owner

Dans le cadre de ce projet individuel, l'étudiant assure le rôle de Product Owner. Il est chargé de:

- identifier et analyser les besoins des utilisateurs ;
- définir les fonctionnalités du système ;
- prioriser les fonctionnalités ;
- élaborer le Product Backlog ;
- définir les critères d'acceptation ;
- vérifier la conformité des fonctionnalités développées avec les besoins identifiés.

5.1.2 Rôle de Scrum Master

L'étudiant assure également le rôle de Scrum Master. Il veille principalement à :

- organiser les différentes étapes du développement ;
- planifier les sprints ;
- suivre l'avancement du projet ;
- identifier les difficultés rencontrées ;
- respecter les objectifs définis pour chaque sprint.

5.1.3 Rôle de développeur

Enfin, l'étudiant assume le rôle de développeur et prend en charge l'ensemble des activités techniques du projet, notamment :

- la conception de la base de données MySQL ;
- le développement du frontend avec React et TypeScript ;
- le développement du backend avec FastAPI et Python ;
- l'intégration du module OCR ;

- le développement du traitement NLP ;
- l'intégration et l'entraînement du modèle Machine Learning ;
- la conception du moteur de règles comptables ;
- l'intégration de l'IA avec l'application ;
- les tests et la correction des anomalies ;
- la préparation du déploiement.

Cette organisation permet d'appliquer les principes de Scrum dans le cadre d'un projet individuel, tout en assurant une gestion structurée et progressive des différentes fonctionnalités du système.

5.2 Étapes de l'élaboration du Product Backlog

Après l'analyse de l'existant et l'identification des besoins des utilisateurs, cette étape consiste à traduire les fonctionnalités attendues du système en éléments de travail pouvant être planifiés et développés progressivement. Dans le cadre de ce projet, la méthode Scrum est utilisée afin d'organiser le développement du système intelligent de génération automatique des écritures comptables.

Le Product Backlog constitue l'ensemble des fonctionnalités, besoins et exigences identifiés pour le système. Son élaboration repose d'abord sur l'analyse des besoins fonctionnels et non fonctionnels. Les besoins fonctionnels correspondent notamment à l'importation des factures, à l'extraction automatique des informations, à leur analyse, à la proposition des comptes comptables et à la génération des écritures. Les besoins non fonctionnels concernent principalement la fiabilité, la sécurité, la traçabilité, la facilité d'utilisation et les performances du système.

Dans un premier temps, les utilisateurs du système et leurs principales interactions avec l'application sont identifiés. Le système doit notamment permettre à un utilisateur autorisé d'importer une facture commerciale sous forme d'image ou de fichier PDF. Après son importation, la facture est soumise à un traitement automatique permettant d'extraire les informations nécessaires à son exploitation comptable.

Les fonctionnalités sont ensuite formulées sous forme d'User Stories. Cette formulation permet de représenter les besoins du point de vue de l'utilisateur. Par exemple, une fonctionnalité peut être exprimée ainsi : « En tant que comptable, je souhaite importer une facture afin que le système puisse extraire automatiquement les informations nécessaires à sa comptabilisation. »

Pour chaque User Story, la validation consiste à déterminer si la fonctionnalité développée répond effectivement au besoin exprimé. Cette étape est particulièrement importante pour les fonctionnalités liées à l'intelligence artificielle, car le résultat proposé par le système doit pouvoir être contrôlé par le professionnel.

Les User Stories sont ensuite priorisées selon leur importance pour le fonctionnement global de l'application. Les fonctionnalités constituant le cœur du système, notamment l'importation des factures, l'OCR, l'analyse intelligente et la génération des écritures comptables, sont considérées comme prioritaires.

Le Product Backlog ainsi constitué sert alors de base à la planification des différents sprints du projet.

5.3 Product Backlog

Le Product Backlog regroupe les fonctionnalités nécessaires à la réalisation du système. Il est organisé selon leur priorité et leur contribution au fonctionnement global de l'application.

Dans le cadre de ce projet, le développement commence par les fonctionnalités générales permettant l'accès à l'application et la gestion des factures. Les fonctionnalités d'intelligence artificielle sont ensuite introduites progressivement avant d'intégrer le processus de génération et de validation des écritures comptables.

Le tableau 16 présente le Product Backlog du projet, avec la priorité attribuée à chaque User Story.

Tableau 16: Product Backlog

ID	User Story	Description	Priorité
US01	Authentification	Permettre à l'utilisateur de se connecter au système	Haute
US02	Importation d'une facture	Permettre l'importation d'une facture PDF ou image	Haute
US03	Consultation d'une facture	Permettre de visualiser les informations d'une facture	Haute
US04	Extraction OCR	Extraire automatiquement le texte et les informations de la facture	Très haute
US05	Analyse du contenu	Identifier la nature et le contenu de l'opération commerciale	Très haute
US06	Proposition des comptes	Proposer les comptes comptables correspondant à l'opération	Très haute

US07	Génération de l'écriture	Générer automatiquement une écriture comptable	Très haute
US08	Vérification de l'équilibre	Vérifier que le total débit est égal au total crédit	Très haute
US09	Modification de l'écriture	Permettre au comptable de corriger l'écriture proposée	Haute
US10	Validation ou rejet	Permettre au comptable de valider ou rejeter une proposition	Haute
US11	Historique	Conserver l'historique des traitements et validations	Moyenne
US12	Apprentissage à partir des corrections	Conserver les corrections validées pour améliorer le modèle	Moyenne

5.4 Sprint Backlog

Le Sprint Backlog correspond à l'ensemble des fonctionnalités sélectionnées dans le Product Backlog pour être réalisées au cours d'un sprint. Dans le contexte de ce projet, les fonctionnalités sont réparties progressivement afin de construire le système par étapes cohérentes.

Le découpage tient compte des dépendances techniques entre les différents modules. Par exemple, la génération automatique d'une écriture comptable ne peut être correctement réalisée avant que les informations nécessaires aient été extraites de la facture et interprétées.

Le projet est ainsi organisé en plusieurs releases comprenant chacune plusieurs sprints.

5.4.1 Présentation du travail à faire

Le tableau 17 présente la répartition des fonctionnalités en releases et en sprints.

Tableau 17: Présentation du travail à faire (releases et sprints)

Release	Sprint	Fonctionnalités principales
Release 1	Sprint 1	Authentification et gestion des utilisateurs
Release 1	Sprint 2	Importation et gestion des factures
Release 2	Sprint 3	Prétraitement et extraction OCR
Release 2	Sprint 4	Analyse NLP et classification par Machine Learning

Release 3	Sprint 5	Génération des écritures comptables et règles comptables
Release 3	Sprint 6	Validation, correction et historique
Release 4	Sprint 7	Intégration complète et tests

Cette organisation permet de construire progressivement le système. Les premiers sprints mettent en place les fonctionnalités générales nécessaires à l'application. Les sprints 3 à 6 introduisent progressivement les différents composants de l'intelligence artificielle et de la comptabilité.

Le dernier sprint est consacré à l'intégration globale, aux tests et à la vérification du fonctionnement de l'ensemble des modules.

5.4.2 Planification du sprint

La planification permet de transformer les User Stories sélectionnées en tâches concrètes de développement. Chaque sprint possède ainsi un objectif précis.

Le tableau 18 détaille l'objectif et les principales tâches de chaque sprint.

Tableau 18: Planification du sprint (tâches par sprint)

Sprint	Objectif	Principales tâches
Sprint 1	Gestion des accès	Base utilisateur, rôles, authentification, autorisation
Sprint 2	Gestion des factures	Import PDF/image, stockage, consultation, suppression
Sprint 3	Extraction	Prétraitement image, OCR, extraction des informations
Sprint 4	Analyse intelligente	NLP, classification, prédiction des comptes, score de confiance
Sprint 5	Comptabilisation	Plan comptable, règles comptables, génération des écritures, équilibre débit/crédit
Sprint 6	Contrôle	Modification, validation, rejet, historique et corrections
Sprint 7	Finalisation	Intégration, tests, correction des anomalies et évaluation

Cette planification permet de respecter les dépendances entre les différentes fonctionnalités. Elle facilite également le suivi de l'avancement du projet et permet d'identifier plus facilement les éventuelles difficultés rencontrées pendant le développement.

5.4.3 Points forts du workflow

- Robustesse : chaque étape (OCR, extraction, classification, règles) est isolée derrière un port du domaine, avec gestion explicite des erreurs et des seuils de rejet
- Performance : EasyOCR traite la majorité des factures, avec un repli automatique sur Tesseract lorsque la confiance est insuffisante
- Flexibilité : chaque adaptateur (OCR, ML, base de données) est remplaçable sans modifier le domaine, grâce à l'architecture en couches
- Scalabilité : le traitement des factures s'exécute en tâche de fond, sans bloquer l'utilisateur pendant l'import
- Traçabilité : journal d'audit en append-only et confiances par champ conservées pour chaque facture traitée

5.5 Dictionnaire des données

Le dictionnaire des données permet de décrire les principales informations manipulées par le système. Il constitue une référence commune pour la conception de la base de données et le développement des différentes fonctionnalités.

Dans le système proposé, les données sont principalement liées aux utilisateurs, aux factures, aux informations extraites, aux résultats de l'analyse intelligente et aux écritures comptables.

Le tableau 19 présente ces données, triées par ordre alphabétique.

Tableau 19: Dictionnaire des données

Rubrique	Description	Type	Taille	Format
commentaire	Commentaire du comptable lors de la validation	AN	500	Texte libre
dateEcriture	Date de l'écriture	D	10	JJ/MM/AAAA
dateFacture	Date d'émission de la facture	D	10	JJ/MM/AAAA
decision	Décision du comptable	AN	10	VALIDÉE / REJETÉE
email	Adresse électronique de l'utilisateur	AN	255	nom@domaine
idEcriture	Identifiant de l'écriture	N	20	Entier
idFacture	Identifiant de la facture	N	20	Entier
idUtilisateur	Identifiant de l'utilisateur	N	20	Entier
libelle	Libellé du compte	AN	255	Texte
montantCredit	Montant porté au crédit d'une ligne	N	15,2	Décimal

Rubrique	Description	Type	Taille	Format
montantDebit	Montant porté au débit d'une ligne	N	15,2	Décimal
montantHT	Montant hors taxes de la facture	N	15,2	Décimal
montantTTC	Montant toutes taxes comprises	N	15,2	Décimal
montantTVA	Montant de la TVA	N	15,2	Décimal
natureOperation	Nature de l'opération identifiée	AN	30	Achat de marchandises, de services, d'immobilisation, autre
nom	Nom de l'utilisateur	AN	150	Texte
numeroCompte	Numéro du compte comptable	AN	10	Numéro du PCG (ex. 6061)
numeroFacture	Numéro de la facture	AN	100	Texte
role	Rôle attribué à l'utilisateur	AN	20	Administrateur, comptable
scoreConfiance	Niveau de confiance de la prédiction	N	5,4	Décimal entre 0 et 1
statut (écriture)	État de validation de l'écriture	AN	10	Proposée, corrigée, validée
statut (facture)	État du traitement de la facture	AN	20	Importée, en cours, analysée, validée, rejetée, en erreur
texteExtrait	Texte obtenu après OCR	AN	16 Mo	Texte libre
totalCredit	Total des montants au crédit de l'écriture	N	15,2	Décimal
totalDebit	Total des montants au débit de l'écriture	N	15,2	Décimal
typeFacture	Type de facture	AN	10	ACHAT / VENTE

Types : AN = alphanumérique, N = numérique, D = date. Pour les montants, la taille 15,2 signifie 15 chiffres au total, dont 2 après la virgule.

Ce dictionnaire constitue une base pour la conception de la base de données MySQL. Il pourra être complété au cours de la réalisation lorsque de nouvelles données nécessaires au fonctionnement de l'application seront identifiées.

5.6 Règles de gestion

Les règles de gestion définissent les contraintes que le système doit respecter afin de garantir la cohérence du traitement des factures et des écritures comptables.

Le système respecte les règles de gestion suivantes :

- RG1 : une facture n'est acceptée que si elle est au format PDF, JPEG ou PNG et si sa taille ne dépasse pas 15 Mo.

- RG2 : une entreprise doit avoir choisi son plan comptable avant de pouvoir importer des factures.
- RG3 : toute facture importée est d'abord enregistrée et rattachée à une entreprise et à l'utilisateur qui l'a déposée, avant tout traitement.
- RG4 : chaque facture est soumise au traitement OCR ; si la confiance de la lecture est inférieure à 0,55, le moteur de secours est utilisé, et en dessous de 0,40 la facture est signalée comme illisible.
- RG5 : une facture donne lieu à une seule écriture comptable proposée.
- RG6 : une écriture n'est équilibrée que si le total des débits est égal au total des crédits.
- RG7 : chaque ligne d'écriture porte un montant soit au débit, soit au crédit, jamais les deux ni aucun.
- RG8 : chaque ligne est imputée à un compte du plan comptable de l'entreprise ; le compte proposé par le modèle doit appartenir à la liste des comptes autorisés par la règle comptable.
- RG9 : une facture en devise étrangère est convertie dans la devise de l'entreprise au taux de change applicable à la date de la facture.
- RG10 : l'écriture proposée par l'intelligence artificielle n'est définitive qu'après validation par un comptable, qui peut aussi la corriger ou la rejeter.
- RG11 : seule une facture dont l'écriture n'est pas encore validée peut être supprimée.
- RG12 : toute correction conserve la valeur d'origine, la nouvelle valeur, son auteur et sa date ; elle sert ensuite de donnée d'apprentissage.
- RG13 : un nouveau modèle n'est mis en service que s'il a été entraîné sur au moins 30 exemples couvrant au moins 2 classes et si son score F1 est au moins égal à celui du modèle actif.
- RG14 : toute action (import, traitement, correction, validation, rejet, gestion des utilisateurs) est enregistrée dans un historique qui ne peut être ni modifié ni supprimé.
- RG15 : un utilisateur n'accède qu'aux factures de son entreprise ; la gestion des utilisateurs est réservée aux administrateurs.

5.7 Diagramme de cas d'utilisation global

Le diagramme de cas d'utilisation permet de représenter les interactions entre les acteurs et le système. Il permet notamment d'identifier les principales fonctionnalités accessibles aux différents profils d'utilisateurs.

Le système fait intervenir trois acteurs principaux :

- L’Utilisateur : il accède aux fonctionnalités générales de l’application (connexion, import et consultation des factures) et lance le traitement automatique ;
- Le Comptable : en plus des fonctionnalités de l’utilisateur, il contrôle, modifie, valide ou rejette les écritures proposées par le système ;
- L’Administrateur : il gère les utilisateurs et leurs rôles, consulte l’historique et lance le réentraînement du modèle.

Le Comptable et l’Administrateur héritent des fonctionnalités de l’Utilisateur : cette relation est représentée dans la figure 7 par une généralisation entre acteurs, ce qui évite de répéter les mêmes associations pour chacun d’eux.

Le traitement automatique constitue une fonctionnalité centrale. Il comprend successivement le prétraitement du document, l’extraction OCR, l’analyse NLP, l’identification de la nature de l’opération, la proposition des comptes comptables et la génération de l’écriture.

La vérification de l’équilibre débit/crédit et l’application des règles comptables interviennent ensuite afin de contrôler la cohérence de l’écriture générée.

La figure 7 présente le diagramme de cas d’utilisation global du système.

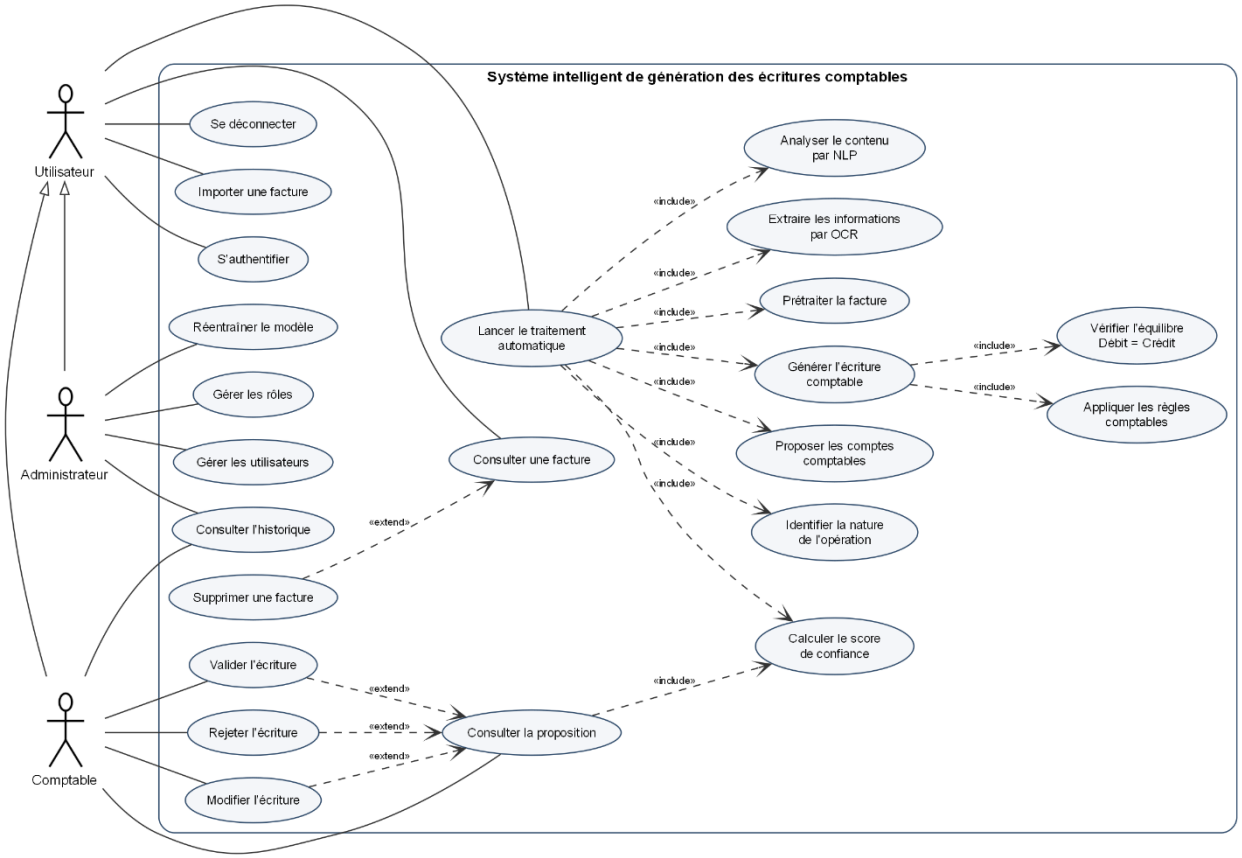


Figure 7: Diagramme de cas d'utilisation global

5.8 Modèle du domaine :

Le modèle du domaine représente les principales entités métier intervenant dans le fonctionnement du système. Il permet de comprendre la structure conceptuelle de l'application indépendamment des technologies utilisées pour son implémentation.

La Facture constitue l'entité centrale du domaine. Elle est associée à un fournisseur dans le cas d'une facture d'achat ou à un client dans le cas d'une facture de vente. Après son importation, elle fait l'objet d'un traitement OCR permettant d'obtenir les informations nécessaires à son analyse.

Les informations extraites sont ensuite utilisées par le module d'analyse intelligente. Celui-ci détermine la nature de l'opération et produit des prédictions concernant les comptes comptables à utiliser.

Les comptes proposés sont confrontés aux règles comptables avant de permettre la génération d'une ÉcritureComptable. Cette dernière est composée d'une ou plusieurs lignes d'écriture, chacune étant associée à un compte comptable.

Enfin, l'écriture peut faire l'objet d'une validation par le comptable. Les modifications et validations sont conservées dans l'historique afin d'assurer la traçabilité du processus.

Le modèle traduit les règles de gestion de la section précédente : chaque entreprise choisit un plan comptable (RG2) ; une facture, rattachée à une entreprise et à l'utilisateur qui l'a importée (RG3), est émise par un seul fournisseur et donne lieu à au plus une écriture (RG5) ; cette écriture se compose d'au moins deux lignes (RG6, RG7), chacune imputée sur un compte du plan comptable autorisé par une règle comptable (RG8) ; une facture en devise étrangère est convertie selon un taux de change (RG9) ; enfin, les validations, les corrections et l'historique conservent l'auteur et la date de chaque action (RG10, RG12, RG14), et les corrections alimentent l'apprentissage du modèle (RG13).

La figure 8 présente le modèle du domaine.

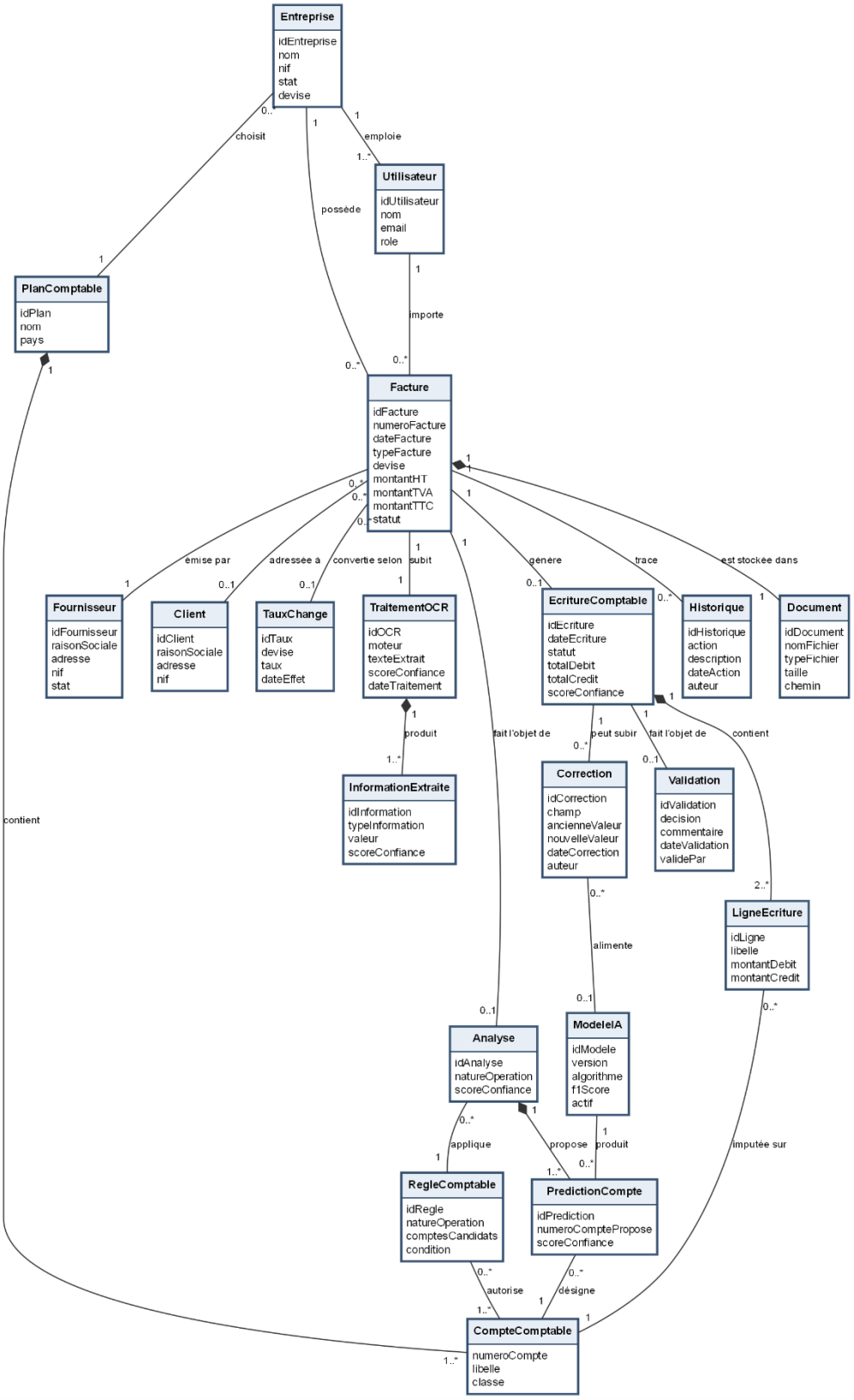


Figure 8: Modèle du domaine

5.9 Diagramme de classe de conception pour chaque Sprint :

La conception des classes est réalisée progressivement afin de suivre l'évolution fonctionnelle du système au cours des différents sprints. Chaque sprint introduit les classes nécessaires aux fonctionnalités développées.

5.9.1 Diagramme de classes de conception du sprint 1

Le premier sprint concerne principalement l'authentification et la gestion des utilisateurs. Les principales classes sont Utilisateur, Role, ainsi que les composants permettant de gérer l'authentification et les autorisations.

La figure 9 présente le diagramme de classes de conception du sprint 1.

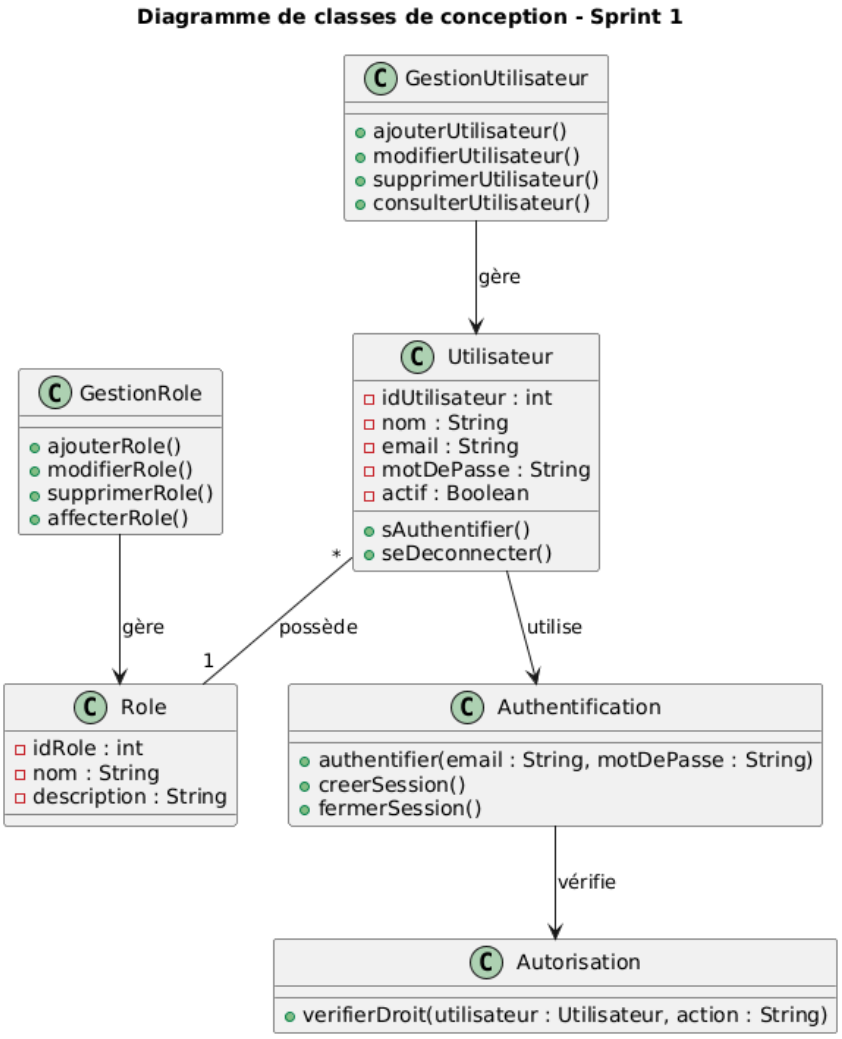


Figure 9: Diagramme de classes de conception – Sprint 1

5.9.2 Diagramme de classes de conception du sprint 2

Le deuxième sprint introduit la gestion des factures. Les classes Facture, Fournisseur et Client permettent de représenter les documents commerciaux et leurs principales parties prenantes.

La figure 10 présente le diagramme de classes de conception du sprint 2.

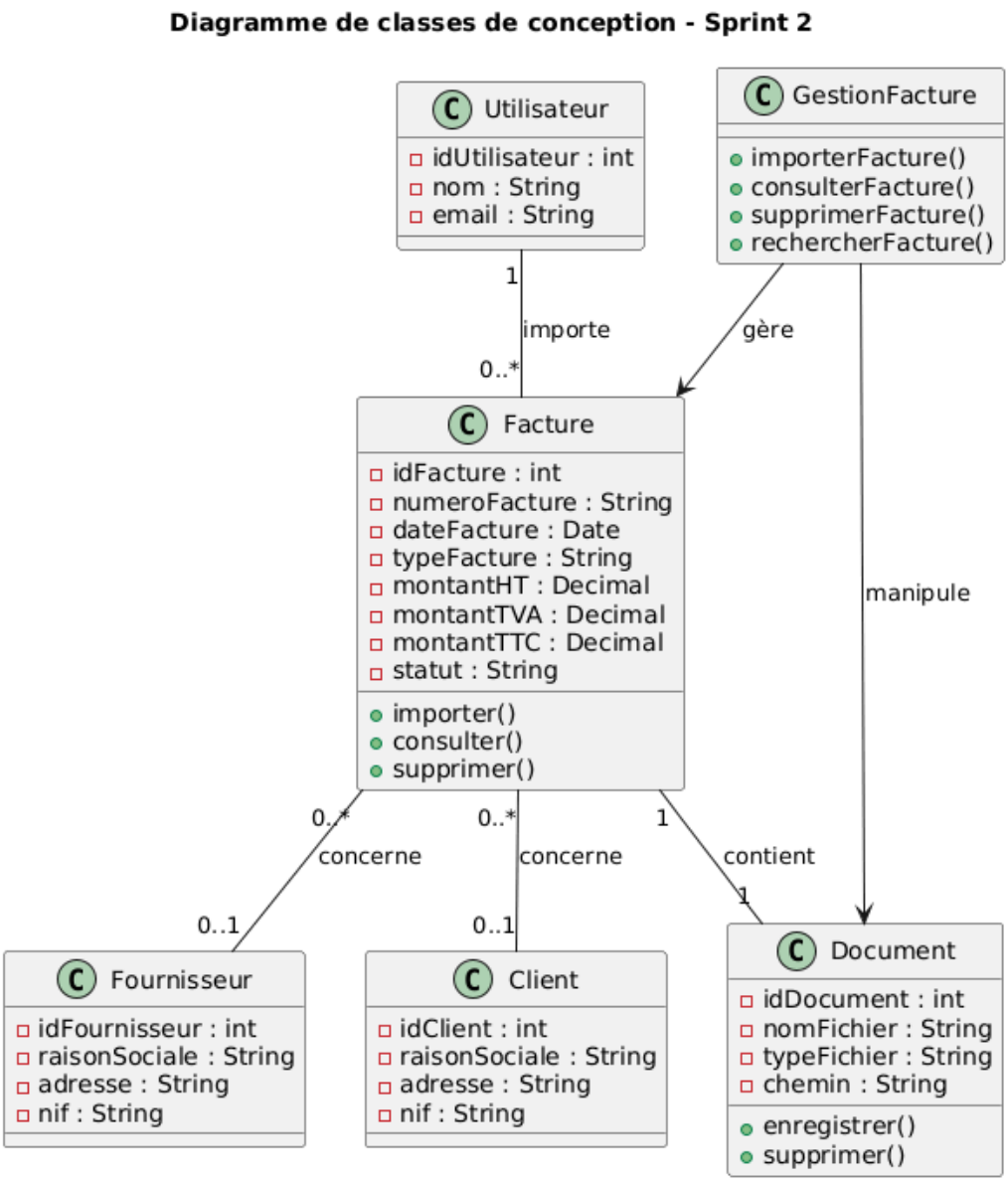


Figure 10: Diagramme de classes de conception – Sprint 2

5.9.3 Diagramme de classes de conception du sprint 3

Le troisième sprint est consacré au traitement OCR. Les classes liées au document, au résultat OCR et aux informations extraites permettent de représenter le résultat du traitement automatique du document.

La figure 11 présente le diagramme de classes de conception du sprint 3.

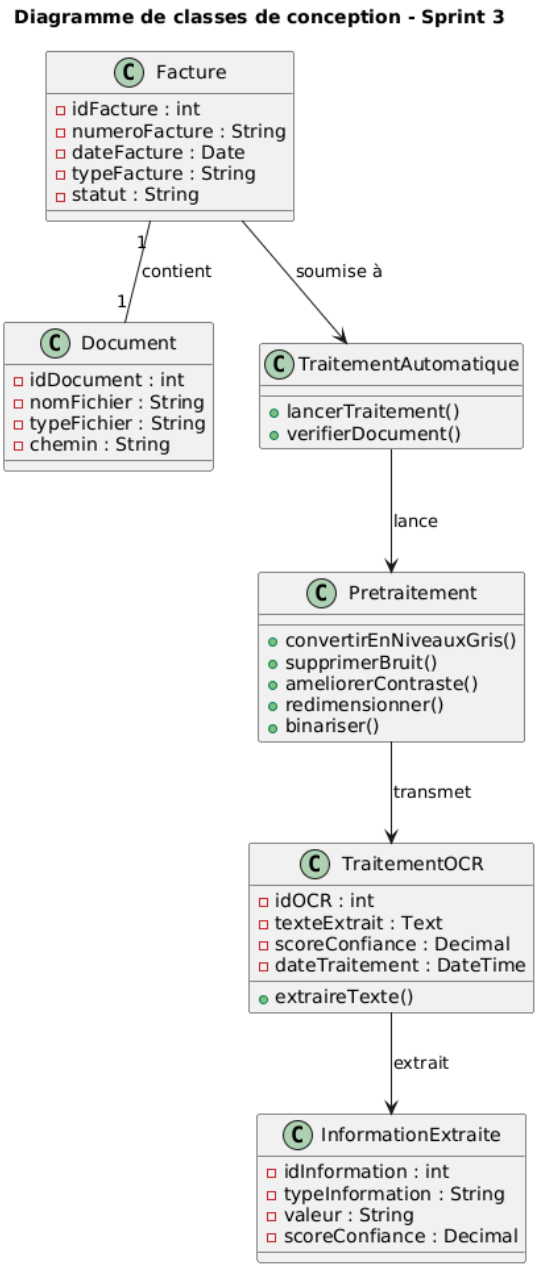


Figure 11: Diagramme de classes de conception – Sprint 3

5.9.4 Diagramme de classes de conception du sprint 4

Le quatrième sprint introduit les composants d'intelligence artificielle permettant d'analyser les informations extraites. Les classes relatives à l'analyse, aux prédictions et au modèle Machine Learning sont alors intégrées.

La figure 12 présente le diagramme de classes de conception du sprint 4.

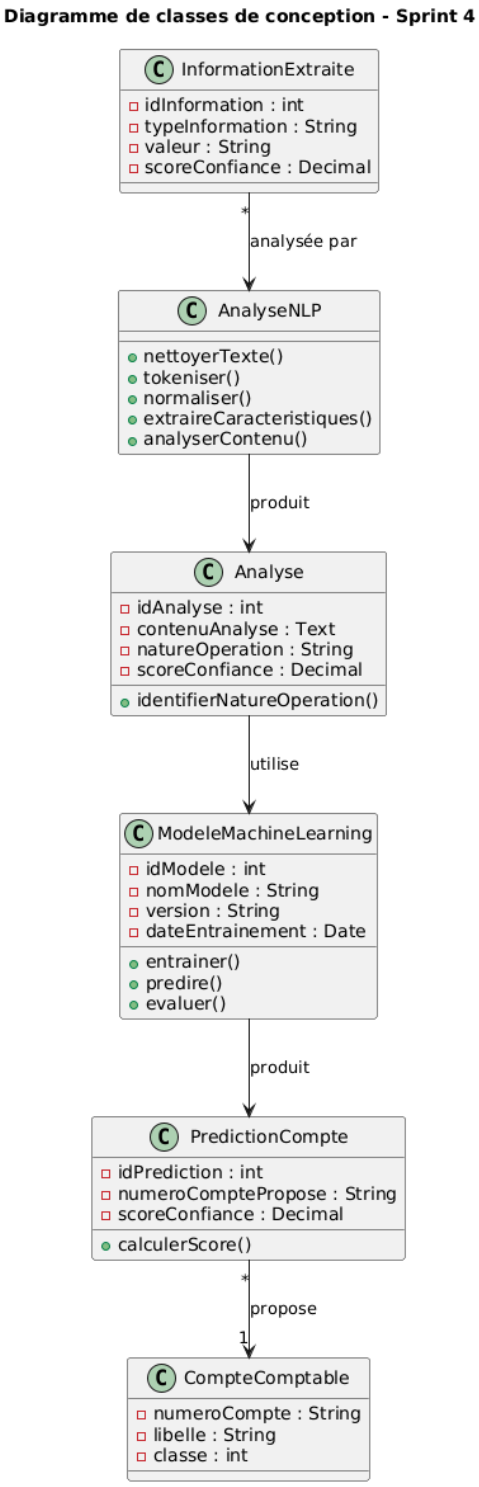


Figure 12: Diagramme de classes de conception – Sprint 4

5.9.5 Diagramme de classes de conception du sprint 5

Le cinquième sprint constitue la partie comptable du système. Il introduit notamment CompteComptable, RegleComptable, EcritureComptable, LigneEcriture et MoteurComptable.

La figure 13 présente le diagramme de classes de conception du sprint 5.

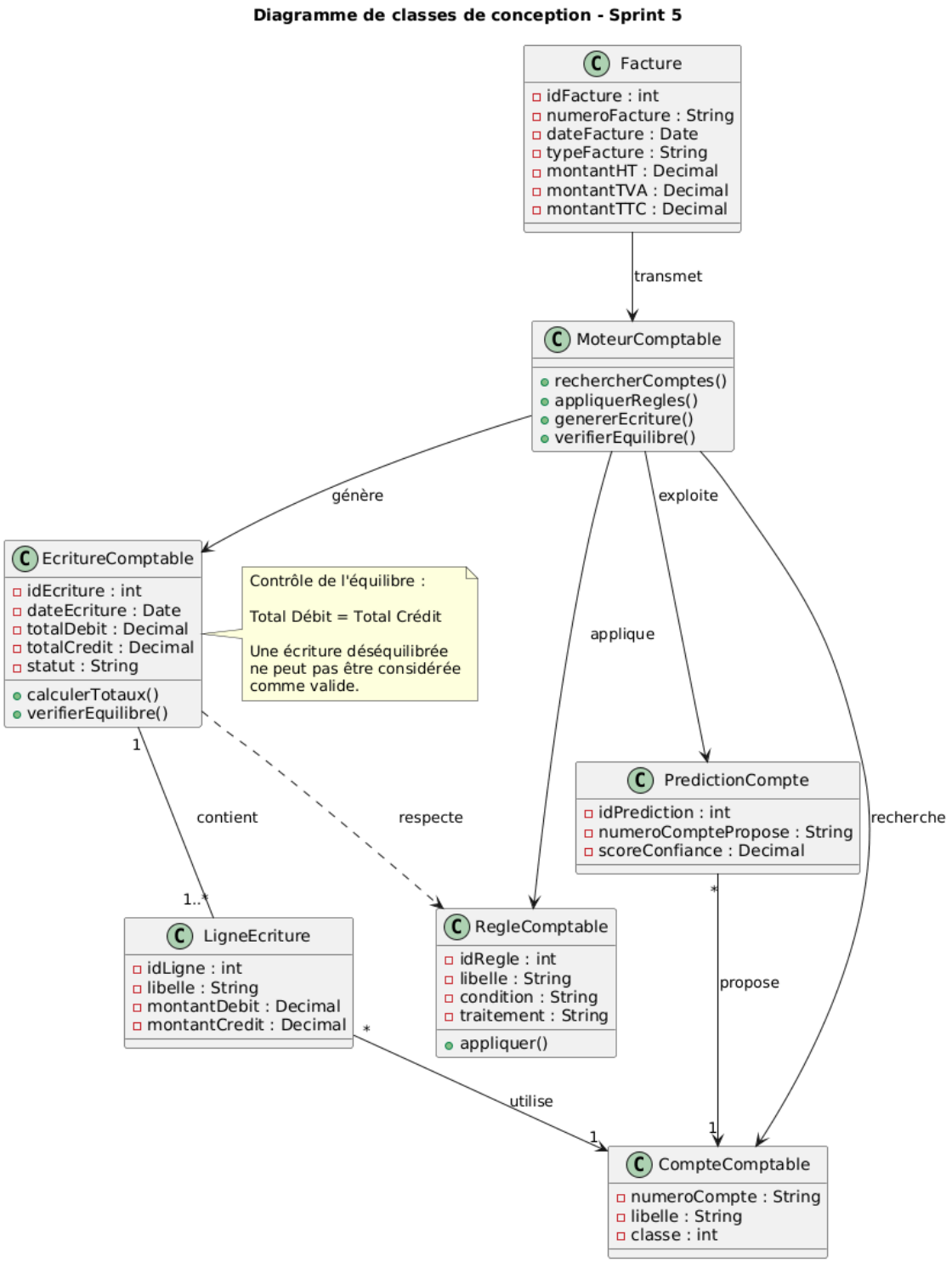


Figure 13: Diagramme de classes de conception – Sprint 5

5.9.6 Diagramme de classes de conception du sprint 6

Le sixième sprint ajoute les fonctionnalités de contrôle humain. Les classes Validation, Historique et Correction permettent de conserver les décisions et les modifications effectuées par le comptable.

La figure 14 présente le diagramme de classes de conception du sprint 6.

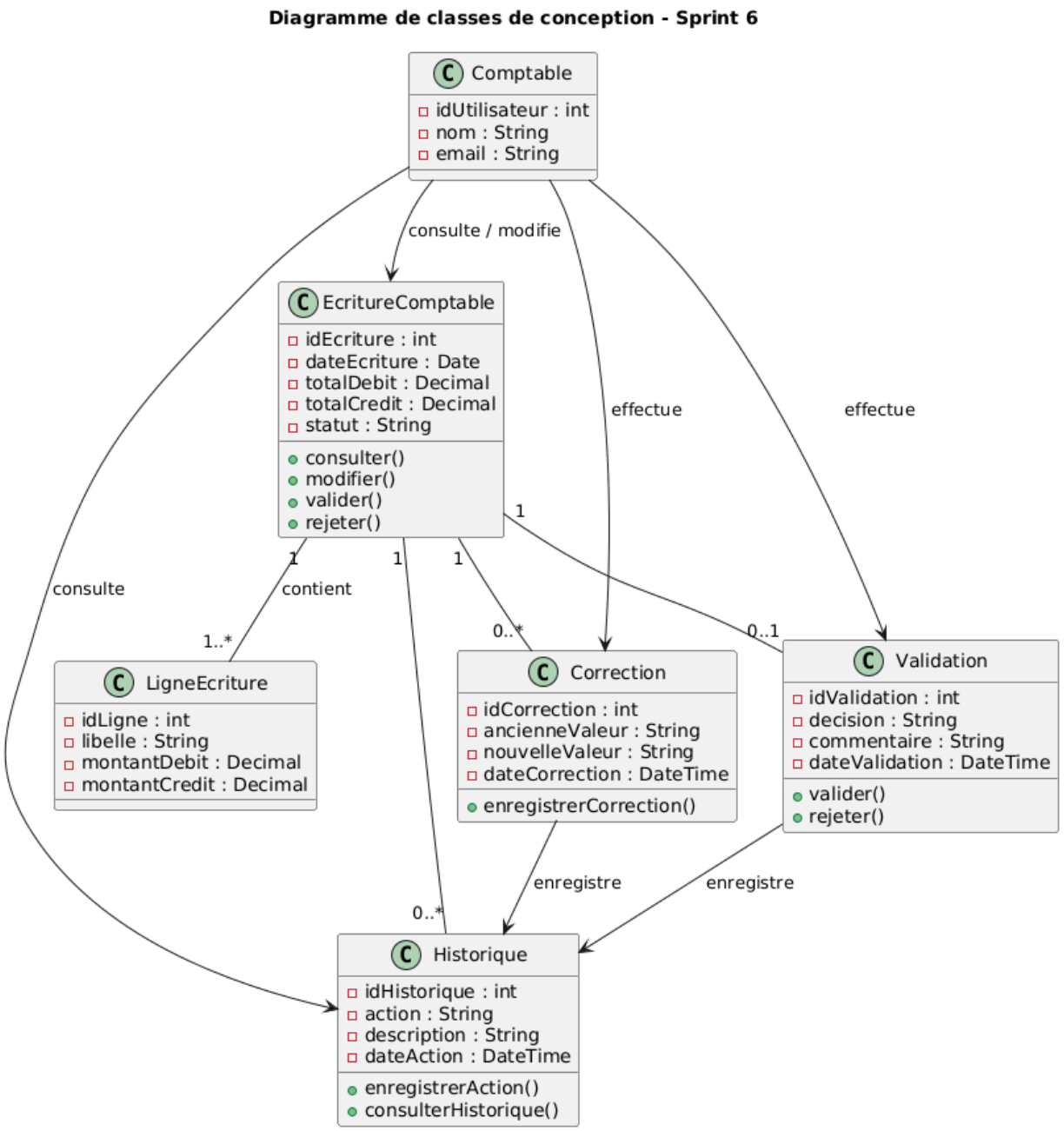


Figure 14: Diagramme de classes de conception – Sprint 6

5.9.7 Diagramme de classes de conception du sprint 7

Le dernier sprint est principalement consacré à l'intégration des différents composants. Les classes développées précédemment sont mises en relation afin de constituer le système complet.

La figure 15 présente le diagramme de classes de conception du sprint 7.

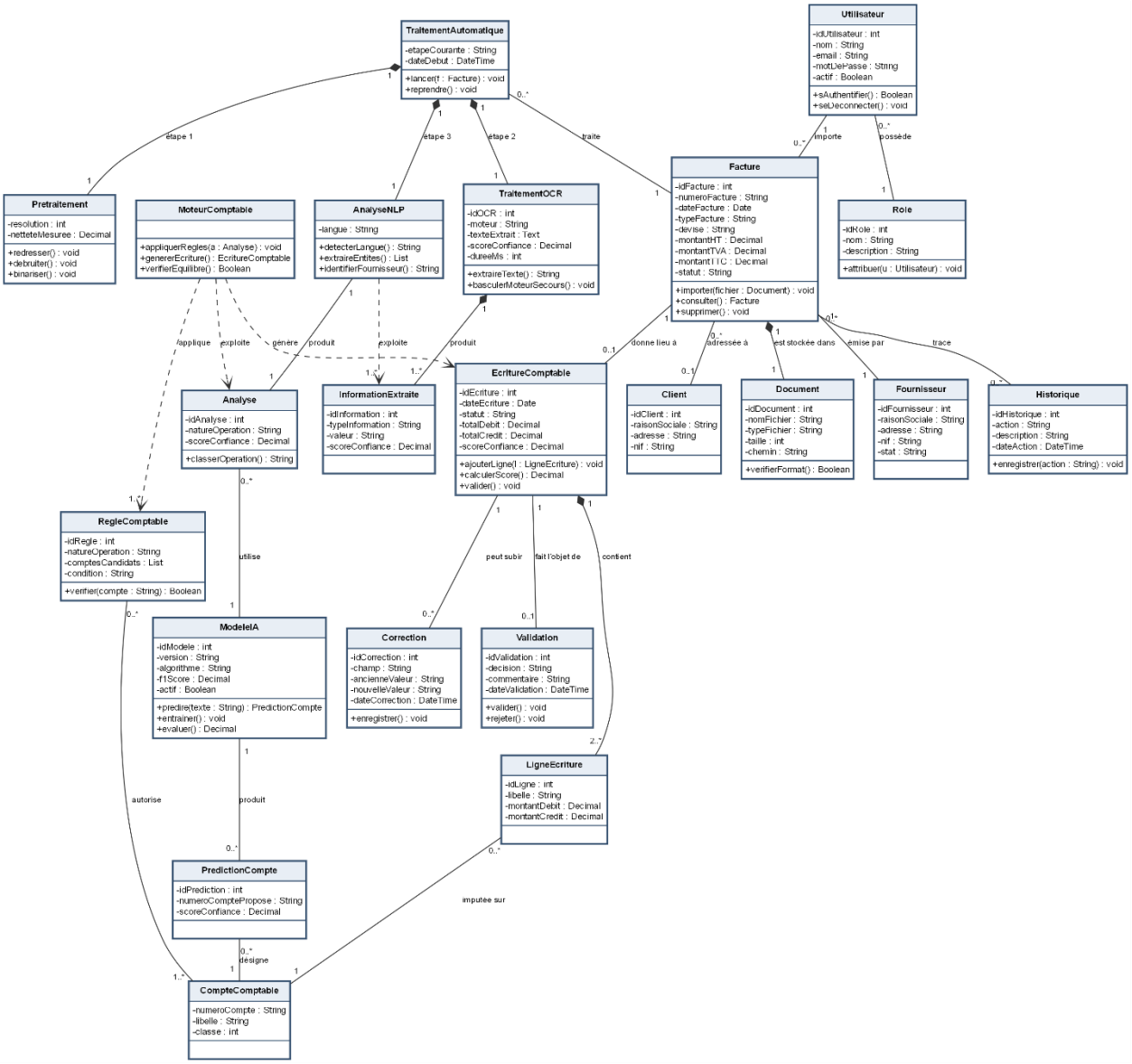


Figure 15: Diagramme de classes de conception – Sprint 7

5.10 Diagramme de classe de conception globale :

Le diagramme de classes global regroupe les différentes classes identifiées pendant les sprints. Il représente la structure générale du système et les relations entre les différents composants métier.

Il permet notamment de visualiser le parcours d'une facture depuis son importation jusqu'à la génération et à la validation de l'écriture comptable.

La classe Facture est ainsi liée au traitement OCR, puis aux informations extraites et à l'analyse intelligente. Les prédictions issues du modèle Machine Learning permettent de proposer des comptes comptables. Le MoteurComptable intervient ensuite pour appliquer les règles comptables et générer une écriture équilibrée.

L’écriture générée peut finalement être modifiée et validée par le comptable. Les corrections réalisées peuvent également être conservées afin de constituer progressivement des données utiles à l’amélioration du modèle.

La figure 16 présente le diagramme de classes de conception globale.

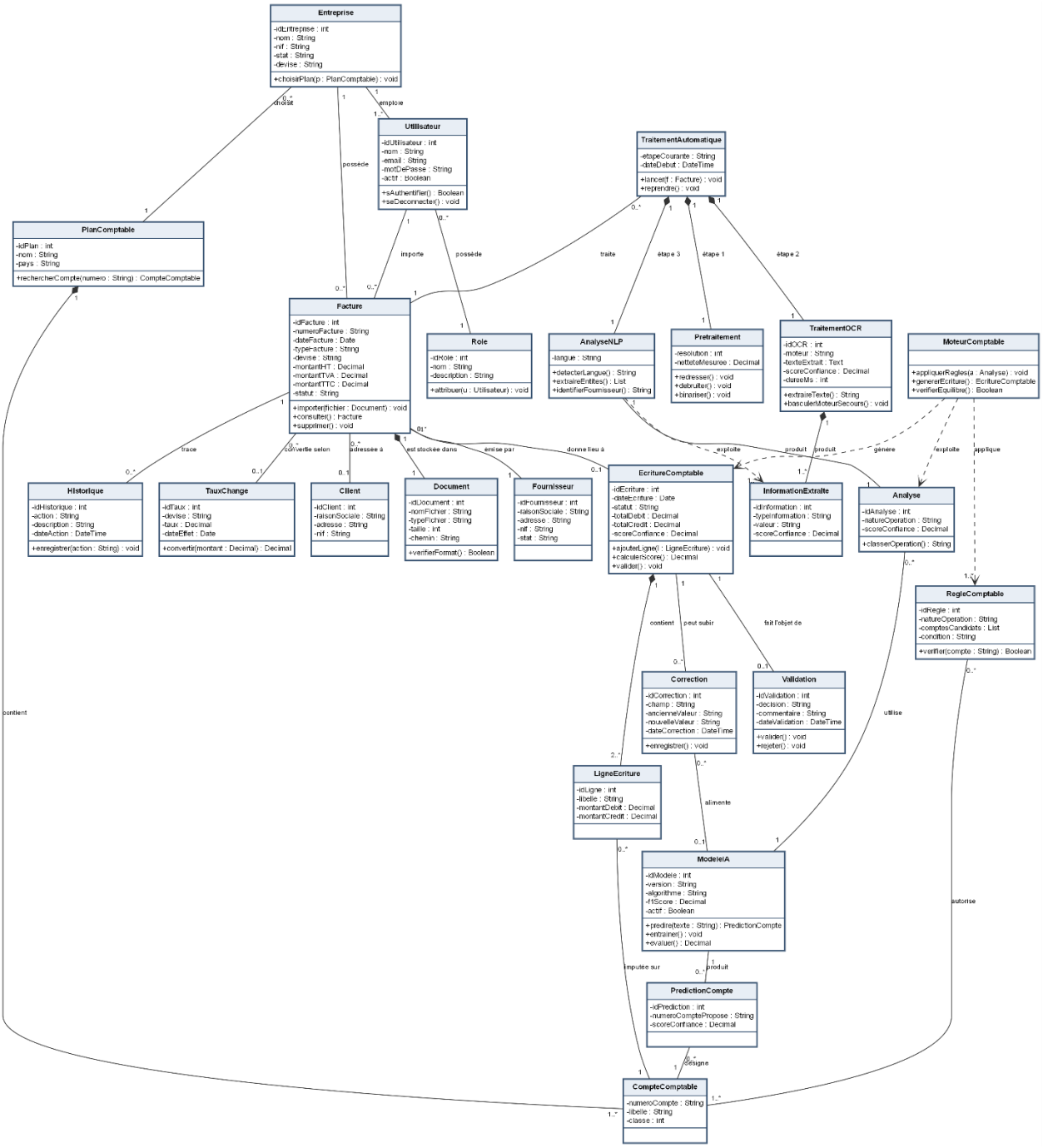


Figure 16: Diagramme de classes de conception globale

5.11 Diagramme de paquetage :

Le diagramme de paquetage permet d'organiser les différents composants logiciels du système en fonction de leurs responsabilités.

L'application est organisée autour de plusieurs modules fonctionnels : authentification, gestion des factures, OCR, NLP, Machine Learning, comptabilité, validation, historique et accès aux données.

Cette organisation permet de limiter le couplage entre les différents composants. Par exemple, le module OCR est responsable de l'extraction du texte, tandis que le module NLP est chargé de l'interprétation du contenu extrait. Le module comptable reste responsable de l'application des règles et de la génération des écritures.

Cette séparation favorise la maintenabilité et l'évolution du système.

La figure 17 présente le diagramme de paquetage de l'application.

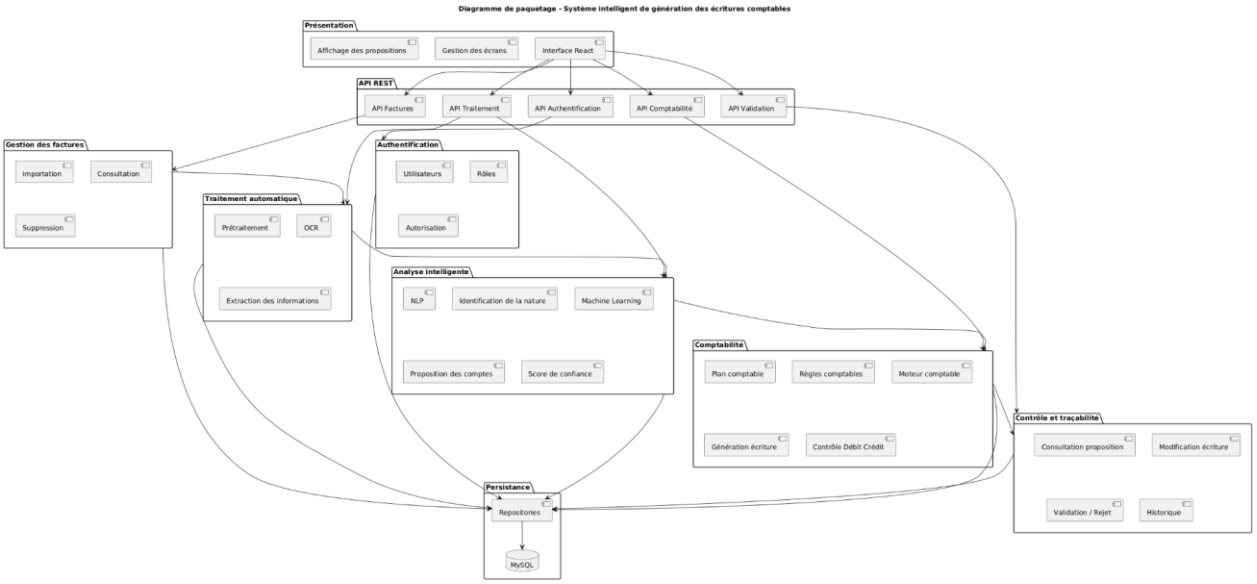


Figure 17: Diagramme de paquetage

5.12 Diagramme de déploiement :

Le diagramme de déploiement représente l'architecture physique et logique nécessaire au fonctionnement de l'application.

L'utilisateur accède au système à partir d'un navigateur Web. L'interface développée avec React et TypeScript communique avec le serveur backend à travers une API REST.

Le backend développé avec FastAPI et Python assure l'orchestration des différentes opérations. Il communique avec les modules OCR, NLP et Machine Learning ainsi qu'avec le moteur de règles comptables.

Les données de l’application sont stockées dans une base de données MySQL. L’architecture obtenue permet donc de séparer l’interface utilisateur, la logique métier, les traitements d’intelligence artificielle et la persistance des données.

En production, ces éléments sont regroupés dans trois conteneurs Docker (base de données, serveur applicatif et interface servie par Nginx) hébergés sur un serveur distant, derrière un point d'entrée Web qui assure le chiffrement des échanges (HTTPS). Cette organisation est détaillée à la section « Déploiement des environnements ».

La figure 18 présente le diagramme de déploiement du système.

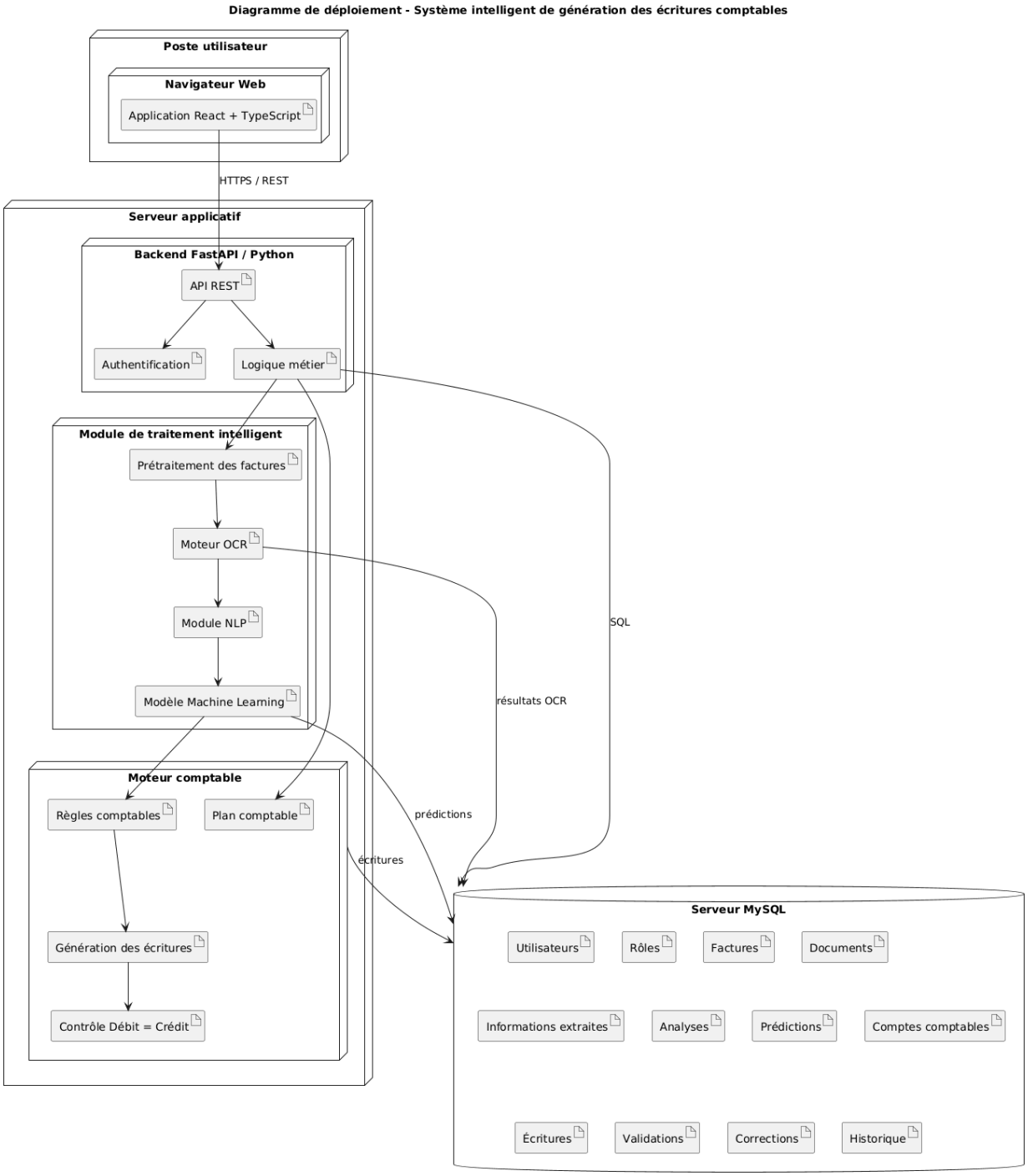


Figure 18: Diagramme de déploiement

CHAPITRE 6 : Intégration IA dans la conception

Ce chapitre précise la place de l'intelligence artificielle dans la conception du système : le rôle du module IA, les données dont il a besoin, le choix du modèle et sa méthode d'intégration.

6.1 Définition du module IA

Dans le cadre de ce projet, l'intelligence artificielle constitue le composant permettant d'automatiser l'analyse des factures commerciales et d'assister la génération des écritures comptables. Le module IA intervient après l'importation d'une facture et permet de transformer progressivement le document brut en informations exploitables par le système comptable.

L'intégration de l'intelligence artificielle repose sur plusieurs techniques complémentaires. L'OCR est utilisé pour extraire les informations textuelles contenues dans les factures sous forme d'images ou de documents PDF. Le NLP permet ensuite d'analyser et d'interpréter les informations extraites afin d'identifier la nature de l'opération réalisée. Le Machine Learning est utilisé pour proposer les comptes comptables correspondant aux opérations détectées à partir des caractéristiques de la facture.

Cependant, le Machine Learning n'est pas utilisé seul pour générer les écritures. Les propositions du modèle sont confrontées à un ensemble de règles comptables définies à partir du Plan Comptable Général malgache. Cette combinaison permet de prendre en compte à la fois la capacité d'apprentissage du modèle et les contraintes déterministes de la comptabilité.

Le fonctionnement général du module peut donc être représenté comme suit :

Facture → OCR → Informations structurées → NLP → Classification ML → Proposition de comptes → Règles comptables → Génération de l'écriture → Validation humaine

Le comptable conserve ainsi un rôle de contrôle. Il peut consulter la proposition générée, la modifier si nécessaire, puis la valider ou la rejeter.

6.2 Données nécessaires au fonctionnement du module IA

Le fonctionnement du module d'intelligence artificielle nécessite trois catégories de données : les données provenant des factures, les données comptables issues du référentiel et les données d'apprentissage constituées des opérations précédemment validées. Elles sont présentées successivement ci-dessous.

6.2.1 Données provenant des factures

Les factures constituent la principale source de données du système. Elles peuvent être fournies sous forme d'image ou de fichier PDF.

Les informations recherchées comprennent notamment :

- numéro de facture ;
- date de facture ;
- identité du fournisseur ou du client ;
- désignation des produits ou services ;
- quantité ;
- prix unitaire ;
- montant hors taxes ;
- montant de la TVA ;
- montant toutes taxes comprises ;
- type de facture ;
- nature de l'opération.

Ces informations sont d'abord extraites par l'OCR, puis structurées afin de pouvoir être exploitées par les traitements NLP et Machine Learning.

6.2.2 Données comptables

Le système nécessite également un référentiel comptable permettant d'associer les opérations identifiées aux comptes correspondants. Ce référentiel est indispensable, car le système ne doit jamais proposer un compte inexistant ou non autorisé : il garantit que chaque imputation respecte la nomenclature du Plan Comptable Général [5] et permet au moteur de règles de vérifier la cohérence entre la nature de l'opération et les comptes utilisés.

Ce référentiel comprend notamment :

- numéro du compte ;
- libellé du compte ;
- classe du compte ;
- type de compte ;
- catégorie de l'opération ;
- règles d'utilisation du compte.

Dans ce projet, le référentiel comptable utilisé est basé sur le Plan Comptable Général malgache.

6.2.3 Données d'apprentissage

Le modèle de Machine Learning nécessite des exemples permettant d'apprendre la correspondance entre les caractéristiques d'une opération commerciale et les comptes comptables appropriés.

Les données d'apprentissage peuvent notamment être constituées de factures associées à des écritures comptables déjà validées. Les corrections effectuées par le comptable peuvent également

être conservées afin de constituer progressivement une base de données exploitable pour l'amélioration du modèle.

Ainsi, le système peut suivre une logique d'apprentissage à partir des données validées :

Facture → prédiction → correction éventuelle → validation → donnée d'apprentissage

Cette approche permet au système de conserver les décisions validées par l'utilisateur et de les exploiter ultérieurement pour améliorer ses prédictions.

6.3 Choix du modèle IA et justification

Plusieurs approches peuvent être envisagées pour automatiser la génération des écritures comptables.

Une première approche consisterait à utiliser uniquement un ensemble de règles comptables. Cette solution permettrait d'obtenir un fonctionnement déterministe et facilement explicable, mais elle serait relativement rigide face à la diversité des factures commerciales.

Une deuxième approche consisterait à utiliser uniquement un modèle de Machine Learning. Cette solution permettrait au système d'apprendre à partir d'exemples existants et de s'adapter à différents types d'opérations. Toutefois, une prédiction statistique ne garantit pas à elle seule le respect des contraintes comptables, notamment l'équilibre entre les montants au débit et au crédit.

Le tableau 20 compare ces approches ainsi que les modèles d'apprentissage envisagés.

Tableau 20: Comparaison des approches et des modèles

Approche / modèle	Principe	Avantages	Limites
Règles comptables seules	Correspondances fixes entre mots-clés, nature de l'opération et comptes	Déterministe, explicable, aucun besoin de données	Rigide face à la diversité des factures ; maintenance manuelle des règles
Classifieur à mots-clés	Score selon la présence de mots-clés multilingues propres à chaque classe	Utilisable dès le démarrage (démarrage à froid), explicable	Précision limitée ; ne s'améliore pas seul
TF-IDF + régression logistique	Vectorisation du texte et modèle linéaire probabiliste	Efficace sur peu de données, rapide, probabilités exploitables comme score de confiance	Ne capte pas les relations complexes entre les termes
XGBoost	Ensemble d'arbres de décision construits par gradient boosting	Souvent plus précis sur des données structurées	Plus sensible au surapprentissage sur de petits volumes ; moins interprétable
Réseaux de neurones profonds (Transformers)	Modèles pré-entraînés sur de très grands corpus	Très performants sur le langage	Exigent beaucoup de données annotées et une carte graphique ; peu explicables

Au vu de cette comparaison, le projet adopte une approche hybride combinant l'intelligence artificielle et les règles comptables. Le classifieur à mots-clés assure le fonctionnement dès le démarrage ; dès que suffisamment de factures sont validées, la régression logistique et XGBoost sont entraînés puis comparés, et seul le meilleur est mis en service. Les règles comptables gardent le dernier mot sur les comptes autorisés, ce qui conserve la rigueur et l'explicabilité exigées en comptabilité.

Le fonctionnement retenu combine donc quatre composants :

OCR + NLP + Machine Learning + moteur de règles comptables

Le Machine Learning fournit une ou plusieurs propositions de comptes accompagnées d'un score de confiance. Le moteur de règles comptables vérifie ensuite la cohérence de ces propositions avec les règles définies dans le système.

Cette approche permet de distinguer deux responsabilités :

- le modèle IA réalise une prédiction à partir des caractéristiques de la facture ;
- le moteur de règles vérifie les contraintes comptables avant la génération ou la validation de l'écriture.

Par exemple, pour une facture donnée, le système peut identifier automatiquement la nature de l'opération et proposer les comptes correspondants. Si l'écriture obtenue ne respecte pas la règle fondamentale d'équilibre entre le débit et le crédit, elle ne peut pas être considérée comme valide automatiquement.

Le comptable intervient alors pour contrôler la proposition et effectuer les corrections nécessaires.

6.4 Méthode d'intégration de l'IA

L'intégration du module IA est réalisée au niveau du backend de l'application. Le frontend développé avec React permet à l'utilisateur d'importer la facture et de consulter les résultats. Le backend développé avec FastAPI assure l'orchestration des différents traitements.

Il s'agit d'un modèle embarqué dans le backend : les moteurs OCR, l'extracteur NLP et le classifieur sont chargés en mémoire par le serveur FastAPI, sans microservice séparé, et leurs résultats sont exposés à l'interface web par l'API REST.

Le processus commence par l'importation d'une facture sous forme de PDF ou d'image. Le document est ensuite prétraité afin d'améliorer sa qualité avant l'application de l'OCR.

Après le calcul du score de confiance, une étape de validation compare ce score à des seuils. Le score est une moyenne pondérée de quatre indicateurs : la qualité de la lecture OCR (25 %),

celle de l'extraction des informations (30 %), la probabilité de la classification (25 %) et la confiance dans les comptes proposés (20 %). Une proposition dont le score atteint 0,80 est marquée « fiable » ; entre 0,50 et 0,80, elle est marquée « à vérifier » ; en dessous de 0,50, elle demande une vérification approfondie. Cette validation n'empêche pas la génération de l'écriture : elle oriente l'attention du comptable, qui reste seul à valider.

L'OCR extrait le contenu textuel du document. Les informations obtenues sont ensuite analysées et structurées. Le module NLP permet d'identifier les éléments importants de la facture et de déterminer les caractéristiques nécessaires à la classification de l'opération.

Le modèle de Machine Learning utilise ensuite ces caractéristiques pour proposer les comptes comptables correspondants. Chaque proposition peut être associée à un score de confiance permettant d'indiquer le niveau de certitude du modèle.

Les résultats sont ensuite transmis au moteur de règles comptables. Celui-ci vérifie la cohérence de la proposition avec les règles définies dans le système et prépare les éléments nécessaires à la génération de l'écriture comptable.

Enfin, l'écriture générée est présentée au comptable pour validation. Celui-ci peut la modifier, la valider ou la rejeter. Les actions effectuées sont enregistrées afin d'assurer la traçabilité des traitements.

La figure 19 décrit ce processus sous forme de diagramme d'activité, et la figure 20 présente l'architecture d'intégration du module IA.

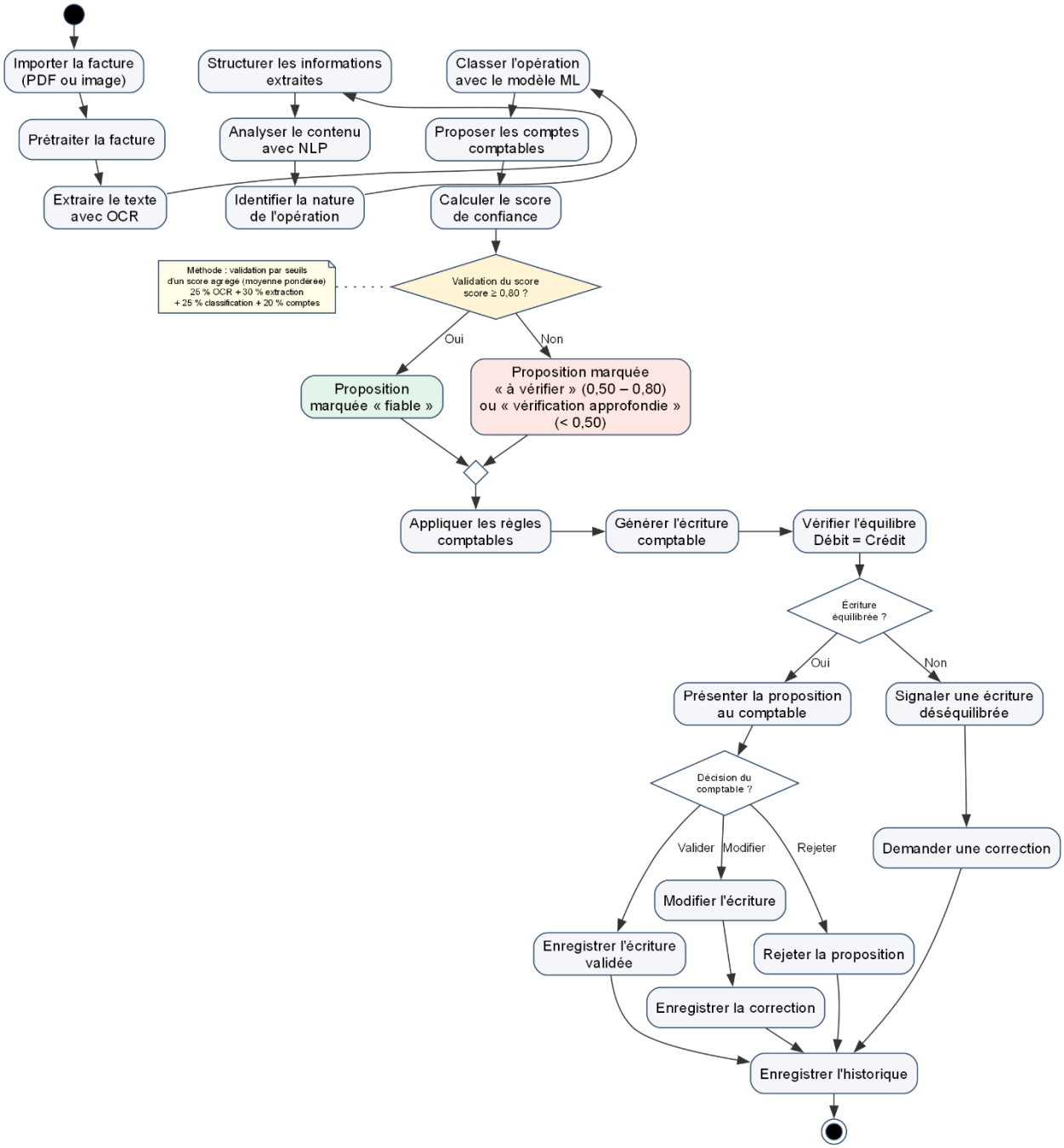


Figure 19: Processus d'intégration de l'IA dans le système (diagramme d'activité)

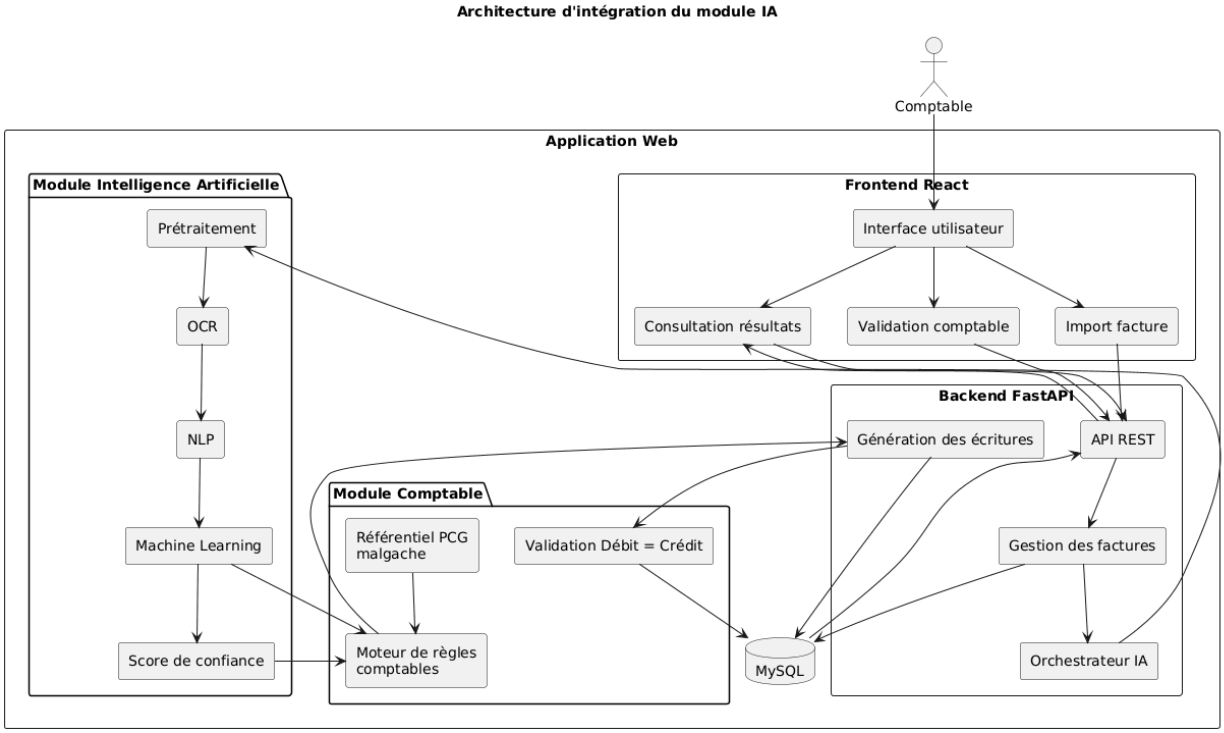


Figure 20: Architecture d'intégration du module IA

PARTIE III. RÉALISATION

CHAPITRE 7 : Mise en place de l'environnement de développement

Ce chapitre présente l'environnement mis en place pour réaliser le système : les outils installés, l'architecture logicielle adoptée et les différents environnements d'exécution.

7.1 Installation et configuration des outils

La mise en œuvre du système repose sur plusieurs briques technologiques complémentaires, chacune adaptée à un besoin précis : un serveur applicatif pour le traitement automatique des factures (lecture, analyse, classification), une base de données pour garantir la rigueur des enregistrements comptables, et une interface web pour que le comptable puisse superviser et valider chaque étape.

7.1.1 Environnement backend

- Un langage de programmation Python [15], installé dans un environnement isolé propre au projet, afin de ne pas interférer avec les autres logiciels du poste de travail ;
- Un serveur web léger (FastAPI) [17] qui reçoit les requêtes de l'interface et y répond, avec redémarrage automatique à chaque modification du code pendant le développement ;
- Un outil de dialogue avec la base de données (SQLAlchemy) [16], qui permet au programme de lire et d'écrire dans MySQL sans avoir à écrire des requêtes SQL à la main pour chaque opération ;
- Des bibliothèques dédiées à la sécurité des comptes utilisateurs (mots de passe chiffrés, jetons de connexion temporaires) ;
- Des outils de traitement d'image et de lecture automatique de texte (préparation de l'image, moteur de lecture principal, avec un second moteur de secours en cas d'échec) ;
- Un outil de reconnaissance du langage naturel, capable de repérer automatiquement des informations comme le nom d'un fournisseur dans un texte libre ;
- Des bibliothèques d'apprentissage automatique, utilisées pour entraîner et sauvegarder les modèles de classification ;
- Un outil de tests automatisés, qui permet de vérifier à chaque modification que le système continue de fonctionner correctement.

L'ensemble de ces outils, avec leur version exacte, est enregistré dans un fichier de configuration du projet, ce qui permet de réinstaller à l'identique le même environnement sur un autre ordinateur.

7.1.2 Environnement frontend

- Un gestionnaire de paquets (Node.js / npm), équivalent côté interface web de l'environnement Python côté serveur ;

- Un framework d'interface (React) associé à un langage qui limite les erreurs de programmation (TypeScript), avec un outil de développement rapide (Vite) ;
- Un outil de mise en forme visuelle (Tailwind CSS), qui accélère la création de composants graphiques cohérents ;
- Des bibliothèques pour la navigation entre pages, la communication avec le serveur, la gestion automatique du cache des données, et l'affichage des graphiques du tableau de bord.

7.1.3 Base de données et outillage complémentaire

- Une base de données MySQL installée localement, dans laquelle est créé le plan comptable général malgache 2005 [5] ;
- Un outil de gestion de versions (Git), qui conserve l'historique de toutes les modifications du code ;
- Un éditeur de code Visual Studio Code, utilisé avec l'assistance d'un outil d'intelligence artificielle d'aide à la programmation pour accélérer certaines tâches techniques et rédactionnelles ;
- Un fichier de configuration privé, qui centralise les informations sensibles (accès à la base de données, clés de sécurité, paramètres par défaut) et n'est jamais partagé ni publié.

7.2 Architecture de l'application

Le serveur applicatif est organisé en quatre grandes zones, chacune avec une responsabilité bien délimitée, afin que les règles comptables restent indépendantes des choix techniques (base de données, moteur de lecture, etc.) et puissent être testées et modifiées sans risquer de casser le reste du système :

- le cœur métier, qui contient les règles comptables fondamentales (par exemple, qu'une écriture doit toujours être équilibrée entre débit et crédit) et définit ce qu'attend le système de chacun de ses composants, sans se soucier de la manière dont ils sont réalisés ;
- la couche d'orchestration, qui décrit les grandes actions possibles (importer une facture, corriger une écriture, la valider...) en faisant appel au cœur métier, sans connaître les détails techniques sous-jacents ;
- les composants techniques concrets : accès à la base de données, moteurs de lecture automatique, extraction d'informations, intelligence artificielle, moteur de règles comptables, stockage des fichiers ;
- la couche d'exposition, qui reçoit les demandes venant de l'interface web et les redirige vers l'action appropriée.

La figure 21 représente cette organisation en couches, inspirée de la Clean Architecture [6].

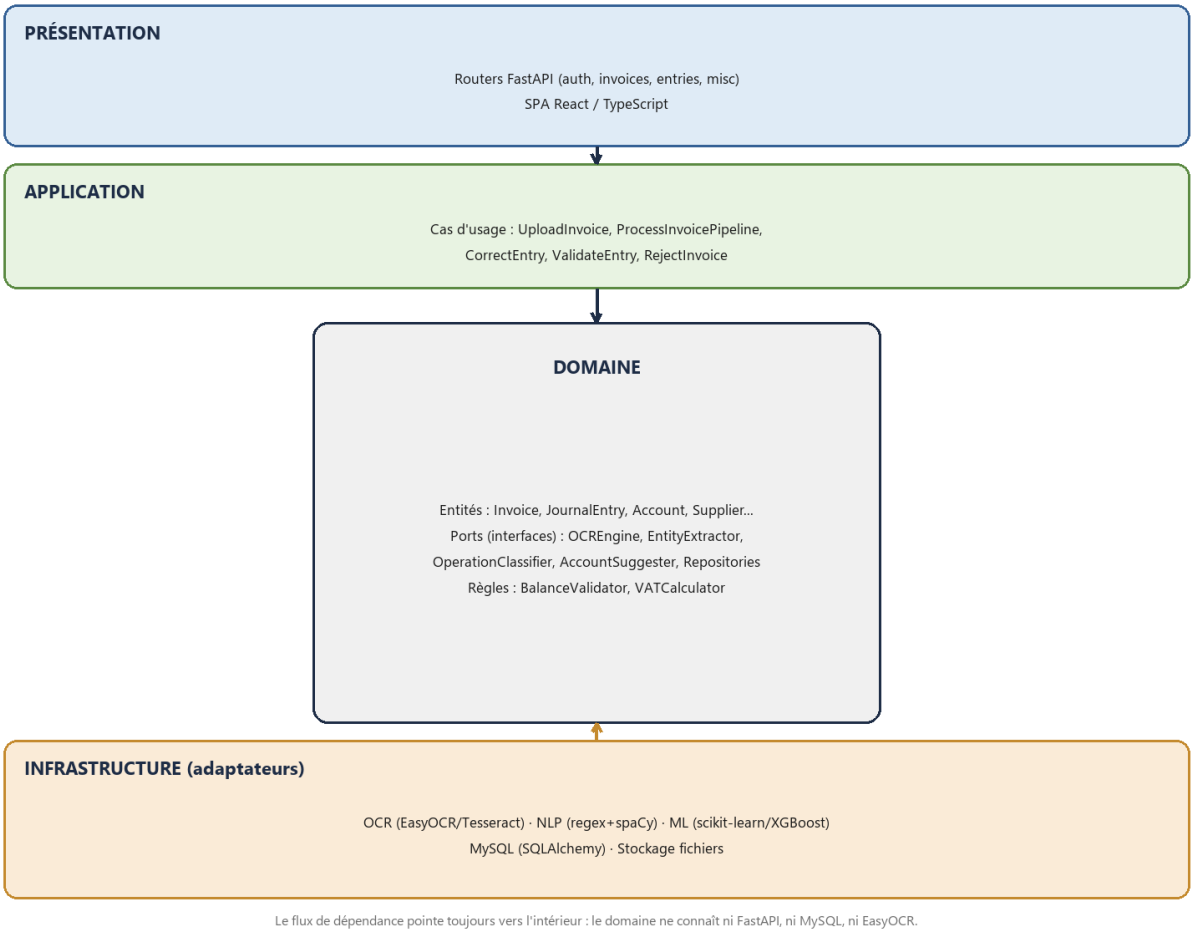


Figure 21: Architecture en couches de l'application (adaptée de la Clean Architecture)

Cette organisation facilite grandement les tests : le cœur métier peut être vérifié sans base de données ni service externe, chaque composant technique peut être testé indépendamment (chapitre 9), et il est possible de remplacer un composant par un autre - par exemple changer de moteur de lecture ou de base de données - sans toucher au cœur métier, tant que le nouveau composant respecte le même contrat.

Côté interface web, un principe équivalent a été appliqué : aucune page ne communique directement avec le serveur ; tous les échanges passent par une couche intermédiaire dédiée, qui centralise notamment l'ajout automatique du jeton de connexion à chaque requête (section 8.2).

7.3 Déploiement des environnements (développement, test, production)

L'application est exécutée dans trois environnements distincts, chacun répondant à un objectif précis : développer, tester puis exploiter le système.

7.3.1 Environnement de développement

En phase de développement, le serveur applicatif et l'interface web fonctionnent séparément, chacun sur son propre port de communication, et se redémarrent automatiquement à chaque modification du code. Un mécanisme de redirection interne permet à l'interface de communiquer avec le serveur de

manière transparente pour l'utilisateur. Cette séparation reproduit l'organisation retenue en production, où l'interface et le serveur s'exécutent dans des conteneurs distincts.

7.3.2 Environnement de test

Les tests automatisés s'exécutent dans un environnement totalement isolé : la base de données de développement n'est jamais sollicitée. A chaque test, une base de données temporaire et vide est recréée en mémoire, avec exactement la même structure que la base réelle. Cela garantit une suite de tests rapide (moins d'une minute pour 103 tests), reproductible sur n'importe quel ordinateur, et sans risque d'interférence entre deux exécutions.

7.3.3 Environnement de production

L'application a été mise en production sur un serveur privé virtuel (VPS) loué chez l'hébergeur Contabo et fonctionnant sous Linux (Ubuntu). Ce serveur héberge déjà d'autres projets, ce qui a imposé de réutiliser son point d'entrée Web existant plutôt que d'en créer un nouveau.

Pour que l'installation soit reproductible, l'application est empaquetée avec Docker : chaque composant s'exécute dans son propre conteneur, c'est-à-dire un environnement isolé qui embarque exactement les logiciels dont il a besoin. L'ensemble est décrit dans un seul fichier de configuration (Docker Compose) qui définit trois conteneurs :

- la base de données MySQL 8, dont les données sont conservées sur un volume persistant (elles survivent à une mise à jour ou à un redémarrage) et qui crée automatiquement son schéma et charge le plan comptable malgache au tout premier démarrage ;
- le serveur applicatif (FastAPI), qui contient aussi les moteurs de lecture, d'extraction et d'intelligence artificielle ; il ne démarre qu'une fois la base de données déclarée en bon état de marche par un contrôle de santé, et non au hasard de l'ordre de lancement ;
- l'interface web, construite en version optimisée puis servie par Nginx, qui joue aussi le rôle d'intermédiaire : les appels de l'interface vers l'API sont transmis au serveur applicatif, avec un délai d'attente étendu (180 secondes) puisque le traitement d'une facture par la lecture automatique et l'intelligence artificielle peut durer plusieurs dizaines de secondes.

Les informations sensibles (clé de signature des jetons de connexion, mot de passe de la base de données) ne figurent pas dans le code : elles sont placées dans un fichier de configuration propre au serveur, avec des valeurs générées aléatoirement.

Le serveur disposait déjà d'un point d'entrée Web (Caddy) qui occupe les ports standards 80 et 443 pour ses autres projets. L'interface de Compta-IA est donc exposée sur un port dédié (8080), et Caddy la publie sous un nom de domaine propre, avec un certificat HTTPS obtenu et renouvelé automatiquement. Ce nom de domaine repose sur le service gratuit nip.io, qui traduit un nom

contenant l'adresse IP du serveur vers cette adresse et évite ainsi l'achat d'un nom de domaine : l'application est accessible à l'adresse indiquée en webographie [11], sans numéro de port et en connexion chiffrée.

Le déploiement se résume à transférer le code sur le serveur, créer le fichier de secrets, puis lancer une commande unique de construction et de démarrage des trois conteneurs. La première construction est longue (plus de vingt minutes, l'image du serveur applicatif approchant 4 Go à cause des bibliothèques d'intelligence artificielle), mais les constructions ultérieures réutilisent les étapes déjà construites et ne prennent que quelques secondes lorsque seul le code change.

La mise en production a fait apparaître quatre problèmes que l'environnement de développement ne révélait pas, tous corrigés puis documentés :

- conflit de port : le port 80 était déjà utilisé par le point d'entrée Web du serveur ; l'interface est désormais exposée sur un port dédié, derrière ce point d'entrée ;
- fins de ligne Windows : le script de démarrage du serveur applicatif, écrit sous Windows, comportait des fins de ligne propres à ce système, que Linux interprète comme une partie du nom du programme à lancer (« bash\r » au lieu de « bash »), si bien que le conteneur redémarrait en boucle ; un fichier de configuration du dépôt impose désormais les fins de ligne Linux pour les scripts, quel que soit l'ordinateur qui les récupère ;
- dépendance oubliée : la bibliothèque de validation des adresses électroniques, installée à la main sur le poste de développement, ne figurait pas dans la liste officielle des dépendances et ne manquait donc que dans un environnement neuf ; elle a été ajoutée à cette liste ;
- accents corrompus : le plan comptable chargé dans la base du conteneur affichait « Ã© » à la place de « é », pour la même raison que l'incident décrit à la section « Création et structuration de la base de données » (scripts lus avec un jeu de caractères par défaut inadapté) ; le serveur de base de données est désormais configuré explicitement en UTF-8 [12] et les libellés déjà enregistrés ont été corrigés.

Ces quatre incidents ont une cause commune : l'écart entre le poste où l'on développe et le serveur où l'on exécute. Décrire l'ensemble de l'installation dans des fichiers versionnés (fichiers de construction Docker, Docker Compose, configuration Nginx) permet de corriger chaque défaut une fois pour toutes et de rejouer l'installation à l'identique.

La mise en production reste déclenchée manuellement. La mise en place d'une chaîne d'intégration continue (tests automatiques puis déploiement à chaque modification) demeure une piste d'amélioration identifiée en conclusion.

CHAPITRE 8 : Développement de l'application

Ce chapitre décrit la réalisation de l'application : la base de données, l'interface web, le serveur applicatif, le module d'intelligence artificielle et leur intégration.

8.1 Création et structuration de la base de données

Le schéma de la base de données traduit directement le dictionnaire des données présenté en section 5.5 : seize tables sous MySQL 8. Deux principes structurent l'ensemble : tous les montants sont stockés dans un format numérique précis qui ne génère jamais d'arrondi imprévisible - une exigence non négociable en comptabilité - et le journal des actions ne peut jamais être modifié ni supprimé après coup, ce qui garantit une traçabilité complète de toutes les actions (import, validation, correction, création d'utilisateur...).

Le plan comptable est initialisé à partir du Plan de comptes recommandé du Décret n° 2004-272 du 18/02/2004 [5] (annexe I, pp. 144-155). Les subdivisions spécifiques introduites par l'article 520-2 du PCG sont explicitement repérées dans le script d'initialisation, afin de garder la traçabilité entre le texte réglementaire et son implantation en base.

Un incident survenu lors du chargement initial des données illustre bien les pièges propres à la manipulation de textes contenant des accents : le fichier source était correctement enregistré, mais son exécution via un outil dont le paramétrage de caractères différait a corrompu les libellés de comptes contenant des accents français directement en base. Plutôt que de ressaisir manuellement plus d'une centaine de libellés, un script correctif a été écrit pour relire le fichier source et régénérer automatiquement les corrections nécessaires, elles-mêmes vérifiées ensuite. Cet épisode a conduit à documenter explicitement, dans la documentation du projet, le paramétrage attendu pour toute exécution manuelle de script sur la base de données. L'incident s'est reproduit à l'identique lors de la mise en production, la base du conteneur MySQL lisant elle aussi ses scripts d'initialisation avec un jeu de caractères par défaut inadapté : la cause a cette fois été traitée à la source, en configurant explicitement le serveur de base de données en UTF-8 (voir la section « Environnement de production »).

8.2 Développement du frontend

L'interface est une application web à page unique, organisée en plusieurs écrans (connexion, tableau de bord, import, détail d'une facture, gestion des écritures, historique, administration) et en éléments graphiques réutilisables (bandeau de navigation, badges de statut et de confiance, boutons d'export, indicateurs de chargement...). La navigation restreint l'accès aux pages sensibles : une page n'est accessible qu'à un utilisateur connecté, et certaines pages (administration) exigent en plus un rôle spécifique - ce qui reflète le contrôle des droits d'accès défini côté serveur.

La communication avec le serveur passe systématiquement par une couche intermédiaire dédiée, avec un module distinct par type de données (utilisateurs, factures, écritures, comptes, administration). Cette couche ajoute automatiquement le jeton de connexion à chaque requête, et déconnecte automatiquement l'utilisateur si le serveur signale que sa session a expiré. Concrètement, cela signifie que chaque page de l'interface n'a jamais besoin de gérer elle-même l'authentification : elle délègue cette responsabilité à cette couche commune, ce qui évite toute incohérence entre les pages.

Cette couche de communication est la seule autorisée à dialoguer avec le serveur ; aucune page n'y accède directement. La mise à jour des données affichées (liste des factures, des écritures, tableau de bord) est gérée automatiquement par un outil dédié, qui évite les rechargements manuels et affiche des indicateurs de chargement ou d'erreur de manière cohérente sur toute l'application.

8.2.1 Interface utilisateur

Les figures 21 à 25 illustrent les principales pages de l'application, telles qu'utilisées par le comptable au fil du traitement d'une facture.

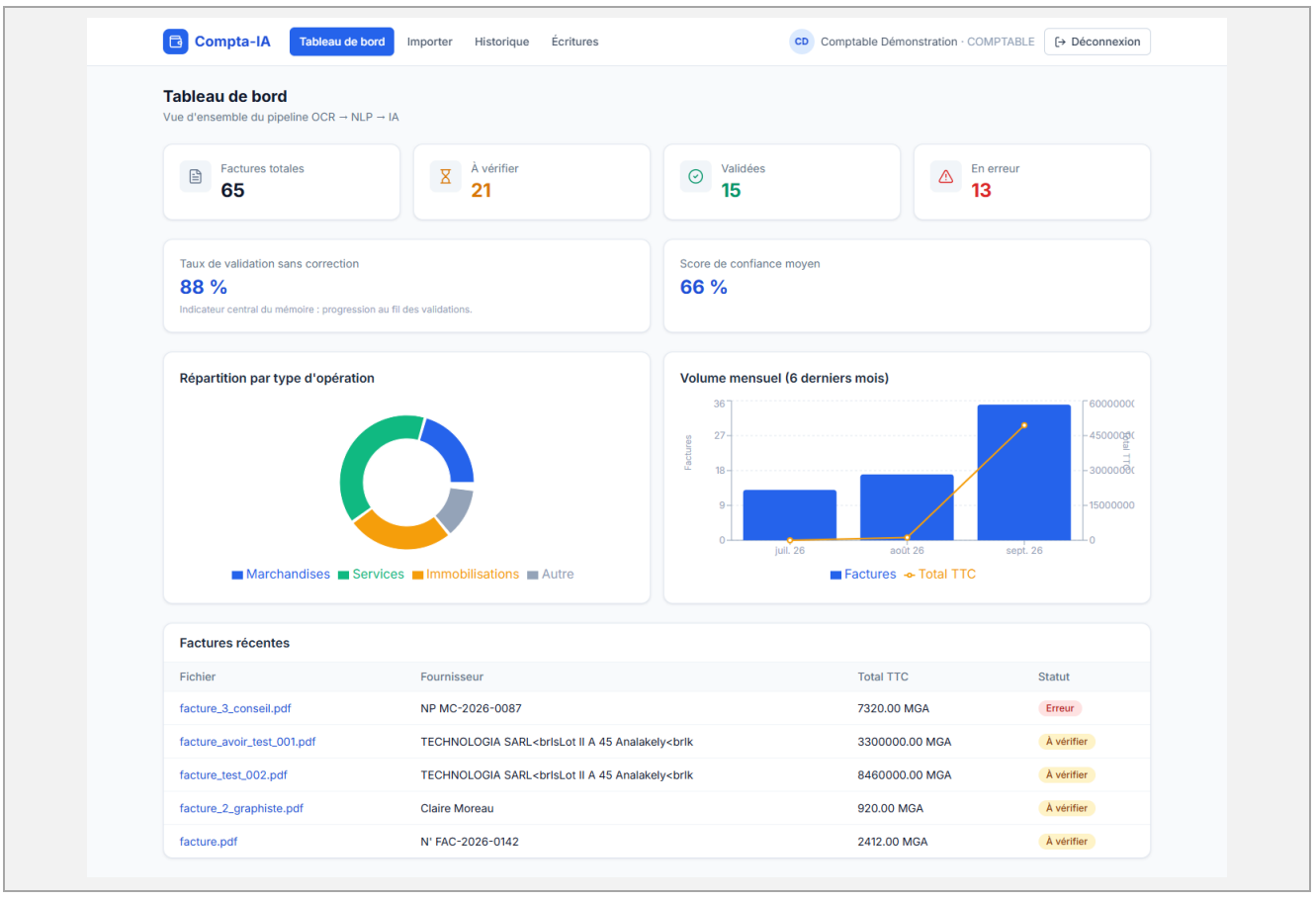


Figure 22: page DashboardPage — vue d'ensemble des factures et indicateurs

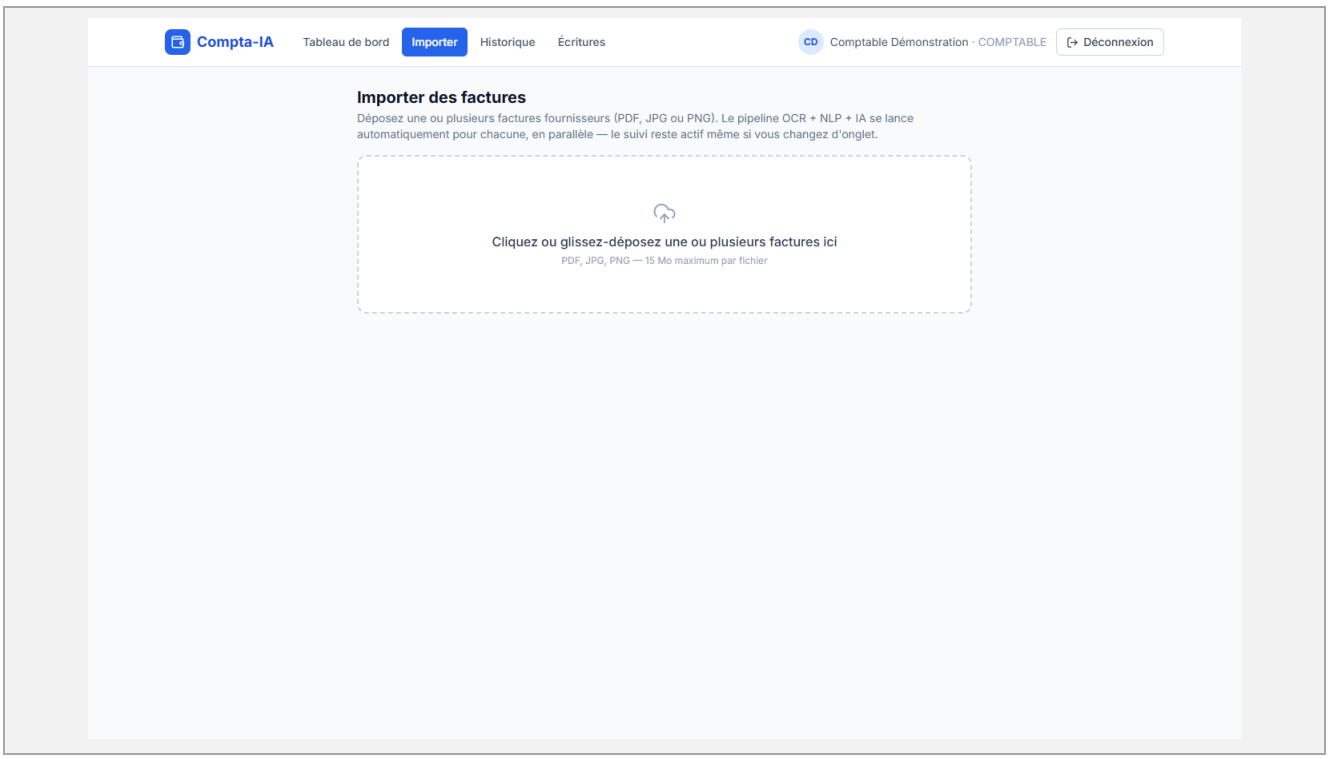


Figure 23: page UploadPage — dépôt d'un PDF ou d'une image de facture

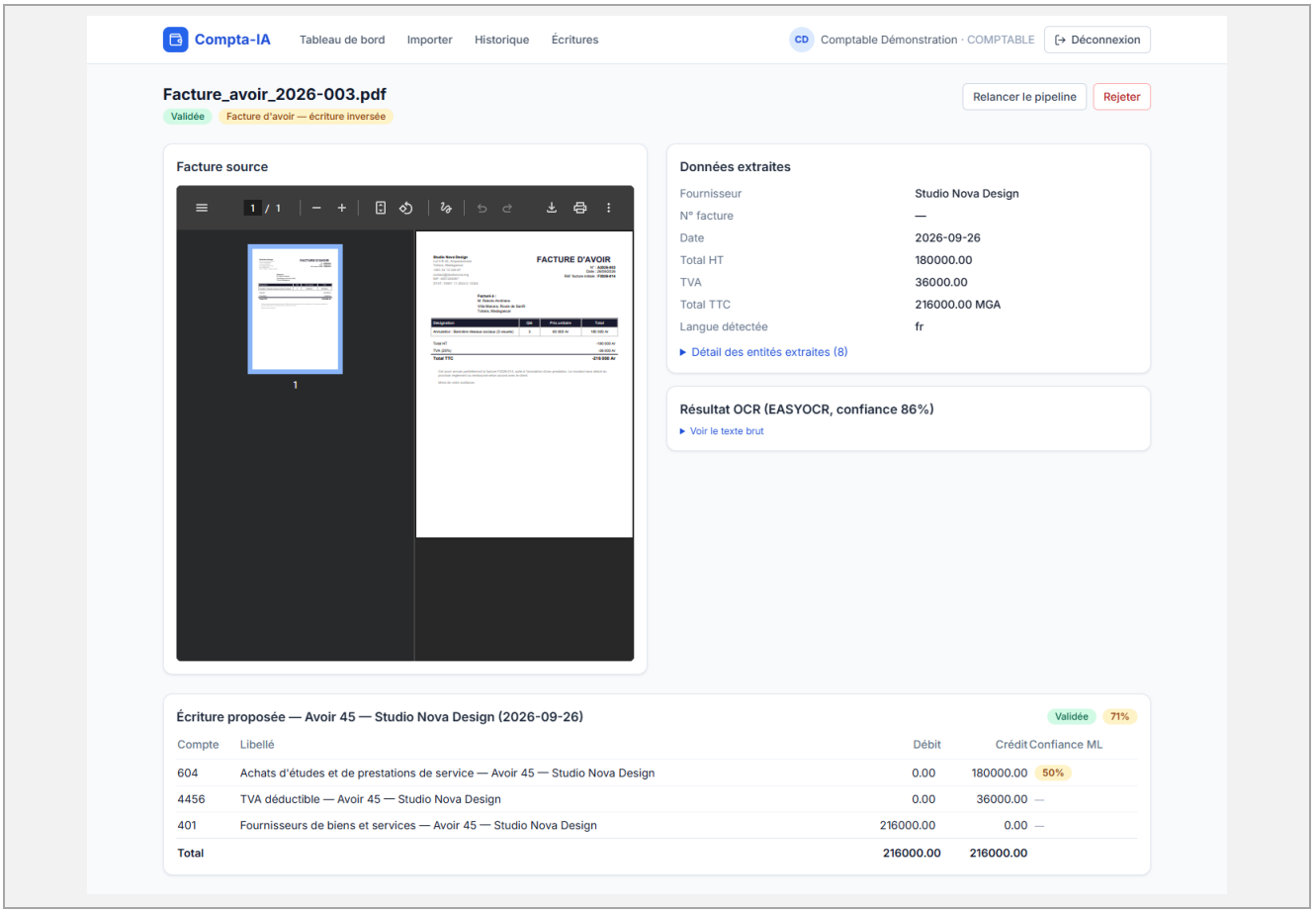


Figure 24: page InvoiceDetailPage — résultat OCR, entités extraites et écriture proposée avec score de confiance

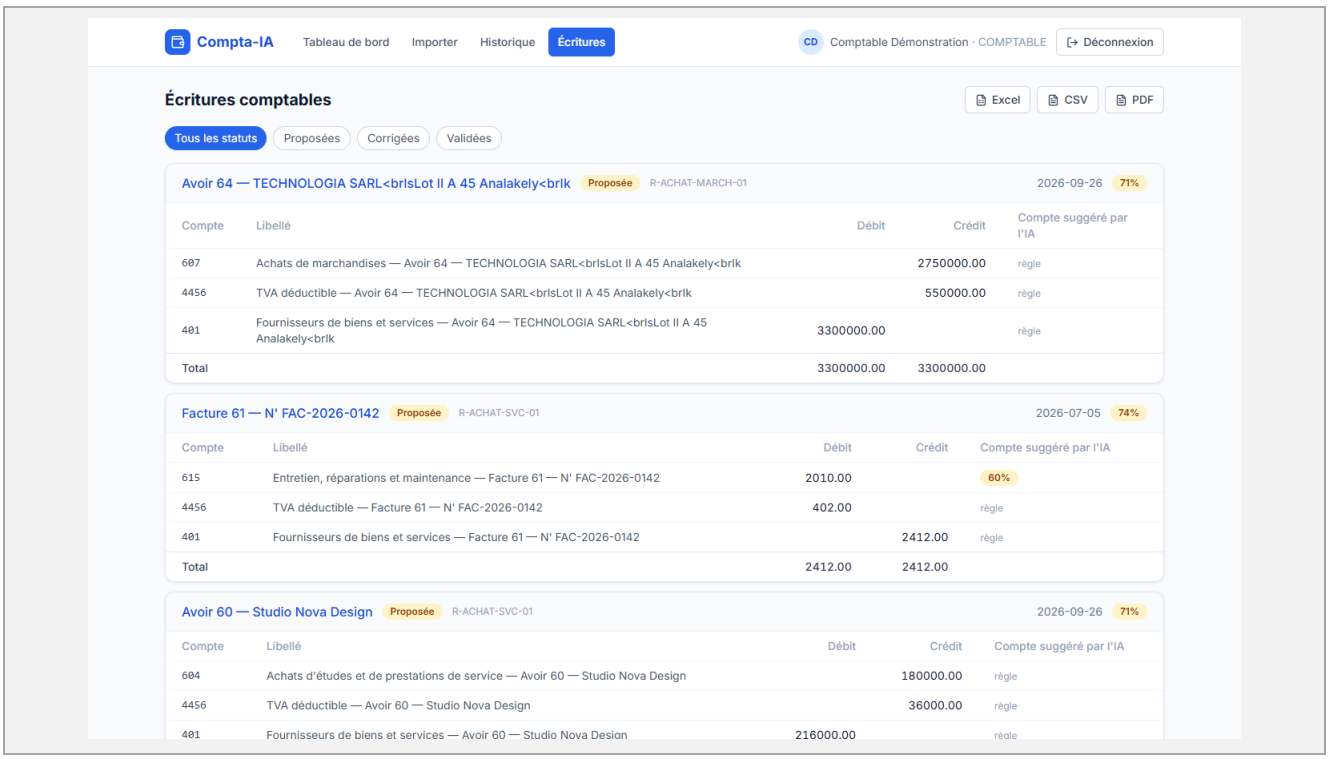


Figure 25: page EntriesPage — correction et validation d'une écriture comptable

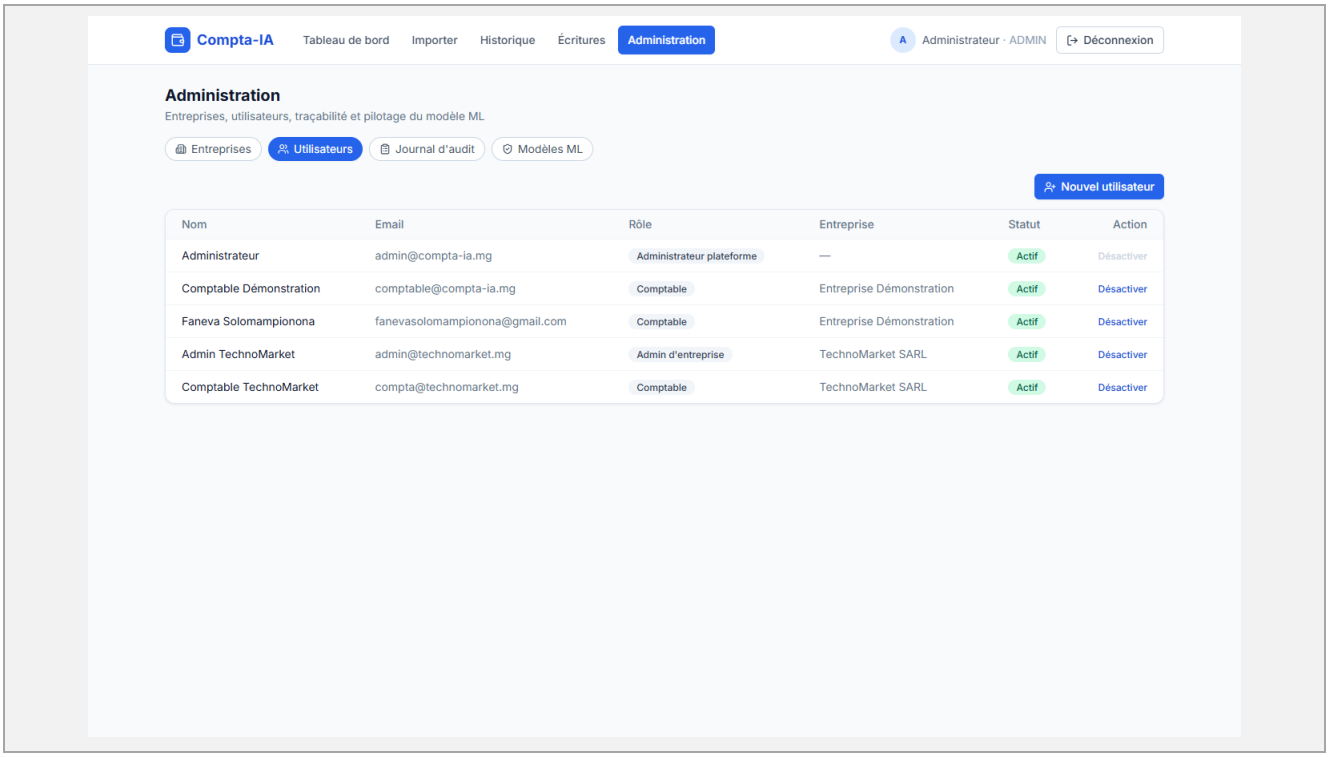


Figure 26: page AdminPage — gestion des utilisateurs et réentraînement du modèle

8.3 Développement du backend

Le serveur applicatif expose une interface de programmation organisée en quatre grandes familles de fonctions : la connexion et la gestion des utilisateurs, l'import et le suivi des factures, la consultation/correction/validation des écritures avec leurs exports, et les fonctions transverses (plan comptable, tableau de bord, journal d'audit, réentraînement du modèle). Chaque fonction se contente

de déclencher l'action correspondante dans la couche d'orchestration ; elle ne contient elle-même aucune règle métier.

La règle la plus critique du système - l'équilibre obligatoire entre débit et crédit d'une écriture comptable - est isolée dans le cœur métier, indépendamment de toute considération technique, et vérifiée à deux moments clés : au moment où l'écriture est proposée automatiquement, puis à nouveau au moment où le comptable la valide définitivement (puisqu'il a pu modifier des montants entre-temps). Cette double vérification garantit qu'aucune écriture déséquilibrée ne peut jamais être enregistrée, même en cas d'erreur humaine ou logicielle.

Un soin particulier a été apporté à la robustesse des traitements qui s'exécutent en arrière-plan, sans qu'aucun utilisateur ne soit directement présent pour réagir en cas de problème - c'est le cas du traitement complet d'une facture, qui démarre juste après que l'utilisateur a envoyé son fichier. Si une erreur imprévue survient à n'importe quelle étape, elle est systématiquement interceptée : la facture est alors marquée en erreur avec un message explicite, plutôt que de laisser le traitement s'interrompre silencieusement sans que personne ne s'en aperçoive.

8.4 Développement et entraînement du module IA

Le module d'intelligence artificielle est organisé en trois grandes familles de composants, chacune responsable d'une étape précise du traitement, dont le code est présenté en annexes D, E et F.

8.4.1 OCR

Le système de lecture automatique combine deux moteurs : un moteur principal, et un moteur de secours qui prend le relais si le premier obtient une confiance trop faible ou détecte trop peu de mots. Si les deux moteurs échouent, la facture est explicitement marquée comme illisible plutôt que de laisser passer une information douteuse sans avertissement. Ces seuils de décision sont ajustables dans la configuration du système. En amont, une chaîne de préparation d'image (mise en niveaux de gris, redressement, réduction du bruit, contraste) améliore la qualité de la lecture, dont le code figure en annexe E.

8.4.2 NLP

L'extraction des informations combine deux approches complémentaires : des motifs de reconnaissance pour les données à format fixe (numéro fiscal, numéro de facture, montants), et un modèle de reconnaissance du langage naturel pour les données à format libre, comme le nom du fournisseur. Chaque information extraite conserve sa méthode d'origine et son niveau de confiance, ce qui permet d'expliquer a posteriori pourquoi le système a proposé telle valeur.

8.4.3 Classification et suggestion de comptes

Deux méthodes de classification du type d'opération coexistent : une première méthode, basée sur la reconnaissance de mots-clés dans plusieurs langues, sert de solution de départ tant qu'aucun modèle statistique n'a été suffisamment entraîné ; une seconde méthode, statistique, prend ensuite le relais une fois assez de données validées disponibles. Dans les deux cas, le système de suggestion de comptes reste volontairement contraint : il ne peut jamais proposer un compte en dehors de la liste autorisée par la règle comptable active, ce qui l'empêche de s'écarter du cadre réglementaire du plan comptable.

L'entraînement du modèle suit une démarche comparative : deux algorithmes différents sont entraînés sur le même jeu de données (les factures les plus anciennes servant à l'entraînement, les plus récentes à la vérification), puis comparés selon leur performance. Seul le meilleur des deux est ensuite comparé à la version actuellement utilisée en production ; il n'est mis en service que s'il est au moins aussi performant - une protection qui empêche qu'un réentraînement dégrade silencieusement la qualité du système.

Chaque version entraînée est conservée de manière versionnée (jamais écrasée) et enregistrée avec ses métriques de performance, le nombre d'exemples utilisés et son statut (active ou archivée). Trois versions successives du classifieur ont déjà été produites au cours du développement, ce qui démontre le bon fonctionnement du cycle d'entraînement itératif ; une évaluation à plus grande échelle est discutée au chapitre 9.

8.5 Intégration IA–application

L'ensemble du traitement d'une facture est orchestré automatiquement, de bout en bout, juste après que l'utilisateur l'a déposée. Le statut de la facture trace précisément sa progression à chaque étape (mise en attente, prétraitement, lecture effectuée, analyse terminée, écriture proposée), avec trois issues possibles à la fin : validée par le comptable, rejetée, ou en erreur.

La figure 27 présente ce pipeline, de l’import de la facture à sa validation comptable.

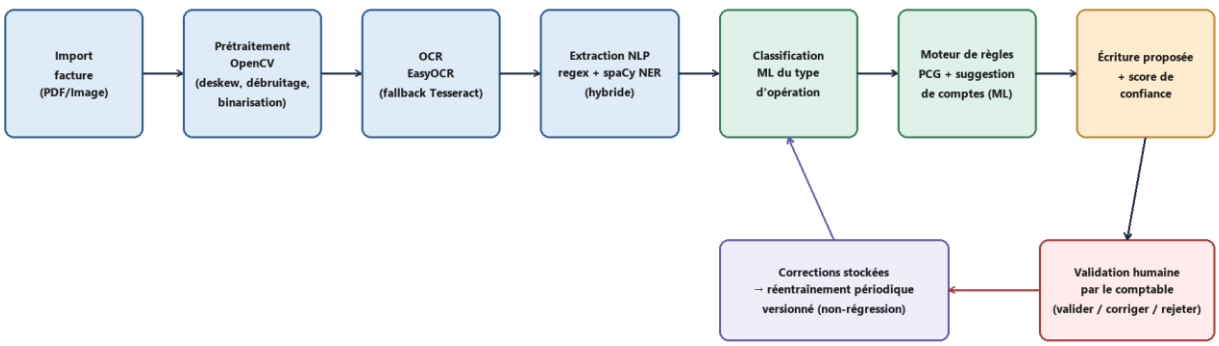


Figure 27: Pipeline de traitement automatisé d'une facture, de l'import à la validation comptable

Le score de confiance affiché au comptable résulte de la combinaison de plusieurs indicateurs (qualité de la lecture, qualité de l'extraction, fiabilité de la classification, confiance des comptes suggérés), réduit en cas d'incohérence entre les montants, de fournisseur inconnu, ou de recours au moteur de lecture de secours. Ce score, présenté sous forme de code couleur (vert, orange, rouge), guide l'attention du comptable vers les factures les plus incertaines sans jamais se substituer à sa décision finale. Chaque correction qu'il apporte est conservée et vient enrichir les données utilisées lors du prochain entraînement du modèle, qui peut être déclenché manuellement par un administrateur ou automatiquement une fois un certain nombre de corrections accumulées.

CHAPITRE 9 : Tests et validation

Ce chapitre présente les vérifications effectuées pour s'assurer du bon fonctionnement du système : les tests automatisés, la validation fonctionnelle et l'évaluation des performances du module IA.

9.1 Tests unitaires et d'intégration

Pour vérifier que le système fonctionne correctement, une suite de 103 tests automatisés a été mise en place : ce sont des programmes qui rejouent automatiquement différentes situations (normales et exceptionnelles) et vérifient que le résultat obtenu est bien celui attendu, sans intervention humaine. L'ensemble s'exécute en moins d'une minute et couvre trois niveaux de vérification, du plus isolé au plus global :

- 18 tests vérifient les règles comptables de base (par exemple, qu'une écriture est toujours équilibrée entre débit et crédit, ou que le calcul de la TVA est correct), indépendamment de toute base de données ou interface ;
- 40 tests vérifient le bon fonctionnement des composants techniques (lecture des factures, extraction des informations, proposition de comptes, entraînement du modèle d'intelligence artificielle) — notamment qu'un nouveau modèle n'est jamais mis en service tant qu'il n'est pas au moins aussi performant que celui déjà utilisé ;
- 45 tests rejouent des scénarios complets d'utilisation de l'application, comme le ferait un vrai utilisateur : connexion, import d'une facture, suivi de son traitement, correction et validation d'une écriture, consultation du tableau de bord, de l'historique et des exports.

L'exécution de cette suite de tests confirme que les 103 scénarios passent avec succès, sans aucun échec :

```
.................................................................... [ 69%]
.............................                                         [100%]
103 passed, 6 warnings in 47.20s
```

Sortie réelle de la suite de tests backend (pytest)

9.2 Validation fonctionnelle

Au-delà de ces vérifications automatiques, le parcours complet d'un utilisateur a également été rejoué de bout en bout : connexion, import d'une facture, consultation du résultat de la lecture automatique et des informations extraites, puis correction et validation de l'écriture proposée. Ces vérifications couvrent notamment le contrôle des droits d'accès (un utilisateur ne peut consulter que les factures qui le concernent), le rejet des fichiers non conformes, et la cohérence du statut de la facture à chaque étape de son traitement.

La chaîne complète de traitement (lecture de l'image, extraction du texte, reconnaissance des informations, classification, puis génération de l'écriture) a également été testée manuellement avec

de vraies factures, déposées directement depuis l'interface. Les images intermédiaires produites à chaque étape de préparation de l'image ont été conservées, ce qui a permis de vérifier visuellement l'effet de chaque transformation avant la lecture automatique. Les cas problématiques (facture illisible, montants incohérents, fournisseur inconnu) ont également été testés : dans chacun de ces cas, le système abaisse son score de confiance ou signale clairement l'anomalie au comptable, plutôt que de laisser passer une information non fiable sans avertissement.

Enfin, l'utilisation de l'application sur le serveur de production, avec le parcours complet (import d'une facture, traitement, validation de l'écriture, export du journal en PDF), a révélé deux défauts que les tests automatiques ne détectaient pas. D'abord, l'en-tête du journal PDF affichait le nom par défaut « Entreprise » au lieu de celui de l'entreprise de l'utilisateur ; il affiche désormais le nom réel (« Toutes les entreprises » pour l'administrateur global) et un test automatique vérifie ce point. Ensuite, pendant le traitement d'une facture, deux mécanismes indépendants (la page de détail et la fenêtre de suivi de progression) interrogeaient chacun le serveur toutes les deux secondes, ce qui doublait inutilement les requêtes. La fenêtre de suivi partage désormais le cache de la page de détail : une mesure en conditions réelles montre une seule requête toutes les deux secondes.

La figure 28 présente le déroulement d’un scénario de validation fonctionnelle complet.

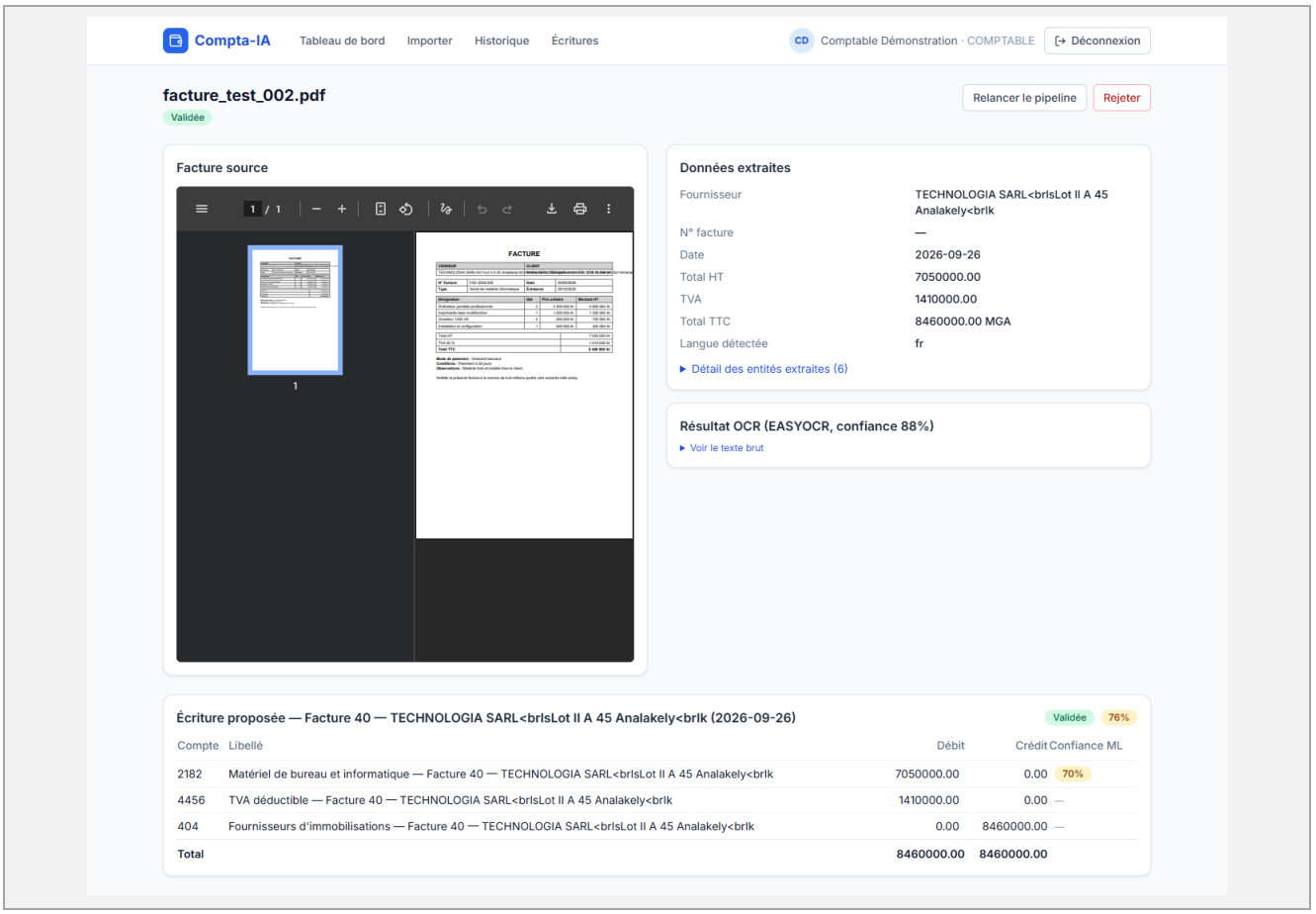


Figure 28: Déroulement d'un scénario de validation fonctionnelle complet

9.3 Évaluation de la performance du module IA

Un protocole précis a été défini pour évaluer objectivement les performances du modèle d'intelligence artificielle : les factures les plus anciennes servent à l'entraîner, et les plus récentes à vérifier qu'il fonctionne bien sur des cas qu'il n'a jamais vus (ce qui est plus réaliste qu'un tirage au hasard, la nature des opérations comptables évoluant dans le temps). Chaque nouvelle version du modèle n'est mise en service que si elle est au moins aussi performante que la version active - une garantie contre toute régression de qualité. Ce protocole a été implémenté et exécuté à plusieurs reprises au cours du développement : trois versions successives du modèle ont ainsi été entraînées et conservées.

Il convient cependant d'être transparent sur une limite assumée : le nombre de factures réelles traitées et validées à ce stade du projet reste très inférieur à la cible de 200 à 300 fixée initialement. Un mécanisme de secours basé sur des mots-clés assure donc encore l'essentiel des propositions, le modèle statistique n'ayant pas encore reçu assez d'exemples valides pour être activé en conditions réelles. Les indicateurs de suivi prévus (taux de propositions acceptées sans correction, performance par version de modèle, temps moyen de traitement) sont déjà calculés par le système, mais une interprétation statistiquement fiable nécessiterait un volume de factures traitées que le calendrier de ce mémoire n'a pas permis d'atteindre. Ce point est repris comme perspective en conclusion.

Le tableau 21 présente les indicateurs mesurés sur les factures traitées par le système, ainsi que la méthode de mesure de chacun.

Tableau 21: Indicateurs d'évaluation du module IA

Indicateur	Méthode de mesure	Résultat
Exactitude de l'extraction par champ (fournisseur, date, numéro, HT, TVA, TTC)	Comparaison des valeurs extraites aux valeurs validées par le comptable	84,8 % (173/204)
Exactitude du type d'opération proposé	Proportion de classifications conservées sans correction	100 % (34/34)
Taux d'écritures validées sans correction	Écritures validées sans correction / écritures validées	88,6 % (31/35)
Temps moyen de traitement automatique d'une facture	Durées journalisées par le pipeline	≈ 1 s (PDF) ≈ 46 s (scan)
Temps moyen de saisie manuelle d'une facture	Chronométrage de la saisie manuelle d'un échantillon de factures	Non mesuré
F1 macro du classifieur statistique	Split temporel 80/20 (table model_versions)	Non mesuré

Nombre de factures évaluées : 34 factures validées, sur 78 importées.

CONCLUSION

Ce travail avait pour objectif de concevoir et de développer un système intelligent capable de générer automatiquement les écritures comptables à partir des factures commerciales, en combinant l'OCR, le NLP et le Machine Learning.

L'analyse du traitement manuel a montré qu'il était lent, exposé aux erreurs de saisie et dépendant de l'expérience du comptable. Pour y répondre, une application web a été réalisée : elle lit automatiquement une facture (PDF ou image), en extrait les informations essentielles, détermine la nature de l'opération et propose une écriture équilibrée conforme au Plan Comptable Général, accompagnée d'un score de confiance. Le comptable garde la décision finale : il valide, corrige ou rejette la proposition, et ses corrections servent à améliorer le modèle.

Les essais menés sur 34 factures validées donnent une exactitude de l'extraction de 84,8 % et 88,6 % d'écritures validées sans correction. Le traitement prend environ une seconde pour un PDF numérique et une quarantaine de secondes pour une image scannée. L'application est déployée sur un serveur distant et s'utilise depuis un simple navigateur, en connexion chiffrée.

Ces résultats restent à confirmer sur un plus grand nombre de factures réelles : le corpus actuel ne suffit pas encore à entraîner le classifieur statistique, et les factures de vente restent à intégrer. Les perspectives portent sur la constitution d'un corpus de 200 à 300 factures réelles annotées, le réentraînement du modèle et l'automatisation du déploiement.

En définitive, ce travail montre qu'une automatisation comptable assistée par l'intelligence artificielle est réalisable, à condition de laisser au comptable le contrôle de chaque écriture.

BIBLIOGRAPHIE

[1] BAEK Y., LEE B., HAN D., YUN S., LEE H., « Character Region Awareness for Text Detection », 2019, Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (IEEE, Long Beach), pp. 9365-9374.

[2] CHEN T., GUESTRIN C., « XGBoost: A Scalable Tree Boosting System », 2016, Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (ACM, San Francisco), pp. 785-794.

[3] FOWLER M., UML Distilled: A Brief Guide to the Standard Object Modeling Language, 3e édition, 2003, Addison-Wesley (Boston), 208 p.

[4] GÉRON A., Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 2e édition, 2019, O'Reilly Media (Sebastopol), 856 p..

[5] MADAGASCAR, Décret n° 2004-272 du 18 février 2004 portant approbation du Plan Comptable Général 2005, 2004, Conseil Supérieur de la Comptabilité (Antananarivo),

[6] MARTIN R. C., Clean Architecture: A Craftsman's Guide to Software Structure and Design, 2017, Prentice Hall (Boston), 432 p.

[7] PEDREGOSA F. et al., « Scikit-learn: Machine Learning in Python », 2011, Journal of Machine Learning Research, vol. 12, pp. 2825-2830.

[8] SMITH R., « An Overview of the Tesseract OCR Engine », 2007, Ninth International Conference on Document Analysis and Recognition (IEEE, Curitiba), pp. 629-633.

WEBOGRAPHIE

[9] https://caddyserver.com/docs/, Caddy Documentation, consulté le 18 septembre 2026.

[10] https://code.visualstudio.com/docs, Visual Studio Code Documentation, consulté le 8 avril 2026.

[11] https://compta-ia.75-119-136-160.nip.io/, Compta-IA – application en production, consulté le 2 octobre 2026.

[12] https://dev.mysql.com/doc/refman/8.0/en/, MySQL 8.0 Reference Manual, consulté le 4 juin 2026.

[13] https://docs.docker.com/, Docker Docs, consulté le 15 septembre 2026.

[14] https://docs.opencv.org/4.x/, OpenCV Documentation, consulté le 21 juillet 2026.

[15] https://docs.python.org/3.10/, Python 3.10 Documentation, consulté le 9 avril 2026.

[16] https://docs.sqlalchemy.org/en/20/, SQLAlchemy 2.0 Documentation, consulté le 16 juillet 2026.

[17] https://fastapi.tiangolo.com/, FastAPI, consulté le 10 juillet 2026.

[18] https://git-scm.com/doc, Git Documentation, consulté le 8 avril 2026.

[19] https://github.com/JaidedAI/EasyOCR, EasyOCR, consulté le 28 juillet 2026.

[20] https://nip.io/, nip.io – Dead simple wildcard DNS for any IP Address, consulté le 18 septembre 2026.

[21] https://react.dev/, React, consulté le 24 juillet 2026.

[22] https://scikit-learn.org/stable/, scikit-learn: machine learning in Python, consulté le 18 août 2026.

[23] https://scrumguides.org/scrum-guide.html, SCHWABER K., SUTHERLAND J., The Scrum Guide, 2020, consulté le 14 avril 2026.

[24] https://spacy.io/, spaCy · Industrial-strength Natural Language Processing in Python, consulté le 5 août 2026.

[25] https://tailwindcss.com/docs, Tailwind CSS Documentation, consulté le 25 juillet 2026.

[26] https://tesseract-ocr.github.io/, Tesseract documentation, consulté le 30 juillet 2026.

[27] https://www.typescriptlang.org/docs/, TypeScript Documentation, consulté le 24 juillet 2026.

[28] https://www.visual-paradigm.com/, Visual Paradigm, consulté le 20 mai 2026.

GLOSSAIRE

Apprentissage automatique (Machine Learning)	Branche de l'intelligence artificielle dans laquelle un modèle apprend, à partir d'exemples, à faire des prédictions ; ici, la nature d'une opération et les comptes à utiliser.
Avoir	Document émis par un fournisseur pour annuler ou réduire tout ou partie d'une facture.
Classe de comptes	Regroupement des comptes du plan comptable par grande catégorie (par exemple, la classe 6 regroupe les charges).
Compte comptable	Rubrique du plan comptable, identifiée par un numéro et un libellé, dans laquelle sont enregistrés les mouvements d'une même nature.
Comptable	Professionnel chargé d'enregistrer les opérations de l'entreprise, de contrôler les écritures et d'établir les comptes.
Conteneur (Docker)	Environnement isolé qui embarque un logiciel et tout ce dont il a besoin pour fonctionner, de façon identique sur n'importe quel serveur.
Crédit	Colonne de droite d'un compte ; une écriture au crédit enregistre une ressource, par exemple une dette envers un fournisseur.
Débit	Colonne de gauche d'un compte ; une écriture au débit enregistre un emploi, par exemple une charge ou l'acquisition d'un bien.
Démarrage à froid (cold start)	Situation d'un système d'apprentissage qui ne dispose pas encore de suffisamment de données pour entraîner un modèle fiable.
Écriture comptable	Enregistrement d'une opération dans la comptabilité, composé de lignes au débit et au crédit de comptes du plan comptable.
Extraction d'entités	Repérage automatique, dans un texte, d'informations précises comme un montant, une date, un numéro de facture ou un nom de fournisseur.
Facture	Document commercial qui détaille les biens ou services vendus, leurs prix, les taxes et le montant à payer.
Fournisseur	Entreprise ou personne qui vend des biens ou des services et émet la facture d'achat.
HT (hors taxes)	Montant d'une facture avant application de la TVA.
Imputation comptable	Choix du compte dans lequel une opération est enregistrée.
Journal comptable	Registre où les écritures sont enregistrées dans l'ordre chronologique.
NIF	Numéro d'identification fiscale attribué à une entreprise à Madagascar.
NLP (traitement automatique du langage naturel)	Ensemble de techniques permettant à un programme d'analyser et d'interpréter un texte écrit en langage humain.

OCR (reconnaissance optique de caractères)	Technique qui transforme l'image d'un document en texte exploitable par un programme.
Partie double	Principe comptable selon lequel toute opération est enregistrée au débit d'au moins un compte et au crédit d'au moins un autre, pour un même montant total.
Plan comptable général (PCG)	Nomenclature officielle des comptes et des règles de comptabilisation ; à Madagascar, le PCG 2005.
Score de confiance	Valeur comprise entre 0 et 1 qui indique le degré de certitude du système sur une proposition.
STAT	Numéro statistique attribué à une entreprise à Madagascar.
Taux de change	Valeur d'une devise exprimée dans une autre, utilisée pour convertir une facture en devise étrangère.
TTC (toutes taxes comprises)	Montant d'une facture TVA incluse ; c'est le montant à payer.
TVA (taxe sur la valeur ajoutée)	Impôt sur la consommation ajouté au prix hors taxes ; la TVA payée sur les achats est déductible.
VPS (serveur privé virtuel)	Machine distante louée auprès d'un hébergeur, sur laquelle l'application est installée.

ANNEXE

A. Extrait du schéma de la base de données (database/schema.sql)

Cet extrait montre les deux tables qui portent l'invariant comptable central du système : une écriture (journal_entries) et ses lignes de partie double (entry_lines), avec la contrainte SQL qui interdit à une ligne d'être à la fois au débit et au crédit.

```
CREATE TABLE journal_entries (
  id               BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  invoice_id       BIGINT UNSIGNED NOT NULL,
  rule_id          BIGINT UNSIGNED NULL,
  entry_date       DATE            NOT NULL,
  label            VARCHAR(255)    NOT NULL,
  status           ENUM('PROPOSED','CORRECTED','VALIDATED')
                   NOT NULL DEFAULT 'PROPOSED',
  confidence_score DECIMAL(5,4)    NULL,
  validated_by     BIGINT UNSIGNED NULL,
  validated_at     DATETIME        NULL,
  created_at       DATETIME        NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id),
  CONSTRAINT fk_entries_invoice FOREIGN KEY (invoice_id)
    REFERENCES invoices(id) ON DELETE RESTRICT
);

CREATE TABLE entry_lines (
  id                 BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  entry_id           BIGINT UNSIGNED NOT NULL,
  account_id         BIGINT UNSIGNED NOT NULL,
  label              VARCHAR(255)    NOT NULL,
  debit              DECIMAL(15,2)   NOT NULL DEFAULT 0.00,
  credit             DECIMAL(15,2)   NOT NULL DEFAULT 0.00,
  account_confidence DECIMAL(5,4)    NULL,
  suggested_by_ml    TINYINT(1)      NOT NULL DEFAULT 0,
  line_order         SMALLINT UNSIGNED NOT NULL DEFAULT 0,
  PRIMARY KEY (id),
  CONSTRAINT fk_lines_entry FOREIGN KEY (entry_id)
    REFERENCES journal_entries(id) ON DELETE CASCADE,
  -- Une ligne est soit au débit, soit au crédit, jamais les deux ni aucun
  CONSTRAINT chk_line_single_side CHECK (
    (debit > 0 AND credit = 0) OR (credit > 0 AND debit = 0)
  )
);
```

B. Fichier de configuration (backend/.env.example)

Ce fichier centralise les paramètres sensibles et les seuils du pipeline IA (jamais versionnés avec des valeurs réelles).

```
DATABASE_URL=mysql+pymysql://compta_user:compta_pass@localhost:3306/compta_ia_db
SECRET_KEY=change-me-in-production
ACCESS_TOKEN_EXPIRE_MINUTES=480
UPLOAD_DIR=./storage/uploads
MODELS_DIR=./storage/models
OCR_LANGUAGES=fr,en
OCR_FALLBACK_THRESHOLD=0.55
OCR_REJECT_THRESHOLD=0.40
DEFAULT_VAT_RATE=0.20
DEFAULT_CURRENCY=MGA
RETRAIN_CORRECTION_THRESHOLD=50
```

C. Liste des points d'entrée de l'API REST

L'API est organisée en quatre routeurs FastAPI, préfixés par /api.

- auth — POST /login, GET /me, POST /users, GET /users, PATCH /users/{id}/active
- invoices — POST /invoices, GET /invoices, GET /invoices/{id}, GET /invoices/{id}/file, GET /invoices/{id}/ocr, GET /invoices/{id}/entities, POST /invoices/{id}/reprocess, POST /invoices/{id}/reject
- entries — GET /entries, GET /entries/by-invoice/{invoice_id}, GET /entries/{id}, PUT /entries/{id}, POST /entries/{id}/validate, GET /entries/export/excel, GET /entries/export/csv, GET /entries/export/pdf
- misc — GET /accounts, GET /dashboard, GET /audit, POST /ml/retrain, GET /ml/versions

D. Code du module d'apprentissage automatique (backend/app/infrastructure/ml)

Le module d'apprentissage automatique propose le type d'opération et le compte comptable. Il contient deux implémentations du même port : un classifieur à mots-clés, déterministe, actif tant que le volume de factures validées est insuffisant (démarrage à froid), et un classifieur scikit-learn (TF-IDF et régression logistique calibrée) chargé depuis un artefact versionné. La suggestion de compte reste contrainte par la liste des comptes candidats de la règle comptable. Le second fichier réalise l'entraînement à partir des corrections de l'utilisateur et la publication d'une nouvelle version du modèle.

D.1 classifiers.py

```
"""Classification du type d'opération et suggestion de comptes (Phase 6,
étapes 6-7).

Deux implémentations du port OperationClassifier :
- KeywordClassifier : version 0.1.0 d'amorçage (cold start), dictionnaire
  multilingue déterministe — active tant que < 150 factures validées ;
- SklearnClassifier : TF-IDF + régression logistique calibrée, chargée
  depuis un artefact joblib versionné (model_versions).

Le suggesteur de comptes est CONTRAINT par la liste `candidates` de la
règle comptable : le ML n'impute jamais hors du cadre du PCG.
"""
import logging
import re
from pathlib import Path

from app.core.config import get_settings
from app.domain.entities import OperationType, Supplier
from app.domain.ports import (
    AccountSuggester, AccountSuggestion, ClassificationResult, OperationClassifier,
)

logger = logging.getLogger(__name__)

# Dictionnaire multilingue (fr / en / mg) par classe d'opération
KEYWORDS: dict[OperationType, list[str]] = {
    OperationType.ACHAT_IMMOBILISATION: [
        "ordinateur", "computer", "laptop", "serveur", "server", "imprimante",
        "printer", "vehicule", "véhicule", "voiture", "camion", "moto", "machine",
        "mobilier", "bureau", "furniture", "climatiseur", "materiel", "matériel",
        "equipement", "équipement", "equipment", "photocopieur", "fiara", "milina",
    ],
    OperationType.ACHAT_SERVICES: [
        "prestation", "service", "honoraires", "consulting", "conseil", "audit",
        "maintenance", "reparation", "réparation", "location", "loyer", "rent",
        "transport", "livraison", "delivery", "formation", "training", "assurance",
        "insurance", "publicite", "publicité", "advertising", "abonnement",
        "internet", "telephone", "téléphone", "electricite", "électricité",
        "jirama", "eau", "hofan-trano", "nettoyage", "gardiennage", "securite",
    ],
    OperationType.ACHAT_MARCHANDISES: [
```

```
        # Bug réel corrigé : "unité", "pièces", "qté", "article", "produit"
        # ont été retirés — ce sont des en-têtes de tableau génériques
        # (« Code article », « Unité de vente », « Qté ») présents sur
        # QUASIMENT TOUTE facture, quelle que soit sa nature. Leur présence
        # ici faisait basculer vers ACHAT_MARCHANDISES même une facture de
        # maintenance ou d'achat d'ordinateur (1 mot-clé métier réel contre
        # 2 mots-clés de tableau génériques). Ne restent que des termes
        # réellement indicatifs d'une revente/négoce de biens.
        "marchandise", "marchandises", "goods", "riz", "vary", "sac", "carton",
        "lot", "entola", "revente",
    ],
}

# Mots-clés → compte précis, pour affiner la suggestion PARMI les candidats
ACCOUNT_KEYWORDS: dict[str, list[str]] = {
    "6061": ["eau", "electricite", "électricité", "jirama", "energie", "énergie", "rano"],
    "6064": ["papier", "stylo", "fourniture", "bureau", "cartouche", "toner"],
    "613":  ["location", "loyer", "bail", "rent", "hofan-trano"],
    "615":  ["entretien", "reparation", "réparation", "maintenance", "repair"],
    "616":  ["assurance", "insurance", "prime"],
    "617":  ["etude", "étude", "recherche", "study"],
    "611":  ["sous-traitance", "subcontract"],
    "621":  ["interim", "intérim", "personnel exterieur"],
    "622":  ["honoraires", "avocat", "notaire", "expert", "comptable", "consulting", "conseil",
"audit"],
    "623":  ["publicite", "publicité", "advertising", "annonce", "affiche", "marketing"],
    "624":  ["transport", "livraison", "fret", "delivery", "shipping"],
    "625":  ["mission", "deplacement", "déplacement", "hotel", "hôtel", "restaurant", "billet"],
    "626":  ["telephone", "téléphone", "internet", "timbre", "postal", "telma", "orange", "airtel"],
    "627":  ["frais bancaires", "commission", "agios", "banque"],
    "2181": ["vehicule", "véhicule", "voiture", "camion", "moto", "fiara"],
    "2182": ["ordinateur", "computer", "laptop", "serveur", "imprimante", "printer", "scanner"],
    "2183": ["mobilier", "bureau", "chaise", "table", "armoire", "furniture"],
    "204":  ["logiciel", "software", "licence", "license"],
}


class KeywordClassifier(OperationClassifier):
    """Classifieur d'amorçage : score = proportion de mots-clés de la classe
    présents dans le texte. Déterministe, explicable, multilingue."""

    VERSION = "keyword-0.1.0"

    def classify(self, text: str, supplier_name: str | None,
                 amount: float | None) -> ClassificationResult:
        lowered = f"{text} {supplier_name or ''}".lower()
        scores: dict[str, float] = {}
        for operation, keywords in KEYWORDS.items():
            hits = sum(1 for kw in keywords if kw in lowered)
            scores[operation.value] = hits
        total_hits = sum(scores.values())
        if total_hits == 0:
            # aucun indice : AUTRE avec probabilité faible → vigilance du comptable
            probabilities = {op.value: 0.25 for op in OperationType}
            return ClassificationResult(operation_type=OperationType.AUTRE,
                                        probability=0.25, probabilities=probabilities,
                                        model_version=self.VERSION)
        probabilities = {op: round(score / total_hits, 4) for op, score in scores.items()}
        probabilities[OperationType.AUTRE.value] = 0.0
        best = max(scores, key=scores.get)
        return ClassificationResult(
            operation_type=OperationType(best),
            probability=min(0.85, probabilities[best]),  # plafond : humilité du cold start
            probabilities=probabilities, model_version=self.VERSION)


class SklearnClassifier(OperationClassifier):
    """Classifieur statistique chargé depuis un artefact joblib
    (pipeline TF-IDF + LogisticRegression calibrée, produit par training.py)."""

    def __init__(self, artifact_path: str, version: str):
        import joblib
```

```
        self.pipeline = joblib.load(artifact_path)
        self.version = version

    def classify(self, text: str, supplier_name: str | None,
                 amount: float | None) -> ClassificationResult:
        document = f"{text} {supplier_name or ''}"
        probabilities = self.pipeline.predict_proba([document])[0]
        classes = list(self.pipeline.classes_)
        proba_map = {cls: round(float(p), 4) for cls, p in zip(classes, probabilities)}
        best = max(proba_map, key=proba_map.get)
        return ClassificationResult(operation_type=OperationType(best),
                                    probability=proba_map[best],
                                    probabilities=proba_map, model_version=self.version)


def load_active_classifier(db=None) -> OperationClassifier:
    """Charge la version active du classifieur depuis model_versions,
    sinon retombe sur le classifieur à mots-clés (cold start)."""
    if db is not None:
        try:
            from sqlalchemy import select
            from app.infrastructure.db.models import ModelVersionModel
            row = db.scalar(select(ModelVersionModel).where(
                ModelVersionModel.name == "operation_classifier",
                ModelVersionModel.is_active.is_(True)))
            if row and row.artifact_path != "builtin" and Path(row.artifact_path).exists():
                return SklearnClassifier(row.artifact_path, row.version)
        except Exception as exc:
            logger.warning("Chargement du modèle actif impossible (%s) : "
                           "repli sur le classifieur à mots-clés", exc)
    return KeywordClassifier()


# « Téléphone : … », « E-mail : … » : étiquettes de coordonnées, jusqu'à la fin de la ligne
CONTACT_LINE_RE = re.compile(
    r"\b(?:t[ée]l[ée]phone|t[ée]l|phone|mobile|fax|e-?mail|courriel|site\s*web)\b\s*:[^\n]*")


class ConstrainedAccountSuggester(AccountSuggester):
    """Suggère le compte précis PARMI les candidats de la règle :
    1. compte habituel du fournisseur s'il figure dans les candidats (0,90) ;
    2. mots-clés du libellé → compte (0,75) ;
    3. compte par défaut de la règle (0,50)."""

    def __init__(self, account_code_by_id: dict[int, str] | None = None,
                 keywords: dict[str, list[str]] | None = None):
        self.account_code_by_id = account_code_by_id or {}
        # Mots-clés propres au plan comptable de l'entreprise (défaut : plan malgache)
        self.keywords = keywords if keywords is not None else ACCOUNT_KEYWORDS

    def suggest(self, text: str, supplier: Supplier | None,
                candidates: list[str], default: str) -> AccountSuggestion:
        # 1. Historique fournisseur (signal le plus fiable)
        if supplier and supplier.default_account_id:
            code = self.account_code_by_id.get(supplier.default_account_id)
            if code and code in candidates:
                return AccountSuggestion(account_code=code, confidence=0.90,
                                         suggested_by_ml=True)
        # 2. Mots-clés du libellé (hors lignes de contact : « Téléphone : 034… » dans l'en-tête
        #    du vendeur ne dit rien de la nature de l'achat)
        lowered = CONTACT_LINE_RE.sub(" ", text.lower())
        best_code, best_hits = None, 0
        for code in candidates:
            hits = sum(1 for kw in self.keywords.get(code, []) if kw in lowered)
            if hits > best_hits:
                best_code, best_hits = code, hits
        if best_code:
            return AccountSuggestion(account_code=best_code,
                                     confidence=min(0.75, 0.5 + 0.1 * best_hits),
                                     suggested_by_ml=True)
        # 3. Défaut de la règle
        return AccountSuggestion(account_code=default, confidence=0.50,
```

```
                                    suggested_by_ml=False)
```

D.2 training.py

```
"""Réentraînement du classifieur d'opérations à partir des validations et
corrections du comptable (Phase 6, étapes 9-10).

Protocole anti-régression validé :
- split TEMPOREL 80/20 (les factures récentes en test) ;
- la nouvelle version n'est activée que si son F1 macro ≥ celui de la
  version active ;
- artefact joblib versionné, jamais d'écrasement (rollback possible).
"""
import logging
from datetime import datetime
from pathlib import Path

from sqlalchemy import select
from sqlalchemy.orm import Session

from app.core.config import get_settings
from app.infrastructure.db import models as m

logger = logging.getLogger(__name__)

MIN_TRAINING_SAMPLES = 30
MODEL_NAME = "operation_classifier"


def collect_training_data(db: Session) -> tuple[list[str], list[str]]:
    """Construit le jeu d'entraînement : pour chaque facture dont l'écriture
    est VALIDÉE, le texte OCR + fournisseur → le type d'opération réellement
    retenu (la classification d'origine, corrigée le cas échéant)."""
    documents, labels = [], []
    rows = db.execute(
        select(m.InvoiceModel, m.ClassificationModel, m.OcrResultModel)
        .join(m.ClassificationModel, m.ClassificationModel.invoice_id == m.InvoiceModel.id)
        .join(m.OcrResultModel, m.OcrResultModel.invoice_id == m.InvoiceModel.id)
        .where(m.InvoiceModel.status == "VALIDATED")
        .order_by(m.InvoiceModel.id)
    ).all()
    for invoice, classification, ocr in rows:
        supplier_name = invoice.supplier.name if invoice.supplier else ""
        documents.append(f"{ocr.raw_text} {supplier_name}")
        labels.append(classification.operation_type)
    return documents, labels


def _split_temporal(documents: list[str], labels: list[str]):
    """Découpage 80/20 TEMPOREL (pas aléatoire) : les factures les plus
    récentes en test, pour refléter la dérive réelle de distribution plutôt
    qu'un mélange artificiellement optimiste."""
    split = int(len(documents) * 0.8)
    return documents[:split], documents[split:], labels[:split], labels[split:]


def train_and_evaluate(documents: list[str], labels: list[str]):
    """Baseline (étape 6.1 de la spec IA) : TF-IDF + régression logistique
    calibrée. Simple, rapide, nativement bien calibrée — la probabilité
    calibrée alimente directement le score de confiance (étape 8)."""
    from sklearn.calibration import CalibratedClassifierCV
    from sklearn.feature_extraction.text import TfidfVectorizer
    from sklearn.linear_model import LogisticRegression
    from sklearn.metrics import accuracy_score, f1_score
    from sklearn.pipeline import Pipeline

    x_train, x_test, y_train, y_test = _split_temporal(documents, labels)

    pipeline = Pipeline([
        ("tfidf", TfidfVectorizer(ngram_range=(1, 2), max_features=5000,
                                  sublinear_tf=True)),
        ("clf", CalibratedClassifierCV(
```

```
            LogisticRegression(max_iter=1000, class_weight="balanced"), cv=3)),
    ])
    pipeline.fit(x_train, y_train)
    predictions = pipeline.predict(x_test)
    return (pipeline,
            round(accuracy_score(y_test, predictions), 4),
            round(f1_score(y_test, predictions, average="macro"), 4))


class LabelDecodingPipeline:
    """Enveloppe autour d'un pipeline sklearn entraîné sur des labels
    entiers — nécessaire pour XGBoost : contrairement à la régression
    logistique, son ré-ajustement interne par `CalibratedClassifierCV` sur
    chaque pli de validation croisée exige des classes entières continues
    (0..n-1), pas les chaînes ACHAT_MARCHANDISES/... Cette enveloppe
    redonne les libellés d'origine en sortie (`classes_`, `predict`) pour
    exposer exactement le même contrat que la régression logistique côté
    `SklearnClassifier.classify()` — interchangeable sans code spécifique."""

    def __init__(self, pipeline, encoder):
        self._pipeline = pipeline
        self._encoder = encoder
        self.classes_ = encoder.classes_

    def predict(self, x):
        return self._encoder.inverse_transform(self._pipeline.predict(x))

    def predict_proba(self, x):
        # Les colonnes de predict_proba suivent l'ordre croissant des labels
        # entiers, qui est celui de encoder.classes_ (LabelEncoder trie les
        # classes d'origine par ordre alphabétique avant de les numéroter) —
        # aucune permutation supplémentaire n'est donc nécessaire ici.
        return self._pipeline.predict_proba(x)


def train_and_evaluate_xgboost(documents: list[str], labels: list[str]):
    """Modèle avancé (étape 6.2 de la spec IA) : TF-IDF + XGBoost, sur le
    MÊME split temporel que la baseline pour une comparaison à données
    égales. XGBoost n'est pas nativement calibré (contrairement à la
    régression logistique) : `CalibratedClassifierCV` est indispensable
    ici puisque la probabilité de classe alimente le score de confiance
    global — un modèle non calibré fausserait ce score."""
    from sklearn.calibration import CalibratedClassifierCV
    from sklearn.feature_extraction.text import TfidfVectorizer
    from sklearn.metrics import accuracy_score, f1_score
    from sklearn.pipeline import Pipeline
    from sklearn.preprocessing import LabelEncoder
    from xgboost import XGBClassifier

    x_train, x_test, y_train, y_test = _split_temporal(documents, labels)

    encoder = LabelEncoder()
    y_train_encoded = encoder.fit_transform(y_train)

    pipeline = Pipeline([
        ("tfidf", TfidfVectorizer(ngram_range=(1, 2), max_features=5000,
                                  sublinear_tf=True)),
        ("clf", CalibratedClassifierCV(
            XGBClassifier(n_estimators=100, max_depth=4, learning_rate=0.1,
                          eval_metric="mlogloss"), cv=3)),
    ])
    pipeline.fit(x_train, y_train_encoded)
    wrapped = LabelDecodingPipeline(pipeline, encoder)
    predictions = wrapped.predict(x_test)
    return (wrapped,
            round(accuracy_score(y_test, predictions), 4),
            round(f1_score(y_test, predictions, average="macro"), 4))


CANDIDATE_ALGORITHMS = [
    ("tfidf+logreg-calibrated", train_and_evaluate),
    ("tfidf+xgboost-calibrated", train_and_evaluate_xgboost),
```

```
]


def retrain(db: Session) -> dict:
    """Point d'entrée du réentraînement (déclenché par l'Admin ou par seuil).

    Démarche comparative de l'étape 6 de la spec IA : baseline (régression
    logistique) ET modèle avancé (XGBoost) sont entraînés sur le MÊME split
    temporel, chacun versionné individuellement dans `model_versions` — le
    tableau de versions constitue lui-même la trace de la comparaison. Seul
    le meilleur des deux (F1 macro) est comparé à la version active pour
    décider d'une promotion (protection anti-régression) ; en cas d'égalité,
    la régression logistique est préférée (plus interprétable, nativement
    calibrée — cf. justification de la spec)."""
    documents, labels = collect_training_data(db)
    if len(documents) < MIN_TRAINING_SAMPLES or len(set(labels)) < 2:
        return {"status": "SKIPPED",
                "reason": f"Données insuffisantes ({len(documents)} exemples, "
                          f"{len(set(labels))} classes) — minimum {MIN_TRAINING_SAMPLES} et 2
classes."}

    import joblib

    active = db.scalar(select(m.ModelVersionModel).where(
        m.ModelVersionModel.name == MODEL_NAME,
        m.ModelVersionModel.is_active.is_(True)))
    active_f1 = float(active.f1_score) if active and active.f1_score is not None else 0.0

    count = db.scalar(select(m.ModelVersionModel.id).where(
        m.ModelVersionModel.name == MODEL_NAME).order_by(
        m.ModelVersionModel.id.desc()).limit(1)) or 0
    models_dir = Path(get_settings().models_dir)
    models_dir.mkdir(parents=True, exist_ok=True)

    candidates = []  # [(algorithm, version, pipeline, accuracy, f1)]
    for i, (algorithm, train_fn) in enumerate(CANDIDATE_ALGORITHMS, start=1):
        pipeline, accuracy, f1 = train_fn(documents, labels)
        version = f"1.{count + i}.0"
        candidates.append((algorithm, version, pipeline, accuracy, f1))

    # Meilleur candidat = F1 macro le plus haut ; égalité → régression
    # logistique (premier de la liste, cf. docstring).
    best = max(candidates, key=lambda c: c[4])
    promote = best[4] >= active_f1
    if promote and active:
        active.is_active = False

    for algorithm, version, pipeline, accuracy, f1 in candidates:
        artifact_path = str(models_dir / f"{MODEL_NAME}-{version}.joblib")
        joblib.dump(pipeline, artifact_path)
        is_winner_and_promoted = promote and version == best[1]
        db.add(m.ModelVersionModel(
            name=MODEL_NAME, version=version, algorithm=algorithm,
            artifact_path=artifact_path, accuracy=accuracy, f1_score=f1,
            training_samples=len(documents), is_active=is_winner_and_promoted,
            trained_at=datetime.utcnow()))

    # marquer les corrections consommées
    for correction in db.scalars(select(m.CorrectionModel).where(
            m.CorrectionModel.used_for_training.is_(False))).all():
        correction.used_for_training = True
    db.commit()

    logger.info("Réentraînement comparatif : %s",
                {algo: f1 for algo, _, _, _, f1 in candidates})
    return {"status": "PROMOTED" if promote else "TRAINED_NOT_PROMOTED",
            "version": best[1], "algorithm": best[0],
            "accuracy": best[3], "f1_score": best[4],
            "previous_f1": active_f1, "training_samples": len(documents),
            "candidates": [{"algorithm": algo, "version": v, "accuracy": acc, "f1_score": f1}
                           for algo, v, _, acc, f1 in candidates]}
```

E. Code du module de reconnaissance optique de caractères (backend/app/infrastructure/ocr)

Ce module transforme un fichier (PDF ou image) en texte positionné. Le prétraitement améliore l'image avant la lecture ; le texte natif des PDF numériques est lu directement ; l'orchestrateur enchaîne les moteurs (EasyOCR puis Tesseract en secours) selon les seuils de confiance définis dans le fichier de configuration.

E.1 preprocessing.py

```
"""Prétraitement d'image (Phase 6, étape 1) — chaîne OpenCV :
gris → redressement (Hough) → débruitage (non-local means) →
binarisation adaptative, avec contrôle de netteté (variance du Laplacien)."""
import logging
from pathlib import Path

import cv2
import numpy as np

logger = logging.getLogger(__name__)

SHARPNESS_THRESHOLD = 100.0  # variance du Laplacien sous laquelle l'image est jugée floue


def pdf_to_images(pdf_path: str, dpi: int = 300) -> list[str]:
    """Convertit chaque page d'un PDF en image PNG 300 DPI (optimum
    précision/temps pour l'OCR). Implémenté avec PyMuPDF (wheel pip
    autonome) plutôt que pdf2image, qui exige le binaire externe Poppler —
    source de friction de déploiement, notamment sous Windows."""
    import fitz  # PyMuPDF

    paths = []
    zoom = dpi / 72.0  # PyMuPDF raisonne en points (72 dpi de base)
    matrix = fitz.Matrix(zoom, zoom)
    with fitz.open(pdf_path) as document:
        for i, page in enumerate(document):
            pixmap = page.get_pixmap(matrix=matrix)
            out = str(Path(pdf_path).with_suffix(f".page{i}.png"))
            pixmap.save(out)
            paths.append(out)
    return paths


def _deskew(gray: np.ndarray) -> np.ndarray:
    """Redressement par transformée de Hough : détecte les lignes quasi
    horizontales et corrige l'angle médian si l'inclinaison dépasse 0,3°."""
    edges = cv2.Canny(gray, 50, 150)
    lines = cv2.HoughLinesP(edges, 1, np.pi / 180, threshold=100,
                            minLineLength=gray.shape[1] // 4, maxLineGap=20)
    if lines is None:
        return gray
    # cv2.HoughLinesP renvoie tantôt (N, 1, 4), tantôt (N, 4) selon la
    # version d'OpenCV : reshape(-1, 4) normalise avant l'itération
    # (sans quoi lines[:, 0] peut ne livrer qu'une colonne de scalaires,
    # provoquant un TypeError au dépaquetage).
    angles = []
    for x1, y1, x2, y2 in lines.reshape(-1, 4):
        angle = np.degrees(np.arctan2(y2 - y1, x2 - x1))
        if abs(angle) < 15:  # on ne garde que les lignes quasi horizontales
            angles.append(angle)
    if not angles:
        return gray
    median_angle = float(np.median(angles))
    if abs(median_angle) < 0.3:
        return gray
    h, w = gray.shape
    matrix = cv2.getRotationMatrix2D((w / 2, h / 2), median_angle, 1.0)
    return cv2.warpAffine(gray, matrix, (w, h), flags=cv2.INTER_CUBIC,
```

```
                            borderMode=cv2.BORDER_REPLICATE)


def preprocess_image(image_path: str) -> tuple[str, dict]:
    """Applique la chaîne complète et retourne (chemin_image_traitée,
    métadonnées journalisées pour la reproductibilité)."""
    image = cv2.imread(image_path)
    if image is None:
        raise ValueError(f"Image illisible : {image_path}")

    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    sharpness = float(cv2.Laplacian(gray, cv2.CV_64F).var())

    gray = _deskew(gray)
    denoised = cv2.fastNlMeansDenoising(gray, h=10)
    binary = cv2.adaptiveThreshold(denoised, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                   cv2.THRESH_BINARY, blockSize=31, C=11)

    out_path = str(Path(image_path).with_suffix(".prep.png"))
    cv2.imwrite(out_path, binary)

    meta = {"sharpness": round(sharpness, 1),
            "low_quality": sharpness < SHARPNESS_THRESHOLD,
            "steps": ["grayscale", "deskew_hough", "nlmeans_denoise",
                      "adaptive_threshold_31_11"]}
    if meta["low_quality"]:
        logger.warning("Image de faible netteté (%.1f) : %s", sharpness, image_path)
    return out_path, meta
```

E.2 pdf_text.py

```
"""Lecture directe de la couche texte d'un PDF « numérique ».

Un PDF produit par un logiciel de facturation (et non un scan) contient déjà
son texte, avec la position exacte de chaque mot. Le rasteriser puis le relire
par OCR ne peut qu'introduire des erreurs (« €4,200.00 » lu « 44,200.00 »).
Ce module restitue cette couche texte sous la forme habituelle d'un résultat
OCR (blocs + coordonnées), pour que l'extraction en aval n'ait pas à savoir
d'où vient le texte ; il renvoie None pour un scan ou une photo, auquel cas
le pipeline retombe sur l'OCR comme avant.

Toutes les pages sont lues : les coordonnées verticales sont décalées de la
hauteur des pages précédentes, de sorte que la page 2 se place sous la page 1
(les heuristiques « le plus bas de la page » gardent leur sens)."""
import logging
import time

from app.domain.ports import OcrBlock, OcrResult

logger = logging.getLogger(__name__)

ENGINE_NAME = "PDF_TEXT"
MIN_WORDS = 25            # en dessous : scan (ou page quasi vide) -> OCR
MAX_GARBAGE_RATIO = 0.10  # police sans table Unicode : texte illisible -> OCR
TEXT_LAYER_CONFIDENCE = 0.99  # le texte est exact ; seule l'extraction peut se tromper


def read_pdf_text_layer(pdf_path: str) -> OcrResult | None:
    import fitz  # PyMuPDF

    start = time.perf_counter()
    blocks: list[OcrBlock] = []
    garbage_chars = total_chars = 0
    y_offset = 0.0
    try:
        with fitz.open(pdf_path) as document:
            for page in document:
                lines: dict[tuple[int, int], list[tuple]] = {}
                for x0, y0, x1, y1, word, block_no, line_no, _ in page.get_text("words"):
                    lines.setdefault((block_no, line_no), []).append((x0, y0, x1, y1, word))
                    total_chars += len(word)
                    garbage_chars += word.count("") + word.count("(cid:")
```

```
                for words in lines.values():
                    words.sort(key=lambda w: w[0])
                    x0 = min(w[0] for w in words)
                    x1 = max(w[2] for w in words)
                    y0 = min(w[1] for w in words) + y_offset
                    y1 = max(w[3] for w in words) + y_offset
                    blocks.append(OcrBlock(
                        text=" ".join(w[4] for w in words),
                        bbox=[[x0, y0], [x1, y0], [x1, y1], [x0, y1]],
                        confidence=TEXT_LAYER_CONFIDENCE))
                y_offset += page.rect.height
    except Exception as exc:  # PDF protégé, corrompu... : l'OCR prendra le relais
        logger.warning("Couche texte du PDF illisible (%s) : repli sur l'OCR", exc)
        return None

    word_count = sum(len(b.text.split()) for b in blocks)
    if word_count < MIN_WORDS:
        return None
    if total_chars and garbage_chars / total_chars > MAX_GARBAGE_RATIO:
        logger.info("Couche texte inexploitable (encodage) : repli sur l'OCR")
        return None

    # Ordre de lecture : de haut en bas, puis de gauche à droite
    blocks.sort(key=lambda b: (round(b.bbox[0][1] / 4), b.bbox[0][0]))
    return OcrResult(engine=ENGINE_NAME,
                     raw_text="\n".join(b.text for b in blocks),
                     mean_confidence=TEXT_LAYER_CONFIDENCE, blocks=blocks,
                     duration_ms=int((time.perf_counter() - start) * 1000))
```

E.3 engines.py

```
"""Adaptateurs OCR — EasyOCR (principal) et Tesseract (fallback),
derrière le port OCREngine, plus l'orchestrateur de fallback (Phase 6, étape 2).

Les modèles EasyOCR sont chargés paresseusement et gardés en mémoire
(singleton par groupe de langues) : les recharger à chaque facture
coûterait 10 à 20 secondes par requête."""
import logging
import threading
import time

from app.core.config import get_settings
from app.domain.ports import OCREngine, OcrBlock, OcrResult

logger = logging.getLogger(__name__)

_easyocr_readers: dict[str, object] = {}
# FastAPI exécute les BackgroundTasks synchrones dans un threadpool : sans
# verrou, plusieurs factures traitées simultanément peuvent chacune rater le
# cache au même instant et déclencher des téléchargements concurrents du
# même modèle (constaté en conditions réelles — plusieurs "Downloading
# detection model" en parallèle dans les logs).
_easyocr_lock = threading.Lock()
# Second verrou, distinct : une fois le Reader créé, plusieurs factures
# importées ensemble (import multiple) peuvent être traitées en parallèle
# et appeler reader.readtext() simultanément sur la MÊME instance PyTorch —
# l'inférence concurrente sur un modèle partagé n'est pas garantie
# thread-safe côté PyTorch (buffers internes potentiellement mutés). On
# sérialise donc l'inférence elle-même ; le reste du pipeline (extraction,
# classification, écriture) continue de tourner en parallèle par facture.
_easyocr_inference_lock = threading.Lock()


def _get_easyocr_reader(languages: list[str]):
    key = ",".join(sorted(languages))
    reader = _easyocr_readers.get(key)
    if reader is not None:
        return reader
    with _easyocr_lock:
        # Second contrôle : un autre thread a pu terminer l'initialisation
        # pendant qu'on attendait le verrou.
        reader = _easyocr_readers.get(key)
```

```
        if reader is None:
            import easyocr
            logger.info("Chargement du Reader EasyOCR pour les langues %s (CPU)", key)
            reader = easyocr.Reader(languages, gpu=False, verbose=False)
            _easyocr_readers[key] = reader
    return reader


class EasyOCRAdapter(OCREngine):
    """Moteur principal : détecteur CRAFT + reconnaissance CRNN,
    robuste aux photos de smartphone et multilingue."""

    def extract_text(self, image_path: str, languages: list[str]) -> OcrResult:
        start = time.perf_counter()
        reader = _get_easyocr_reader(languages)
        with _easyocr_inference_lock:
            raw = reader.readtext(image_path)  # [(bbox, text, confidence), ...]
        blocks = [OcrBlock(text=text, bbox=[[float(x), float(y)] for x, y in bbox],
                           confidence=float(conf))
                  for bbox, text, conf in raw]
        mean_conf = sum(b.confidence for b in blocks) / len(blocks) if blocks else 0.0
        return OcrResult(engine="EASYOCR",
                         raw_text="\n".join(b.text for b in blocks),
                         mean_confidence=round(mean_conf, 4), blocks=blocks,
                         duration_ms=int((time.perf_counter() - start) * 1000))


class TesseractAdapter(OCREngine):
    """Moteur de repli : Tesseract 5 (LSTM), rapide en CPU sur scans propres."""

    def extract_text(self, image_path: str, languages: list[str]) -> OcrResult:
        import pytesseract
        from PIL import Image

        start = time.perf_counter()
        lang_map = {"fr": "fra", "en": "eng"}
        tess_langs = "+".join(lang_map.get(lang, lang) for lang in languages)
        data = pytesseract.image_to_data(Image.open(image_path), lang=tess_langs,
                                         output_type=pytesseract.Output.DICT)
        blocks, current_line, line_key = [], [], None
        for i, word in enumerate(data["text"]):
            if not word.strip():
                continue
            conf = float(data["conf"][i])
            if conf < 0:
                continue
            key = (data["block_num"][i], data["line_num"][i])
            x, y, w, h = (data["left"][i], data["top"][i],
                          data["width"][i], data["height"][i])
            token = {"text": word, "conf": conf / 100.0,
                     "bbox": [[x, y], [x + w, y], [x + w, y + h], [x, y + h]]}
            if key != line_key and current_line:
                blocks.append(_merge_line(current_line))
                current_line = []
            line_key = key
            current_line.append(token)
        if current_line:
            blocks.append(_merge_line(current_line))
        mean_conf = sum(b.confidence for b in blocks) / len(blocks) if blocks else 0.0
        return OcrResult(engine="TESSERACT",
                         raw_text="\n".join(b.text for b in blocks),
                         mean_confidence=round(mean_conf, 4), blocks=blocks,
                         duration_ms=int((time.perf_counter() - start) * 1000))


def _merge_line(tokens: list[dict]) -> OcrBlock:
    """Fusionne les mots d'une même ligne Tesseract en un bloc, pour obtenir
    la même granularité que les blocs EasyOCR (une ligne visuelle)."""
    xs = [pt[0] for tok in tokens for pt in tok["bbox"]]
    ys = [pt[1] for tok in tokens for pt in tok["bbox"]]
    return OcrBlock(text=" ".join(tok["text"] for tok in tokens),
                    bbox=[[min(xs), min(ys)], [max(xs), min(ys)],
```

```
                        [max(xs), max(ys)], [min(xs), max(ys)]],
                    confidence=sum(tok["conf"] for tok in tokens) / len(tokens))


class OcrOrchestrator:
    """Stratégie de fallback validée en Phase 6 :
    EasyOCR d'abord ; si confiance < seuil (0,55) ou < 10 mots → Tesseract ;
    on garde le meilleur des deux ; si les deux < 0,40 → échec (illisible)."""

    def __init__(self, primary: OCREngine | None = None,
                 fallback: OCREngine | None = None):
        self.primary = primary or EasyOCRAdapter()
        self.fallback = fallback or TesseractAdapter()
        self.settings = get_settings()

    def run(self, image_path: str, languages: list[str]) -> OcrResult | None:
        try:
            result = self.primary.extract_text(image_path, languages)
        except Exception as exc:
            # Le moteur principal peut échouer pour des raisons indépendantes
            # de la qualité de l'image (dépendance absente, modèle non
            # téléchargé...) : on dégrade vers le fallback plutôt que de
            # faire échouer tout le pipeline.
            logger.warning("Moteur OCR principal indisponible (%s) : fallback Tesseract", exc)
            result = None

        if result is not None:
            word_count = len(result.raw_text.split())
            if result.mean_confidence >= self.settings.ocr_fallback_threshold and word_count >= 10:
                return result
            logger.info("Confiance EasyOCR insuffisante (%.2f, %d mots) : fallback Tesseract",
                        result.mean_confidence, word_count)

        try:
            fallback_result = self.fallback.extract_text(image_path, languages)
        except Exception as exc:  # Tesseract absent du système, par exemple
            logger.warning("Fallback Tesseract indisponible : %s", exc)
            fallback_result = None

        if result is None and fallback_result is None:
            return None  # les deux moteurs sont indisponibles : échec explicite

        best = result if result is not None else fallback_result
        if (fallback_result is not None and result is not None
                and fallback_result.mean_confidence > result.mean_confidence):
            best = fallback_result
        if best.mean_confidence < self.settings.ocr_reject_threshold:
            return None  # facture illisible : le système renonce explicitement
        return best
```

F. Code du module de traitement du langage naturel (backend/app/infrastructure/nlp)

L'extracteur d'entités est hybride : des expressions régulières ancrées sur les coordonnées OCR pour les entités à format fixe (montants, dates, NIF, STAT, numéro de facture) et une reconnaissance d'entités nommées spaCy, facultative, complétée par une heuristique de zone pour le nom du fournisseur. Les ancres sont multilingues (français, anglais, malgache).

F.1 extractor.py

```
"""Extraction d'entités hybride (Phase 6, étape 4) :
- regex ancrées sur les COORDONNÉES OCR pour les entités à format fixe
  (montants, dates, NIF, STAT, n° facture) — un montant est associé à son
  libellé s'ils sont alignés sur la même ligne visuelle ;
- spaCy NER (optionnel) + heuristique de zone pour le nom du fournisseur.
"""
import logging
import re
```

```
from dataclasses import dataclass
from datetime import datetime
from decimal import Decimal, InvalidOperation

from app.domain.entities import DocumentType, ExtractedEntity, ExtractionMethod
from app.domain.ports import EntityExtractor, OcrBlock, OcrResult

logger = logging.getLogger(__name__)

# ---------------------------------------------------------- Ancres multilingues
ANCHORS = {
    "TOTAL_TTC": [r"total\s*ttc", r"net\s*[àa]\s*payer", r"montant\s*ttc",
                  r"amount\s*due", r"grand\s*total", r"totaly", r"total\s*$",
                  r"total\s*due", r"balance\s*due", r"total\s*payable",
                  r"amount\s*payable", r"total\s*[àa]\s*payer"],
    "TOTAL_HT": [r"total\s*h\.?t", r"montant\s*h\.?t", r"sub\s*-?\s*total",
                 r"total\s*hors\s*taxes?", r"sous\s*-?\s*total", r"total\s*net",
                 r"net\s*total", r"base\s*imposable"],
    "TOTAL_VAT": [r"t\.?v\.?a", r"\bvat\b", r"taxe"],
    "INVOICE_NUMBER": [r"facture\s*(n[°ºo]|num|#)", r"invoice\s*(no|#|number)",
                       r"fact\.?\s*n[°ºo]", r"faktiora", r"\bn[°ºo]?\s*:"],
    "NIF": [r"\bn\.?i\.?f\b"],
    "STAT": [r"\bstat\b"],
    "DATE": [r"\bdate\b", r"\bdaty\b", r"\ble\b\s*:"],
}

# Séparateurs de milliers : espace, point, virgule (format anglo-saxon
# « 115,123.75 »), espaces insécables. Sans la virgule, « 115,123.75 » se
# lisait « 3.75 » : toutes les factures au format international échouaient.
AMOUNT_RE = re.compile(r"(\d{1,3}(?:[ .,  ]\d{3})+(?:[.,]\d{1,2})?|\d+(?:[.,]\d{1,2})?)")
DATE_RES = [
    re.compile(r"(\d{1,2})[/\-.](\d{1,2})[/\-.](\d{4})"),
    re.compile(r"(\d{1,2})[/\-.](\d{1,2})[/\-.](\d{2})\b"),
    re.compile(r"(\d{4})-(\d{2})-(\d{2})"),
]
MONTHS = {  # fr / en / mg
    "janvier": 1, "january": 1, "janoary": 1, "février": 2, "fevrier": 2,
    "february": 2, "febroary": 2, "mars": 3, "march": 3, "martsa": 3,
    "avril": 4, "april": 4, "aprily": 4, "mai": 5, "may": 5, "mey": 5,
    "juin": 6, "june": 6, "jona": 6, "juillet": 7, "july": 7, "jolay": 7,
    "août": 8, "aout": 8, "august": 8, "aogositra": 8, "septembre": 9,
    "september": 9, "septambra": 9, "octobre": 10, "october": 10, "oktobra": 10,
    "novembre": 11, "november": 11, "novambra": 11, "décembre": 12,
    "decembre": 12, "december": 12, "desambra": 12,
    # abréviations courantes (fr / en) : « 09 Oct 2026 », « 12 sept. 2026 »
    "jan": 1, "janv": 1, "feb": 2, "fev": 2, "fév": 2, "févr": 2, "fevr": 2,
    "mar": 3, "apr": 4, "avr": 4, "jun": 6, "juil": 7, "jul": 7, "aug": 8,
    "sep": 9, "sept": 9, "oct": 10, "nov": 11, "dec": 12, "déc": 12,
}
# Date dont les composantes ont été séparées par l'OCR en plusieurs blocs
# (« 17 / 08 » + « 2026 ») puis rejointes par un simple espace : séparateurs
# optionnels, année à 4 chiffres obligatoire pour limiter les faux positifs.
DATE_SPLIT_RE = re.compile(r"(?<!\d)(\d{1,2})\s*[/\-.]?\s*(\d{1,2})\s*[/\-.]?\s*(20\d{2})(?!\d)")
TEXT_DATE_RE = re.compile(r"(?<!\d)(\d{1,2})\s+([A-Za-zÀ-ÿ]+)\.?\s+(\d{4})(?!\d)")
# Mention d'exonération de TVA : « TVA non applicable, art. 293 B du CGI »,
# « VAT exempt », « exonéré de TVA ». Le nombre qui l'accompagne (ici un
# numéro d'article de loi) n'est PAS un montant de taxe : la TVA vaut 0.
VAT_EXEMPT_RE = re.compile(
    r"(tva|vat|taxe)[^.:;]{0,40}(non\s+applicable|exon[ée]r|exempt)"
    r"|(non\s+applicable|exon[ée]r|exempt)[^.:;]{0,40}(tva|vat)"
    r"|art(?:icle|\.)?\s*293\s*b", re.IGNORECASE)
# --- Choix entre plusieurs montants candidats --------------------------------
# Une facture comporte souvent plusieurs « totaux » : TTC, net à payer (après
# acompte), net avec escompte, HT avant/après remise... Prendre « le plus bas de
# la page » ou « le plus proche de 70 % de la hauteur » revenait à tirer au sort.
# On classe les libellés (rang 0 = le plus fiable) puis on arbitre par le calcul.
TTC_RANKS = [
    (0, r"total\s*ttc|montant\s*ttc|grand\s*total|total\s*(?:tax|vat)\s*incl|total\s*toutes?\s*taxes"),
    (1, r"net\s*[àa]\s*payer|amount\s*due|total\s*due|balance\s*due|total\s*payable"
        r"|amount\s*payable|total\s*[àa]\s*payer"),
]
HT_RANKS = [
```

```
    (2, r"avant\s*remise|before\s*discount|hors\s*remise|\bbrut\b|\bgross\b"),
    (1, r"sub\s*-?\s*total|sous\s*-?\s*total"),
]
VAT_PRIMARY = r"total\s*(?:tva|vat|tax)|montant\s*(?:de\s*la\s*)?(?:tva|vat)|vat\s*amount|tax\s*amount"
# Libellés qui ne désignent jamais le montant cherché : un escompte est
# conditionnel (jamais le TTC), « N° TVA » est un identifiant, pas une taxe.
EXCLUDED_ANCHORS = {
    "TOTAL_TTC": re.compile(r"escompte|early\s*payment|prompt\s*payment|rebate", re.IGNORECASE),
    "TOTAL_VAT":
re.compile(r"n\s*[°º]|\bid\b|identif|num[ée]ro|number|siret|siren|registration|intracom",
                           re.IGNORECASE),
}
TOLERANCE = Decimal("0.02")  # écart d'arrondi toléré entre HT + TVA et TTC

# Montants purement informatifs (n'entrent pas dans l'écriture) : montrés au comptable
INFO_ANCHORS = {
    "DEPOSIT_PAID": r"acompte|deposit|advance\s*payment",
    "EARLY_PAYMENT_DISCOUNT": r"escompte|early\s*payment\s*discount",
    "NET_TO_PAY":
r"net\s*[àa]\s*payer|reste\s*[àa]\s*payer|amount\s*due|balance\s*due|total\s*due|net\s*payable",
}
# Un « net à payer » assorti d'un escompte est conditionnel : on ne le retient pas.
INFO_EXCLUDED_ANCHORS = {
    "NET_TO_PAY": re.compile(r"escompte|early\s*payment|prompt\s*payment|rebate", re.IGNORECASE),
}

CURRENCY_RE = re.compile(r"(?<![A-Za-z])(ariary|mga|ar|eur|usd|gbp|chf)(?![A-Za-z])|(€|\$|£)",
re.IGNORECASE)

# Titres de document et en-têtes de colonne génériques (garde-fou pour
# l'extraction du fournisseur : un bloc "FACTURE", "FACTURE D'AVOIR",
# "VENDEUR"... n'est jamais un nom de tiers, même s'il satisfait par
# ailleurs l'heuristique de position/casse — bug réel corrigé : sur une
# facture d'avoir, l'en-tête de colonne "VENDEUR" précédait le vrai nom
# du fournisseur et était retourné à sa place).
# Étiquettes qui désignent explicitement l'émetteur de la facture (« Vendeur : Boutique X »).
# Elles doivent porter un « : » : un en-tête de colonne « VENDEUR » (nom en dessous, voisin
# « CLIENT » à droite) n'est pas une étiquette de valeur.
SUPPLIER_LABEL_RE = re.compile(
    r"^\s*(?:vendeur|fournisseur|[ée]metteur|[ée]mise\s+par|seller|supplier|vendor|issued\s+by)"
    r"\s*[: ]\s*(?P<value>.*)$", re.IGNORECASE)

DOCUMENT_TITLE_PATTERNS = [
    r"^factures?\b", r"\bavoir\b", r"note\s*de\s*cr[ée]dit", r"credit\s*note",
    r"^devis\b", r"^invoice\b", r"^quotation\b",
    r"^vendeurs?\b", r"^clients?\b", r"^fournisseurs?\b", r"^acheteurs?\b",
    r"^destinataires?\b",
]
AVOIR_RE = re.compile(r"facture\s*d.?avoir|note\s*de\s*cr[ée]dit|credit\s*note|\bavoir\b",
                      re.IGNORECASE)


def detect_document_type(raw_text: str) -> DocumentType:
    """Une facture d'avoir (note de crédit fournisseur) doit générer une
    écriture au sens inversé (cf. RuleEngine.generate_entry) — on la détecte
    tôt, sur le texte OCR brut, pour piloter à la fois ce comportement et
    l'exclusion du titre de la recherche du nom de fournisseur."""
    if AVOIR_RE.search(raw_text or ""):
        return DocumentType.AVOIR
    return DocumentType.FACTURE


def normalize_amount(raw: str) -> str | None:
    """Normalise un montant : gère espace/point comme séparateurs de milliers
    et virgule/point décimal. Ambiguïté résolue par la position : un séparateur
    suivi d'exactement 3 chiffres en fin de groupe = milliers."""
    text = raw.strip().replace(" ", " ")
    text = re.sub(r"[^\d.,\s]", "", text).strip()
    if not text:
        return None
    # dernier séparateur = décimal s'il est suivi de 1 ou 2 chiffres
    match = re.match(r"^([\d\s.,]*?)[.,](\d{1,2})$", text)
```

```
    if match:
        integer_part = re.sub(r"[\s.,]", "", match.group(1))
        return f"{integer_part}.{match.group(2)}" if integer_part else None
    return re.sub(r"[\s.,]", "", text) or None


_CONFUSABLE_LETTERS = set("OolISB")  # lettres que l'OCR confond avec des chiffres


def _fix_numeric_ocr_errors(text: str) -> str:
    """Corrections O→0, l/I→1, S→5, B→8 uniquement dans les tokens
    majoritairement numériques (jamais dans le texte : noms propres).

    Un préfixe alphabétique réel suivi d'un séparateur et de chiffres
    (« INV-2026-8849 », « FAC-00045 ») est un code de pièce, pas un nombre
    mal lu : le corriger donnait « 1NV-2026-8849 ». Le préfixe n'est
    préservé que s'il contient une lettre non confondable avec un chiffre,
    pour que « OO-123 » (deux zéros mal lus) reste corrigé."""
    def fix_token(token: str) -> str:
        prefix = re.match(r"^([A-Za-z]{2,})[\-/_.]?\d", token)
        if prefix and not set(prefix.group(1)) <= _CONFUSABLE_LETTERS:
            return token
        digits = sum(ch.isdigit() for ch in token)
        if digits >= max(2, len(token) // 2):
            return (token.replace("O", "0").replace("o", "0")
                    .replace("l", "1").replace("I", "1")
                    .replace("S", "5").replace("B", "8"))
        return token
    return " ".join(fix_token(tok) for tok in text.split(" "))


def _block_center_y(block: OcrBlock) -> float:
    return sum(pt[1] for pt in block.bbox) / len(block.bbox)


def _block_center_x(block: OcrBlock) -> float:
    return sum(pt[0] for pt in block.bbox) / len(block.bbox)


def _block_height(block: OcrBlock) -> float:
    ys = [pt[1] for pt in block.bbox]
    return max(ys) - min(ys)


def _same_visual_line(a: OcrBlock, b: OcrBlock) -> bool:
    """Deux blocs sont sur la même ligne visuelle si leurs centres verticaux
    sont distants de moins d'une demi-hauteur de bloc."""
    tolerance = max(_block_height(a), _block_height(b)) * 0.6
    return abs(_block_center_y(a) - _block_center_y(b)) <= tolerance


def _matches_any(text: str, patterns: list[str]) -> bool:
    lowered = text.lower()
    return any(re.search(p, lowered) for p in patterns)


def _visual_lines(blocks: list[OcrBlock]) -> list[list[OcrBlock]]:
    """Regroupe les blocs en lignes visuelles (de gauche à droite).

    L'OCR découpe souvent une même ligne en plusieurs blocs (« 17 / 08 » puis
    « 2026 » ; cellules d'un tableau) : les entités qui s'étalent sur
    plusieurs blocs, comme une date, ne sont visibles qu'une fois la ligne
    reconstituée."""
    lines: list[list[OcrBlock]] = []
    for block in sorted(blocks, key=_block_center_y):
        if lines and _same_visual_line(lines[-1][0], block):
            lines[-1].append(block)
        else:
            lines.append([block])
    return [sorted(line, key=lambda b: b.bbox[0][0]) for line in lines]
```

```
def _is_label_like(text: str) -> bool:
    """Un libellé de total (« Total TTC », « VAT / Sales Tax (Reverse Charge
    0%) ») est court. Une phrase de conditions générales qui contient le mot
    « VAT » ou « total » n'est pas une ancre de montant : sur un document de
    plusieurs pages, ces paragraphes produiraient des candidats parasites."""
    return len(text) <= 70 and len(text.split()) <= 12


def _looks_like_money(raw: str) -> bool:
    """Distingue un montant d'un entier quelconque (fin d'IBAN, quantité,
    numéro de page) : décimales, séparateur de milliers, ou au moins 4 chiffres."""
    digits = re.sub(r"\D", "", raw)
    return bool(re.search(r"[.,]\d{1,2}$", raw.strip()) or re.search(r"\d[ ., ]\d{3}", raw)
                or len(digits) >= 4)


@dataclass
class _Candidate:
    """Un montant candidat pour un total, avec le rang du libellé qui l'a désigné."""
    raw: str
    normalized: str | None
    value: Decimal | None
    block: OcrBlock
    confidence: float
    rank: int
    exempt: bool = False

    @classmethod
    def make(cls, raw: str, block: OcrBlock, confidence: float, rank: int) -> "_Candidate":
        normalized = normalize_amount(raw)
        try:
            value = Decimal(normalized) if normalized else None
        except InvalidOperation:
            value = None
        return cls(raw, normalized, value, block, confidence, rank)


class HybridExtractor(EntityExtractor):
    """Implémentation du port EntityExtractor : regex ancrées + NER optionnel."""

    def __init__(self, use_spacy: bool = True):
        self.use_spacy = use_spacy
        self._nlp = None  # chargé paresseusement

    # ------------------------------------------------------------------ API
    def extract(self, ocr_result: OcrResult, language: str | None) -> list[ExtractedEntity]:
        blocks = [OcrBlock(text=_fix_numeric_ocr_errors(b.text), bbox=b.bbox,
                           confidence=b.confidence) for b in ocr_result.blocks]
        entities: list[ExtractedEntity] = []
        entities += self._extract_amounts(blocks)
        entities += self._extract_informative_amounts(blocks)
        entities += self._extract_identifiers(blocks)
        entities += self._extract_date(blocks)
        entities += self._extract_currency(
            blocks, preferred=[e.source_block["text"] for e in entities
                               if e.entity_type in ("TOTAL_TTC", "TOTAL_HT")])
        supplier = self._extract_supplier(blocks, language)
        if supplier:
            entities.append(supplier)
        return entities

    # ------------------------------------------------- Montants ancrés
    def _extract_amounts(self, blocks: list[OcrBlock]) -> list[ExtractedEntity]:
        max_y = max((_block_center_y(b) for b in blocks), default=0)
        pools = {t: self._collect_candidates(t, blocks) for t in ("TOTAL_TTC", "TOTAL_HT",
"TOTAL_VAT")}

        def position(entity_type: str, c: "_Candidate") -> float:
            # À rang égal : le TTC le plus bas de la page ; HT/TVA vers 70 % de la hauteur
            y = _block_center_y(c.block)
            return y if entity_type == "TOTAL_TTC" else -abs(y - max_y * 0.7)
```

```
        # Arbitrage par le calcul : seul le trio (HT, TVA, TTC) qui vérifie
        # HT + TVA = TTC est cohérent. Indépendant de la langue et de la mise en page.
        chosen: dict[str, _Candidate] = {}
        if all(pools.values()):
            best_key, best = None, None
            for ht in pools["TOTAL_HT"]:
                for vat in pools["TOTAL_VAT"]:
                    for ttc in pools["TOTAL_TTC"]:
                        if abs(ht.value + vat.value - ttc.value) > TOLERANCE:
                            continue
                        key = (-(ht.rank + vat.rank + ttc.rank), position("TOTAL_TTC", ttc),
                               position("TOTAL_HT", ht), position("TOTAL_VAT", vat))
                        if best_key is None or key > best_key:
                            best_key, best = key, (ht, vat, ttc)
            if best:
                chosen = {"TOTAL_HT": best[0], "TOTAL_VAT": best[1], "TOTAL_TTC": best[2]}
        # Sinon (un total manquant, ou aucun trio cohérent) : meilleur candidat par type
        for entity_type, pool in pools.items():
            if entity_type not in chosen and pool:
                chosen[entity_type] = max(pool, key=lambda c: (-c.rank, position(entity_type, c)))

        return [ExtractedEntity(
                    entity_type=t, raw_value=c.raw, normalized_value="0.00" if c.exempt else
c.normalized,
                    confidence=round(c.confidence, 4), method=ExtractionMethod.REGEX,
                    source_block={"bbox": c.block.bbox, "text": c.block.text})
                for t, c in chosen.items()]

    @staticmethod
    def _rank(entity_type: str, anchor_text: str) -> int:
        lowered = anchor_text.lower()
        if entity_type == "TOTAL_TTC":
            return next((r for r, p in TTC_RANKS if re.search(p, lowered)), 2)
        if entity_type == "TOTAL_HT":
            return next((r for r, p in HT_RANKS if re.search(p, lowered)), 0)
        return 0 if re.search(VAT_PRIMARY, lowered) else 1

    def _collect_candidates(self, entity_type: str, blocks: list[OcrBlock]) -> list["_Candidate"]:
        own_candidates, right_candidates, left_candidates = [], [], []
        exempt_anchors = []  # mentions « TVA non applicable » (TVA seulement)
        excluded = EXCLUDED_ANCHORS.get(entity_type)
        for anchor in blocks:
            if not _is_label_like(anchor.text):
                continue
            if not _matches_any(anchor.text, ANCHORS[entity_type]):
                continue
            if excluded and excluded.search(anchor.text):
                continue
            rank = self._rank(entity_type, anchor.text)
            # 1) montant dans le bloc ancre lui-même — on exclut les
            # nombres suivis de '%' : pour l'ancre TVA, "20%" est un taux,
            # pas le montant de la taxe (bug corrigé après tests unitaires).
            is_exempt = entity_type == "TOTAL_VAT" and VAT_EXEMPT_RE.search(anchor.text)
            if is_exempt:
                exempt_anchors.append(anchor)
            own = [] if is_exempt else [
                m for m in AMOUNT_RE.findall(anchor.text)
                if not re.search(re.escape(m) + r"\s*%", anchor.text)]
            if own:
                own_candidates.append(_Candidate.make(own[-1], anchor, anchor.confidence, rank))
            # 2) montant dans un bloc aligné horizontalement (même ligne)
            for other in blocks:
                if other is anchor or not _same_visual_line(anchor, other):
                    continue
                amounts = [m for m in AMOUNT_RE.findall(other.text)
                           if not re.search(re.escape(m) + r"\s*%", other.text)]
                if not amounts:
                    continue
                candidate = _Candidate.make(amounts[-1], other,
                                            min(anchor.confidence, other.confidence), rank)
                if _block_center_x(other) > _block_center_x(anchor):
                    right_candidates.append(candidate)
```

```
                elif _looks_like_money(amounts[-1]):
                    # Bug réel : sur une facture à deux colonnes, « Net
                    # Subtotal » (colonne de droite) partageait sa ligne avec
                    # un IBAN (colonne de gauche) dont le dernier chiffre,
                    # « 2 », était pris pour le montant HT.
                    left_candidates.append(candidate)
        # Priorité systématique au montant ALIGNÉ (ligne voisine) sur un
        # nombre trouvé dans le bloc-ancre lui-même : le second n'est
        # utilisé qu'en repli. Bug réel corrigé : l'OCR peut corrompre
        # un taux ("20%") en un nombre qui ressemble à un montant valide
        # ("209" si le "%" est lu comme un "9"), sans le signe "%" que le
        # garde-fou ci-dessus cherche — cette valeur parasite l'emportait
        # alors à tort sur le véritable montant aligné.
        candidates = [c for c in (right_candidates or left_candidates or own_candidates)
                      if c.value is not None]
        if not candidates and exempt_anchors:
            # Aucun montant de taxe, mais une mention d'exonération : TVA = 0
            # (et non « absente », ce qui ferait appliquer le taux par défaut).
            candidates = [_Candidate("0", "0.00", Decimal("0"), exempt_anchors[0],
                                     exempt_anchors[0].confidence, 0, exempt=True)]
        return candidates

    def _extract_informative_amounts(self, blocks: list[OcrBlock]) -> list[ExtractedEntity]:
        """Acompte déjà versé, escompte : à porter à la connaissance du comptable
        mais jamais dans l'écriture (l'acompte a déjà été enregistré à son
        versement ; l'escompte n'existe que si le client paie avant l'échéance)."""
        found = []
        for entity_type, pattern in INFO_ANCHORS.items():
            entity = None
            for anchor in blocks:
                if entity or not (_is_label_like(anchor.text) and re.search(pattern,
anchor.text.lower())):
                    continue
                excluded = INFO_EXCLUDED_ANCHORS.get(entity_type)
                if excluded and excluded.search(anchor.text):
                    continue
                for other in blocks:
                    if other is anchor or not _same_visual_line(anchor, other):
                        continue
                    if _block_center_x(other) <= _block_center_x(anchor):
                        continue
                    amounts = AMOUNT_RE.findall(other.text)
                    # un escompte est une déduction : montant signé « - », jamais le net à payer voisin
                    if not amounts or (entity_type == "EARLY_PAYMENT_DISCOUNT"
                                       and not re.match(r"\s*[---]", other.text)):
                        continue
                    normalized = normalize_amount(amounts[-1])
                    if normalized:
                        entity = ExtractedEntity(
                            entity_type=entity_type, raw_value=amounts[-1],
normalized_value=normalized,
                            confidence=round(min(anchor.confidence, other.confidence), 4),
                            method=ExtractionMethod.REGEX,
                            source_block={"bbox": other.bbox, "text": other.text})
                        break
            if entity:
                found.append(entity)
        return found

    # ------------------------------------------- NIF / STAT / n° facture
    def _extract_identifiers(self, blocks: list[OcrBlock]) -> list[ExtractedEntity]:
        entities = []
        specs = [("NIF", ANCHORS["NIF"], r"(\d[\d\s]{7,})"),
                 ("STAT", ANCHORS["STAT"], r"([\d][\d\s\-/]{5,})"),
                 ("INVOICE_NUMBER", ANCHORS["INVOICE_NUMBER"], r"([A-Za-z]{0,4}[\-/]?\d[\w\-/]*)")]
        for entity_type, anchors, value_pattern in specs:
            for anchor in blocks:
                if not _matches_any(anchor.text, anchors):
                    continue
                zone = anchor.text
                for other in blocks:
                    if other is not anchor and _same_visual_line(anchor, other):
```

```
                        zone += " " + other.text
                # la valeur suit l'ancre dans la zone
                lowered = zone.lower()
                anchor_end = 0
                for pattern in anchors:
                    match = re.search(pattern, lowered)
                    if match:
                        anchor_end = max(anchor_end, match.end())
                value_match = re.search(value_pattern, zone[anchor_end:])
                if value_match:
                    raw = value_match.group(1).strip()
                    entities.append(ExtractedEntity(
                        entity_type=entity_type, raw_value=raw,
                        normalized_value=re.sub(r"\s", "", raw),
                        confidence=round(anchor.confidence, 4),
                        method=ExtractionMethod.REGEX,
                        source_block={"bbox": anchor.bbox, "text": anchor.text}))
                    break
            # une seule occurrence par type
        return entities

    # ------------------------------------------------------------- Date
    def _extract_date(self, blocks: list[OcrBlock]) -> list[ExtractedEntity]:
        candidates: list[tuple[datetime, OcrBlock, bool, str]] = []
        # Unités de recherche : chaque bloc, plus chaque ligne visuelle
        # reconstituée (une date découpée par l'OCR en « 17 / 08 » + « 2026 »
        # n'existe dans aucun bloc pris isolément).
        units = list(blocks)
        for line in _visual_lines(blocks):
            if len(line) > 1:
                xs = [pt[0] for b in line for pt in b.bbox]
                ys = [pt[1] for b in line for pt in b.bbox]
                units.append(OcrBlock(
                    text=" ".join(b.text for b in line),
                    bbox=[[min(xs), min(ys)], [max(xs), min(ys)], [max(xs), max(ys)], [min(xs),
max(ys)]],
                    confidence=min(b.confidence for b in line)))
        for block in units:
            near_anchor = _matches_any(block.text, ANCHORS["DATE"])
            for regex in DATE_RES + [DATE_SPLIT_RE]:
                for match in regex.finditer(block.text):
                    parsed = self._parse_date(match)
                    if parsed:
                        candidates.append((parsed, block, near_anchor, match.group(0)))
            # dates textuelles : « 15 janvier 2026 », « 09 Oct 2026 », « 12 sept. 2026 »
            for text_match in TEXT_DATE_RE.finditer(block.text):
                month = MONTHS.get(text_match.group(2).lower())
                if month is None:
                    continue
                try:
                    parsed = datetime(int(text_match.group(3)), month, int(text_match.group(1)))
                except ValueError:
                    continue
                candidates.append((parsed, block, near_anchor, text_match.group(0)))
        if not candidates:
            return []
        # plausibilité ± 2 ans, priorité aux dates proches d'une ancre, sinon la plus ancienne
        now = datetime.now()
        plausible = [c for c in candidates if abs((c[0] - now).days) < 730] or candidates
        anchored = [c for c in plausible if c[2]]
        best = min(anchored or plausible, key=lambda c: c[0])
        return [ExtractedEntity(entity_type="INVOICE_DATE", raw_value=best[3],
                                normalized_value=best[0].strftime("%Y-%m-%d"),
                                confidence=round(best[1].confidence, 4),
                                method=ExtractionMethod.REGEX,
                                source_block={"bbox": best[1].bbox, "text": best[1].text})]

    @staticmethod
    def _parse_date(match: re.Match) -> datetime | None:
        groups = match.groups()
        try:
            if len(groups[0]) == 4:  # AAAA-MM-JJ
```

```
            return datetime(int(groups[0]), int(groups[1]), int(groups[2]))
        year = int(groups[2])
        if year < 100:
            year += 2000
        return datetime(year, int(groups[1]), int(groups[0]))  # JJ/MM/AAAA
    except ValueError:
        return None

# ------------------------------------------------------------ Devise
def _extract_currency(self, blocks: list[OcrBlock],
                      preferred: list[str] | None = None) -> list[ExtractedEntity]:
    mapping = {"ariary": "MGA", "mga": "MGA", "ar": "MGA", "eur": "EUR", "€": "EUR",
               "usd": "USD", "$": "USD", "gbp": "GBP", "£": "GBP", "chf": "CHF"}
    # La devise la plus fiable est celle écrite à côté des montants retenus
    # (« 47 104,03 € ») : un « € » isolé en pied de page (capital social)
    # ne doit pas l'emporter sur des montants en Ariary.
    preferred = set(preferred or [])
    ordered = [b for b in blocks if b.text in preferred] + list(blocks)
    for block in ordered:
        match = CURRENCY_RE.search(block.text)
        if match:
            symbol = match.group(1) or match.group(2)
            return [ExtractedEntity(
                entity_type="CURRENCY", raw_value=symbol,
                normalized_value=mapping.get(symbol.lower(), "MGA"),
                confidence=round(block.confidence, 4),
                method=ExtractionMethod.REGEX,
                source_block={"bbox": block.bbox, "text": block.text})]
    return []

# ------------------------------------------------------- Fournisseur
def _extract_supplier(self, blocks: list[OcrBlock],
                      language: str | None) -> ExtractedEntity | None:
    """NER spaCy (ORG) restreint au tiers supérieur de la page ; à défaut,
    heuristique : premier bloc textuel majuscule significatif du haut de page."""
    if not blocks:
        return None
    labelled = self._supplier_from_label(blocks)
    if labelled is not None:
        return labelled
    max_y = max(_block_center_y(b) for b in blocks)
    top_blocks = [b for b in blocks if _block_center_y(b) < max_y / 3
                  and not _matches_any(b.text.strip(), DOCUMENT_TITLE_PATTERNS)]

    if self.use_spacy:
        nlp = self._get_nlp(language)
        if nlp is not None:
            for block in top_blocks:
                doc = nlp(block.text)
                for ent in doc.ents:
                    if ent.label_ == "ORG" and len(ent.text) >= 3:
                        return ExtractedEntity(
                            entity_type="SUPPLIER_NAME", raw_value=ent.text,
                            normalized_value=ent.text.strip().upper(),
                            confidence=round(block.confidence * 0.9, 4),
                            method=ExtractionMethod.NER,
                            source_block={"bbox": block.bbox, "text": block.text})
    for block in top_blocks:
        text = block.text.strip()
        if (len(text) >= 4 and sum(c.isalpha() for c in text) >= 3
                and not _matches_any(text, sum(ANCHORS.values(), []))
                and not AMOUNT_RE.fullmatch(text)):
            return ExtractedEntity(
                entity_type="SUPPLIER_NAME", raw_value=text,
                normalized_value=text.upper(),
                confidence=round(block.confidence * 0.6, 4),
                method=ExtractionMethod.NER,
                source_block={"bbox": block.bbox, "text": block.text})
    return None

@staticmethod
def _supplier_from_label(blocks: list[OcrBlock]) -> ExtractedEntity | None:
```

```
        """Valeur placée après une étiquette explicite (« Vendeur : … »), dans le même
        bloc ou dans le bloc voisin de droite sur la même ligne. C'est le signal le plus
        fiable : il l'emporte sur l'heuristique de position, qui prenait un sous-titre."""
        for label in blocks:
            match = SUPPLIER_LABEL_RE.match(label.text)
            if not match:
                continue
            value, source, confidence = match.group("value").strip(), label, label.confidence
            if not value:
                neighbours = [b for b in blocks if b is not label and _same_visual_line(label, b)
                              and _block_center_x(b) > _block_center_x(label)]
                neighbours.sort(key=_block_center_x)
                for other in neighbours:
                    text = other.text.strip()
                    if (len(text) >= 3 and sum(c.isalpha() for c in text) >= 2
                            and not SUPPLIER_LABEL_RE.match(text) and not text.rstrip().endswith(":")
                            and not _matches_any(text, DOCUMENT_TITLE_PATTERNS)):
                        value, source, confidence = text, other, min(label.confidence,
other.confidence)
                        break
            if len(value) >= 3 and sum(c.isalpha() for c in value) >= 2:
                return ExtractedEntity(
                    entity_type="SUPPLIER_NAME", raw_value=value, normalized_value=value.upper(),
                    confidence=round(confidence * 0.9, 4), method=ExtractionMethod.REGEX,
                    source_block={"bbox": source.bbox, "text": source.text})
        return None

    def _get_nlp(self, language: str | None):
        if self._nlp is None:
            try:
                import spacy
                model = "fr_core_news_md" if (language or "fr").startswith("fr") else "xx_ent_wiki_sm"
                try:
                    self._nlp = spacy.load(model)
                except OSError:
                    logger.warning("Modèle spaCy %s absent : NER désactivé "
                                   "(python -m spacy download %s)", model, model)
                    self._nlp = False
            except ImportError:
                self._nlp = False
        return self._nlp or None


def detect_language(text: str) -> str | None:
    try:
        from langdetect import detect
        return detect(text)
    except Exception:
        return None
```

G. Captures d'écran de la base de données compta_ia_db (phpMyAdmin)

Les figures A.1 à A.4, réalisées dans phpMyAdmin, montrent les 16 tables de la base compta_ia_db avec leur nombre de lignes, puis la structure des tables qui portent le coeur du modèle comptable.

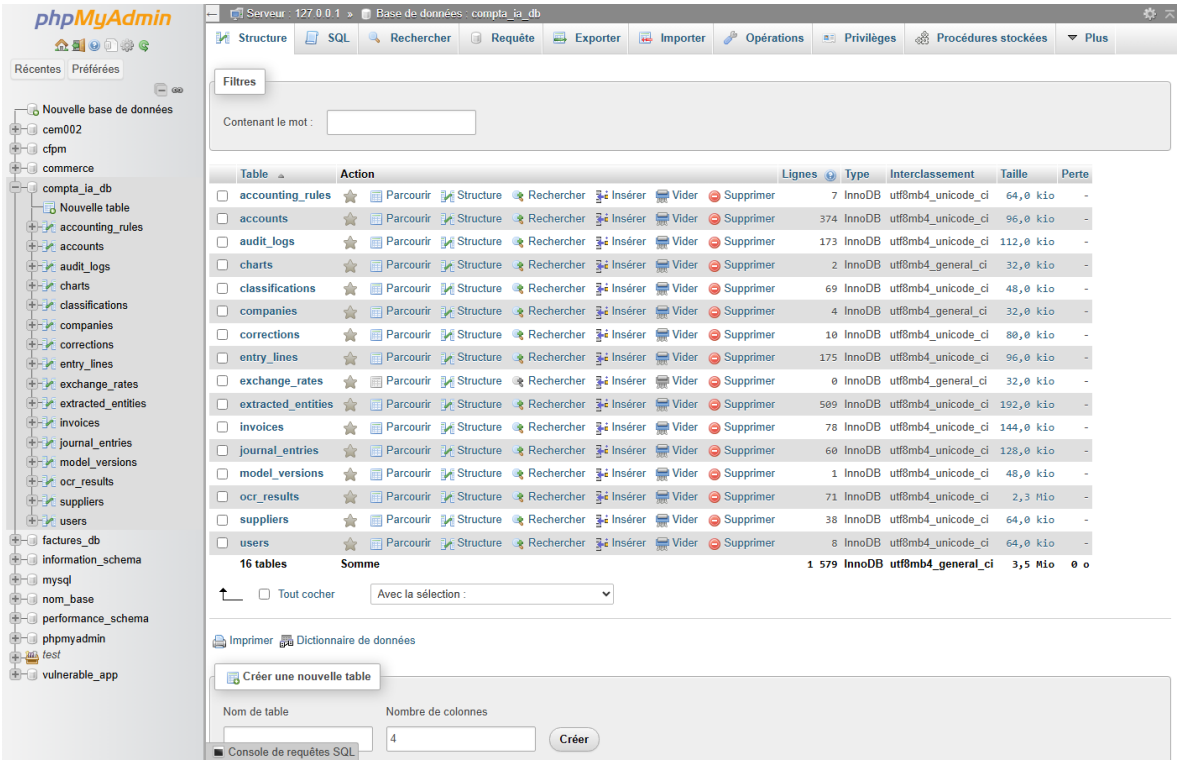


Figure A.1 : Liste des tables de la base compta_ia_db dans phpMyAdmin

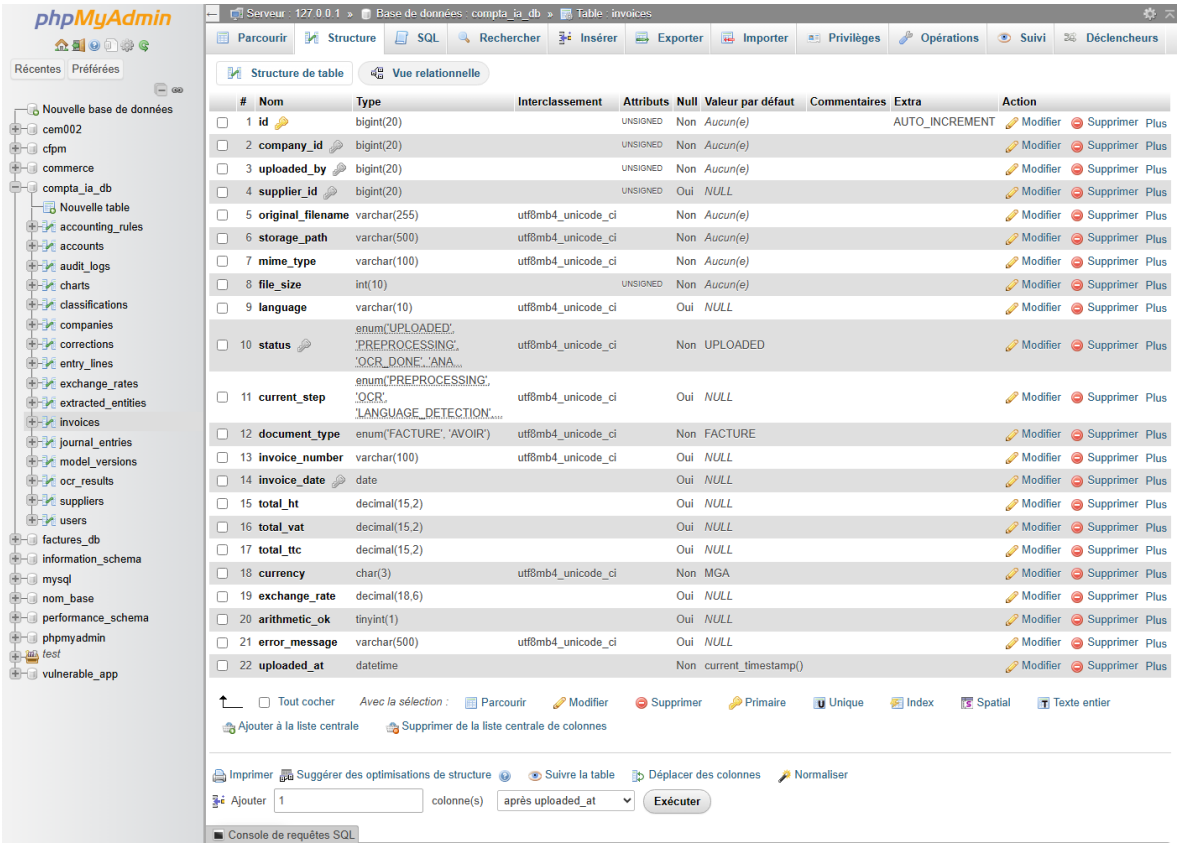


Figure A.2 : Structure de la table invoices

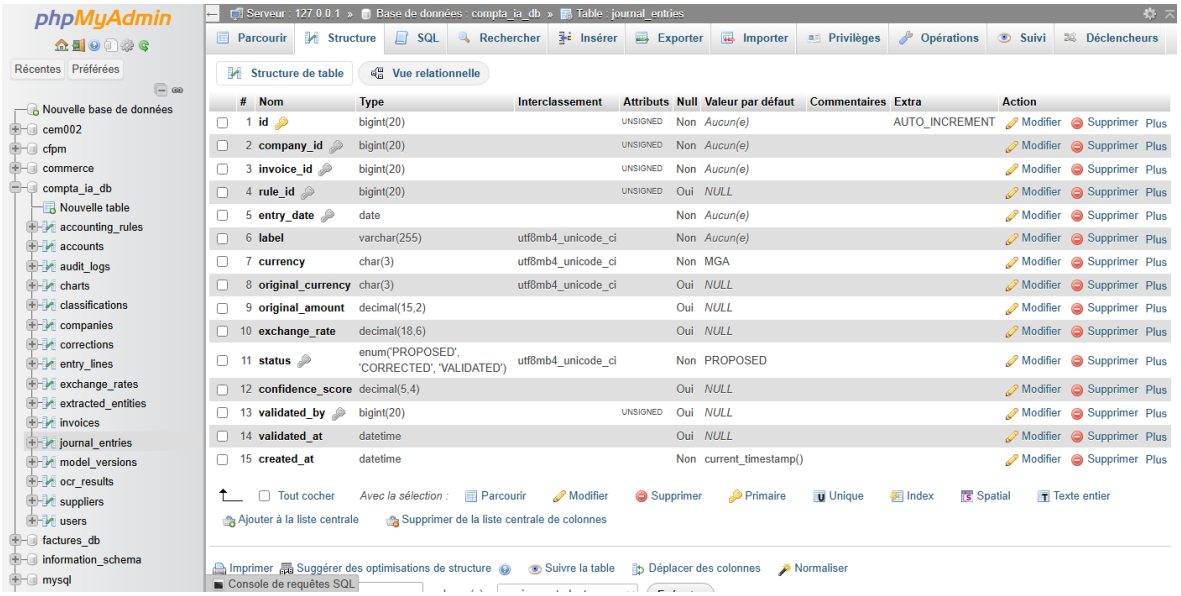


Figure A.3 : Structure de la table journal_entries

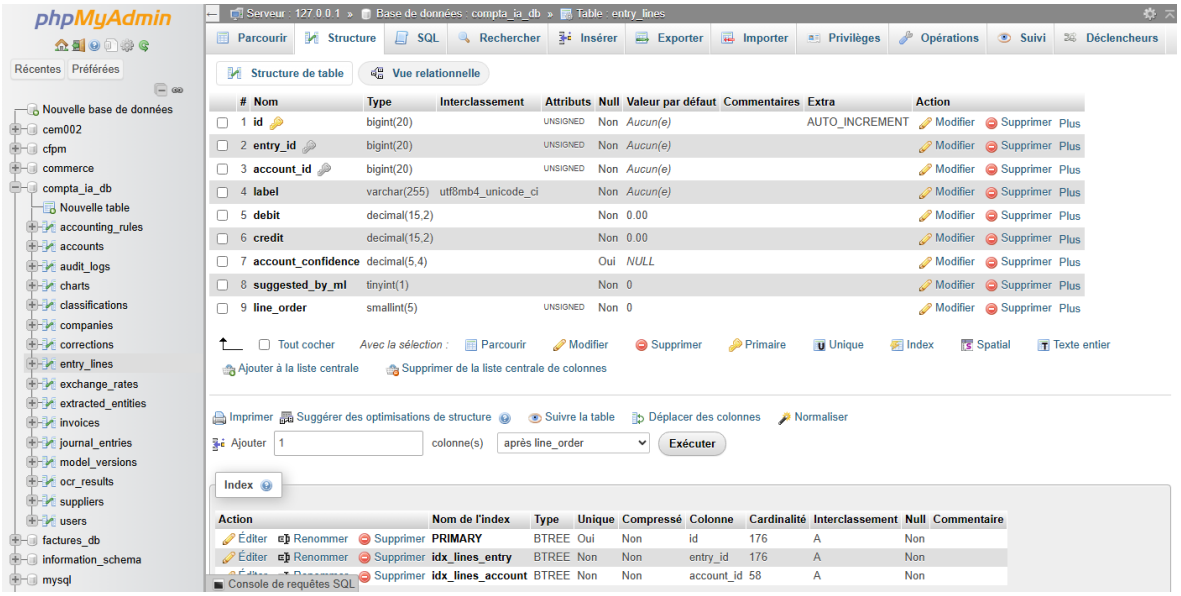


Figure A.4 : Structure de la table entry_lines

H. Modèles de factures utilisés pour les essais

Deux modèles de factures fictives illustrent l'éventail de documents que le système doit traiter : une facture simple, en ariary, sur une seule page, et une facture complexe, en euros, sur deux pages, avec des remises par ligne, plusieurs types de prix unitaires (à l'heure, au mois, forfait) et un bloc de coordonnées bancaires.

H.1 Facture simple

La figure A.5 présente une facture simple, établie en ariary sur une seule page. Elle comporte un fournisseur identifié par son NIF et son STAT, trois lignes d'articles (désignation, quantité, prix unitaire et montant hors taxes) et un récapitulatif des totaux HT, TVA à 20 % et TTC. Ce modèle correspond au cas le plus courant traité par le système.

SARL MADA BUREAUTIQUE
Lot IVG 52 Ankorondrano
101 Antananarivo, Madagascar
NIF : 4001234567 STAT : 47411 11 2019 0 10234
Tél : 034 12 345 67

FACTURE
N° FA-2026-0142
Date : 12/10/2026

Facturé à
Société Exemple SA
Immeuble Le Pavé, Analakely
101 Antananarivo
NIF : 3009876543

Conditions
Paiement : à 30 jours
Devise : MGA (Ariary)
Échéance : 11/11/2026

Désignation	Qté	Prix unitaire	Montant HT
Ordinateur portable 15 pouces	2	2 400 000	4 800 000
Imprimante laser multifonction	1	1 350 000	1 350 000
Ramettes de papier A4 (carton de 5)	4	62 500	250 000

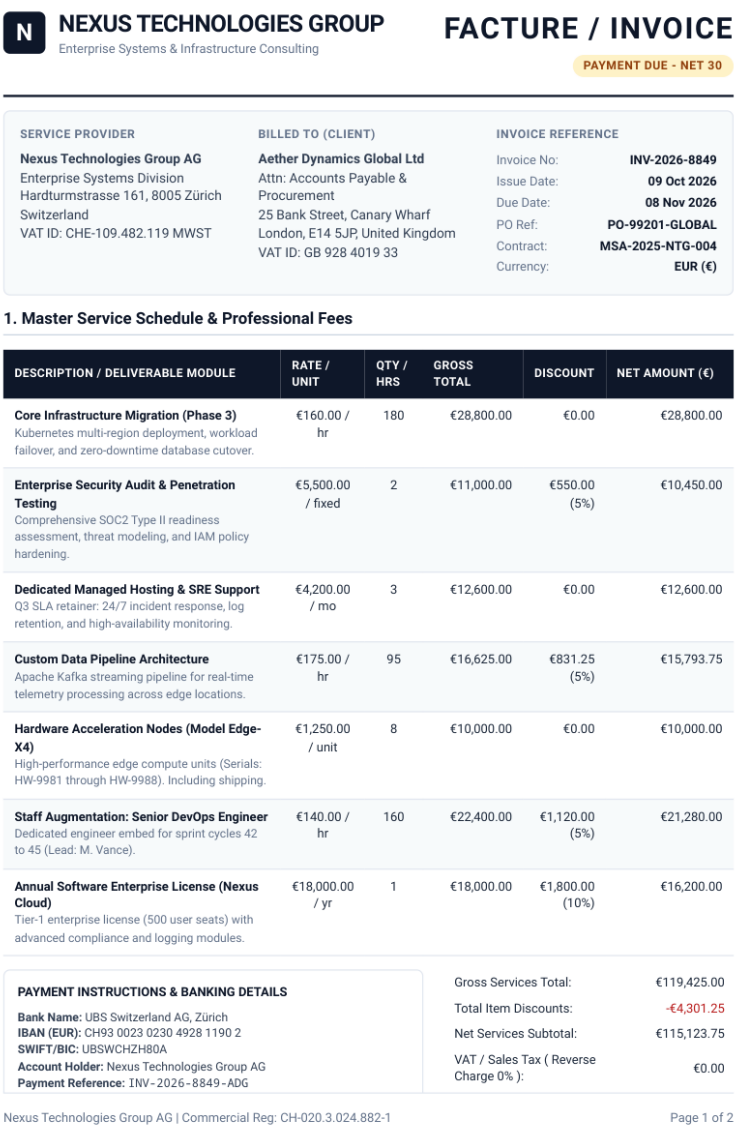
Total HT	6 400 000 Ar
TVA 20 %	1 280 000 Ar
Total TTC	**7 680 000 Ar**

Facture fictive établie à titre d'exemple. Arrêtée à la somme de sept millions six cent quatre-vingt mille Ariary.

Figure A.5 : Modèle de facture simple (une page, ariary)

H.2 Facture complexe

Les figures A.6 et A.7 présentent une facture complexe de deux pages, rédigée en anglais et libellée en euros. La première page contient sept lignes de prestations facturées selon des unités différentes (à l'heure, au forfait, au mois, à l'unité ou à l'année), des remises appliquées ligne par ligne, des coordonnées bancaires et un récapitulatif des totaux, avec une TVA en autoliquidation (0 %).



NEXUS TECHNOLOGIES GROUP
Enterprise Systems & Infrastructure Consulting

FACTURE / INVOICE

PAYMENT DUE - NET 30

SERVICE PROVIDER	BILLED TO (CLIENT)	INVOICE REFERENCE
Nexus Technologies Group AG Enterprise Systems Division Hardturmstrasse 161, 8005 Zürich Switzerland VAT ID: CHE-109.482.119 MWST	**Aether Dynamics Global Ltd** Attn: Accounts Payable & Procurement 25 Bank Street, Canary Wharf London, E14 5JP, United Kingdom VAT ID: GB 928 4019 33	Invoice No: **INV-2026-8849** Issue Date: **09 Oct 2026** Due Date: **08 Nov 2026** PO Ref: **PO-99201-GLOBAL** Contract: **MSA-2025-NTG-004** Currency: **EUR (€)**

1. Master Service Schedule & Professional Fees

DESCRIPTION / DELIVERABLE MODULE	RATE / UNIT	QTY / HRS	GROSS TOTAL	DISCOUNT	NET AMOUNT (€)
Core Infrastructure Migration (Phase 3) Kubernetes multi-region deployment, workload failover, and zero-downtime database cutover.	€160.00 / hr	180	€28,800.00	€0.00	€28,800.00
Enterprise Security Audit & Penetration Testing Comprehensive SOC2 Type II readiness assessment, threat modeling, and IAM policy hardening.	€5,500.00 / fixed	2	€11,000.00	€550.00 (5%)	€10,450.00
Dedicated Managed Hosting & SRE Support Q3 SLA retainer: 24/7 incident response, log retention, and high-availability monitoring.	€4,200.00 / mo	3	€12,600.00	€0.00	€12,600.00
Custom Data Pipeline Architecture Apache Kafka streaming pipeline for real-time telemetry processing across edge locations.	€175.00 / hr	95	€16,625.00	€831.25 (5%)	€15,793.75
Hardware Acceleration Nodes (Model Edge-X4) High-performance edge compute units (Serials: HW-9981 through HW-9988). Including shipping.	€1,250.00 / unit	8	€10,000.00	€0.00	€10,000.00
Staff Augmentation: Senior DevOps Engineer Dedicated engineer embed for sprint cycles 42 to 45 (Lead: M. Vance).	€140.00 / hr	160	€22,400.00	€1,120.00 (5%)	€21,280.00
Annual Software Enterprise License (Nexus Cloud) Tier-1 enterprise license (500 user seats) with advanced compliance and logging modules.	€18,000.00 / yr	1	€18,000.00	€1,800.00 (10%)	€16,200.00

PAYMENT INSTRUCTIONS & BANKING DETAILS

Bank Name: UBS Switzerland AG, Zürich
IBAN (EUR): CH93 0023 0230 4928 1190 2
SWIFT/BIC: UBSWCHZH80A
Account Holder: Nexus Technologies Group AG
Payment Reference: INV-2026-8849-ADG

Gross Services Total:	€119,425.00
Total Item Discounts:	-€4,301.25
Net Services Subtotal:	€115,123.75
VAT / Sales Tax (Reverse Charge 0%):	€0.00

Nexus Technologies Group AG | Commercial Reg: CH-020.3.024.882-1

Page 1 of 2

Figure A.6 : Modèle de facture complexe, page 1 (euros, remises par ligne)

La seconde page détaille l'échéancier de facturation par jalons, les frais remboursables (déplacements, hébergement, certification) et les conditions contractuelles. Cette facture sert à éprouver l'extraction sur un document de plusieurs pages, en langue étrangère et en devise étrangère.

2. Milestone Billing Schedule & Project Status

PROJECT DELTA MILESTONE COMPLETION SUMMARY

15 Aug 2026	15 Sep 2026	09 Oct 2026	30 Nov 2026
Milestone 1: Architecture	**Milestone 2: Security & QA**	**Milestone 3: Cutover**	**Milestone 4: Post-Handover**
COMPLETED (€25,000)	COMPLETED (€35,000)	BILLED HERE (€56,505)	PENDING (€20,000)

3. Reimbursable Expenses & Secondary Fees Breakdown

DATE	EXPENSE CATEGORY / DESCRIPTION	REFERENCE	AMOUNT (€)
12 Sep 2026	On-Site Deployment Travel: Flights Zurich - London (3 Engineers)	EXP-TRV-8821	€1,420.50
14 Sep 2026	Client Data Center On-Site Lodging & Subsistence (4 nights)	EXP-LOD-9012	€1,850.00
20 Sep 2026	Dedicated Fiber Test Connection & Temporary Bandwidth Provisioning	EXP-NET-3321	€680.00
28 Sep 2026	Third-party Hardware Compliance Certification (ISO/IEC 27001)	EXP-CRT-1049	€2,100.00
02 Oct 2026	High-Priority Encrypted Storage Express Courier Shipping	EXP-SHP-0042	€250.99

4. Enterprise Terms, Compliance & Regulatory Notes

1. Reverse Charge & Tax Compliance: Cross-border supply of services within the European Union / United Kingdom is subject to the Reverse Charge mechanism pursuant to EU Council Directive 2006/112/EC and UK VAT Act 1994. The recipient of services (Aether Dynamics Global Ltd) is required to account for VAT accordingly under VAT ID GB 928 4019 33.

2. Late Payment Surcharges: Payment terms are strict Net 30 days. Payments received after 08 November 2026 will incur a late interest penalty of 1.5% per month (or the maximum statutory rate permitted by Swiss Commercial Law), calculated daily from the due date until full settlement.

3. Governing Law & Dispute Resolution: This invoice is issued under Master Services Agreement MSA-2025-NTG-004. Any disputes or claims arising out of or in connection with this invoice shall be governed by and construed in accordance with the substantive laws of Switzerland, with jurisdiction in Zurich.

Authorized Officer: Dr. Aris Thorne, Managing Director
Nexus Technologies Group AG - Finance & Billing Dept.

Electronically Approved & Signed
REF: SIG-2026-90218-NTG

Figure A.7 : Modèle de facture complexe, page 2

TABLE DES MATIÈRES

CURRICULUM VITAE i
SOMMAIRE iii
REMERCIEMENTS v
LISTE DES FIGURES vi
LISTE DES TABLEAUX vii
LISTE DES ABRÉVIATIONS viii
INTRODUCTION GÉNÉRALE 1
PARTIE I. PRÉSENTATIONS 3
CHAPITRE 1 Présentation de l'Ecole Nationale d'Informatique 3
Information d'ordre générale 3
Missions et historiques 3
Organigramme institutionnel 5
Domaine de spécialisation 6
Architecture des formations pédagogiques 7
Relation de l'ENI avec les organismes externes 8
Débouchés professionnels et diplômés 9
Ressources humaines 11
CHAPITRE 2 : Présentation de l'établissement d'accueil 12
2.1 Présentation générale du groupe Manao 12
2.2 Historique 12
2.3 Philosophie et valeurs 13
2.4 Domaines d'expertise 13
2.5 Branches et activités 14
2.6 Implantations 14
2.7 La plateforme logicielle Manao 14
2.8 Organisation et organigramme 15
2.9 Place du stagiaire et lien avec le projet 16
CHAPITRE 3 : Description du projet 17
3.1 Formulation : 17
3.2 Objectif et besoin de l'utilisateur : 18
3.2.1 Objectif : 18
3.2.2 Besoin de l'utilisateur : 19
3.3 Moyens nécessaires à la réalisation du projet : 19
3.3.1 Moyens humains : 19
3.3.2 Moyens matériels : 20

3.3.3 Moyens logiciels : 20
3.3.4 Langages, bibliothèques et composants : 21
3.4 Résultats Attendus 22
3.4.1 Extraction et compréhension des factures 22
3.4.2 Génération automatique des écritures comptables 22
3.4.3 Contrôle et validation par le comptable 22
3.4.4 Gain de temps et réduction des erreurs 22
3.4.5 Traçabilité et sécurité 23
3.4.6 Architecture évolutive 23
3.5 Chronogramme : 23
PARTIE II. ANALYSE ET CONCEPTION 25
CHAPITRE 4 : Analyse préalable 25
4.1 Analyse de l'existant 25
4.1.1 Organisation actuelle 25
4.1.2 Inventaire des moyens matériels et logiciels 30
4.2 Critique de l'existant 32
4.2.1 Points forts 32
4.2.2 Points faibles 32
4.2.3 Temps 32
4.2.4 Risque d'erreurs 32
4.2.5 Difficulté d'interprétation des factures 33
4.2.6 Risque de mauvaise imputation comptable 33
4.2.7 Dépendance à l'expertise humaine 34
4.2.8 Limites de la traçabilité 34
4.3 Conception avant-projet 34
4.3.1 Proposition des solutions 34
4.3.2 Méthodes de conception et outils utilisés 35
CHAPITRE 5 : Analyse conceptuelle 37
5.1 Désignation des rôles de l'équipe SCRUM 37
5.1.1 Rôle de Product Owner 37
5.1.2 Rôle de Scrum Master 37
5.1.3 Rôle de développeur 37
5.2 Étapes de l'élaboration du Product Backlog 38
5.3 Product Backlog 39
5.4 Sprint Backlog 40
5.4.1 Présentation du travail à faire 40

5.4.2 Planification du sprint 41
5.4.3 Points forts du workflow 41
5.5 Dictionnaire des données 42
5.6 Règles de gestion 43
5.7 Diagramme de cas d'utilisation global 44
5.8 Modèle du domaine : 46
5.9 Diagramme de classe de conception pour chaque Sprint : 48
5.9.1 Diagramme de classes de conception du sprint 1 49
5.9.2 Diagramme de classes de conception du sprint 2 49
5.9.3 Diagramme de classes de conception du sprint 3 50
5.9.4 Diagramme de classes de conception du sprint 4 51
5.9.5 Diagramme de classes de conception du sprint 5 52
5.9.6 Diagramme de classes de conception du sprint 6 53
5.9.7 Diagramme de classes de conception du sprint 7 54
5.10 Diagramme de classe de conception globale : 55
5.11 Diagramme de paquetage : 58
5.12 Diagramme de déploiement : 58
CHAPITRE 6 : Intégration IA dans la conception 61
6.1 Définition du module IA 61
6.2 Données nécessaires au fonctionnement du module IA 61
6.2.1 Données provenant des factures 61
6.2.2 Données comptables 62
6.2.3 Données d'apprentissage 62
6.3 Choix du modèle IA et justification 63
6.4 Méthode d'intégration de l'IA 64
PARTIE III. RÉALISATION 68
CHAPITRE 7 : Mise en place de l'environnement de développement 68
7.1 Installation et configuration des outils 68
7.1.1 Environnement backend 68
7.1.2 Environnement frontend 68
7.1.3 Base de données et outillage complémentaire 69
7.2 Architecture de l'application 69
7.3 Déploiement des environnements (développement, test, production) 70
7.3.1 Environnement de développement 70
7.3.2 Environnement de test 71
7.3.3 Environnement de production 71

CHAPITRE 8 : Développement de l'application 73
8.1 Création et structuration de la base de données 73
8.2 Développement du frontend 73
8.2.1 Interface utilisateur 74
8.3 Développement du backend 76
8.4 Développement et entraînement du module IA 77
8.4.1 OCR 77
8.4.2 NLP 77
8.4.3 Classification et suggestion de comptes 78
8.5 Intégration IA–application 78
CHAPITRE 9 : Tests et validation 80
9.1 Tests unitaires et d'intégration 80
9.2 Validation fonctionnelle 80
9.3 Évaluation de la performance du module IA 82
CONCLUSION 83
BIBLIOGRAPHIE x
WEBOGRAPHIE xi
GLOSSAIRE xii
ANNEXE xiv

RÉSUMÉ

Dans une entreprise, le traitement des factures fournisseurs reste aujourd'hui, dans la plupart des cas, un travail manuel : le comptable lit chaque facture, en extrait les informations utiles, puis choisit lui-même les comptes à utiliser selon le plan comptable en vigueur à Madagascar. Cette tâche répétitive prend du temps et expose à des erreurs de saisie ou des oublis, en particulier lorsque le volume de factures augmente.

Ce mémoire présente une application web conçue pour alléger ce travail. Lorsqu'une facture est importée (photographie ou fichier PDF), le système la lit automatiquement grâce à une technologie de reconnaissance de texte, puis repère les informations importantes : nom du fournisseur, montants, taxes, numéro de facture. Il détermine ensuite la nature de l'opération et propose une écriture comptable complète, accompagnée d'un indice de confiance qui indique au comptable à quel point la proposition mérite d'être vérifiée attentivement. Le comptable garde à chaque étape la maîtrise de la décision : il peut valider, corriger ou rejeter la proposition, et chacune de ses corrections aide le système à s'améliorer progressivement.

L'application, dotée d'une interface simple et d'une base de données sécurisée, a été testée à travers plus de 100 vérifications automatisées couvrant l'ensemble de ses fonctions. Les résultats montrent que l'outil fonctionne de manière fiable et qu'il peut réellement faire gagner du temps au comptable, tout en le laissant décideur final de chaque écriture. L'application a été mise en ligne sur un serveur distant et s'utilise depuis un simple navigateur, en connexion chiffrée. Les prochaines étapes consistent à l'éprouver sur un plus grand nombre de factures réelles et à automatiser davantage ses mises à jour.

Mots-clés : application web, automatisation comptable, intelligence artificielle, plan comptable général malgache, reconnaissance de texte.

ABSTRACT

In most companies, processing supplier invoices is still largely a manual task: the accountant reads each invoice, extracts the relevant information, and then chooses the accounts to use according to the chart of accounts in force in Madagascar. This repetitive work takes time and is prone to input errors or omissions, especially as the volume of invoices grows.

This dissertation presents a web application designed to ease this work. When an invoice is imported (a photograph or a PDF file), the system reads it automatically using text-recognition technology, then identifies the key information: supplier name, amounts, taxes, invoice number. It then determines the nature of the operation and proposes a complete accounting entry, together with a confidence indicator showing the accountant how carefully the proposal should be checked. The accountant remains in control at every step: they can validate, correct or reject the proposal, and each correction helps the system improve over time.

The application, built with a simple interface and a secure database, was tested through more than 100 automated checks covering all of its functions. The results show that the tool works reliably and can genuinely save the accountant time, while keeping them as the final decision-maker for every entry. The application has been deployed on a remote server and is used from an ordinary web browser over an encrypted connection. Next steps include trialling it on a larger set of real invoices and further automating its updates.

Keywords: accounting automation, artificial intelligence, Malagasy chart of accounts, text recognition, web application.